Autonomous Search and Rescue Drone with Onboard AI-Based Casualty Detection

Dmytro Chuprynyuk, *et al.*

AENGM0074 Group Design Project — University of Bristol — March 2026

Executive Summary

An autonomous search and rescue (SAR) drone was designed, implemented, and tested for the AENGM0074 group project. The system locates a lost hiker within the designated search area at Fenswood Farm, subject to geofencing constraints including a Site of Special Scientific Interest (SSSI) no-fly zone, a 50 m altitude ceiling, and defined flight boundaries. The project satisfies all twelve brief requirements (R01–R12), with automatic geofencing, operator-in-the-loop verification, and a public MIT-licensed GitHub repository.

Mission objective. The drone autonomously takes off, searches a polygon-defined area using a lawnmower pattern, detects a casualty via onboard computer vision, reports the target's GPS coordinates and imagery to a ground operator for confirmation, lands within 10 m of the casualty to deploy a first-aid kit, and returns to the take-off location. An operator-in-the-loop verification stage prevents autonomous action on false positives.

System overview. The platform is a Hexsoon EDU-450 hexacopter controlled by a Cube Orange+ autopilot running ArduPilot. A Raspberry Pi 5 companion computer runs all onboard intelligence: a YOLOv8n object-detection model exported to TensorFlow Lite, a lawnmower search-pattern generator, GPS-based target localisation with inverse-variance-weighted spatial clustering, and a finite state machine with **20 states** and 32 transitions that orchestrates the complete mission. A browser-based ground station streams live MJPEG video and provides per-target classification controls (confirm, reject, mark as interest, mark as false positive) over Wi-Fi. Four MCDA trade studies informed the selection of detection model, search pattern, companion computer, and ground-station architecture.

Key design decisions. Three architectural choices define the system. First, a *modular single-codebase* design isolates computer vision, path planning, flight control, and configuration into independent modules with narrow interfaces, allowing identical code to run across laptop simulation, bench hardware, and the Pi without modification. Second, a *dual-backend vision pipeline* automatically selects between Ultralytics/PyTorch on the development laptop and TFLite on the Pi, enabling rapid iteration without deployment friction. Third, a *simulation-first development methodology* validated each subsystem through a **five-tier progressive testing framework**—SITL simulation, hardware component checks, bench integration, passive field observation, and autonomous flight—with **58 test scripts** organised across six categories, catching interface and timing errors before integration.

Results achieved. The detection model, retrained on 366 mixed synthetic and real-world images at native camera resolution (1456×1088), achieves $\mathbf{mAP}_{50} = 0.995$. On-device inference runs at **206 ms** per frame (**4.8 FPS**) on the Pi 5 CPU via the XNNPACK delegate, sufficient for the altitude-dependent search speed (8 m/s at the nominal 35 m altitude). Lens distortion correction adds only 1.5 ms overhead. Post-hoc analysis of DJI flight video over the test site confirmed detection across a 15–50 m altitude range and GPS target localisation with a circular error probable $\mathbf{CEP}_{50} = 2.3\,\mathbf{m}$. The lawnmower pattern covers the survey polygon in approximately 1.1 minutes with 18 waypoints, verified via dry-run to avoid SSSI encroachment. The report contains **14 figures** presenting system architecture, state diagrams, detection results, and field calibration data.

Demonstration and weather adaptation. Full hardware integration—Pi, Cube, camera, and GPS—was validated on the assembled airframe during a scheduled field day. Adverse weather prevented powered flight; the team adapted by executing the full bench-testing protocol on-site, producing lens calibration data (RMS = 0.399), FOV characterisation (5.46 mm focal length), and inference benchmarks confirming 100% detection at 0.966 mean confidence across 50 consecutive frames. This weather-adapted session yielded quantitative calibration results that directly improved the vision pipeline for subsequent flights. The complete mission pipeline, from arming through search, detection, operator verification, and landing, was exercised end-to-end in simulation. The progressive testing methodology, operator-in-the-loop architecture, and automatic geofencing together demonstrate that effective autonomous SAR capability is achievable with consumer-grade hardware and open-source software.

Introduction

Context and Background

A hiker has gone missing in Fenswood Wilderness, a semi-rural landscape of mixed grassland and scrub managed by the University of Bristol for field research. Search and rescue teams have been alerted, but the terrain is uneven, visibility is limited by vegetation, and the search area spans several hectares. At some point during the search the hiker activates a Personal Locator Beacon (PLB), narrowing the probable location to a Focus Area—but a casualty on the ground, potentially incapacitated or partially concealed, still requires visual confirmation before help can reach them.

This is the scenario posed by the AENGM0074 module. Each student company is provided with identical hardware—a Hexsoon EDU-450 hexacopter airframe, a Cube Orange+ flight controller, a Raspberry Pi 5 companion computer, and a global-shutter camera—and tasked with building an autonomous unmanned aerial vehicle (UAV) capable of:

- systematically searching a defined area while respecting a no-fly zone over a Site of Special Scientific Interest (SSSI);
- detecting a life-size casualty surrogate (dummy) using onboard computer vision;
- reporting the casualty's estimated GPS coordinates and imagery to a ground operator;
- landing within 10 m of the casualty (but no closer than 5 m) and deploying a simulated first-aid kit; and
- returning to the designated take-off location.

The scenario is deliberately constrained to a single outdoor demonstration flight, preceded by progressive bench and flight testing, reflecting the compressed timeline of a single-term MSc module. All software must be released publicly under the MIT licence, and the system must include automatic geofencing to enforce flight-area and SSSI boundaries without relying on pilot intervention. The provided platform is a Hexsoon EDU-450 hexacopter airframe equipped with a Cube Orange+ flight controller, Here 3+ GPS, Raspberry Pi 5 companion computer, and an IMX296 global-shutter camera.

Company Description

Our company was formed at the start of the AENGM0074 module as a team of five MSc students from the Department of Aerospace Engineering. We adopted an agile, workstream-based methodology: five parallel streams—Computer Vision, Hardware Integration, Search Logic, Ground Station, and Safety/Testing—fed into defined integration milestones where subsystems were combined and verified before advancing to the next stage. All development was managed through a shared GitHub repository[1], with a single protected main branch and feature branches for individual work.

Day-to-day coordination relied on a group chat for informal updates and weekly in-person meetings for design decisions, integration planning, and blocker resolution. A shared status tracker documented every group decision, assigned action items with owners and deadlines, and maintained a running log of blockers and dependencies. This lightweight process allowed team members to work independently on their respective streams while ensuring that integration points were well-defined and that no subsystem was developed in isolation from the others.

Team Members and Roles

Dmytro Chuprynyuk — Lead Software Developer. MSc student in Robotics with a background in software engineering and embedded systems. Dmytro served as the lead software developer for the project, designing and implementing the entire autonomous software stack: the finite state machine, computer vision pipeline (dual-backend YOLOv8n inference on both laptop and Raspberry Pi), lawnmower path planner, GPS-based target localisation, web-based ground station, and the software-in-the-loop simulation environment. He also built the model training pipeline, including synthetic dataset generation and retraining on real flight imagery, and developed the progressive testing framework used to validate the system from simulation through to outdoor flight.

[NAME] — **Flight Dynamics and State Machine Testing.** MSc student in Aerospace Engineering. Contributed to the flight dynamics analysis and testing of the autonomous state machine, verifying state transitions and flight parameter tuning in both simulation and bench testing. Supported integration of the Cube Orange+ flight controller with the mission software and assisted with flight mode configuration (GUIDED, LOITER, STABILIZE, LAND).

[NAME] — **Designated Pilot and RC Systems.** MSc student in Aerospace Engineering with prior experience in radio-controlled aircraft. Served as the company's designated pilot, responsible for RC transmitter configuration, kill-switch setup, and manual override procedures. Maintained readiness to assume manual control during all autonomous flight tests, ensuring compliance with the requirement that a pilot with override authority be present at all times.

[NAME] — **Hardware Integration.** MSc student in Mechanical Engineering. Led the physical assembly of the drone platform, including airframe construction, wiring of the Cube Orange+ to the Raspberry Pi (serial TELEM2 connection), camera mounting, GPS antenna placement, and compass calibration. Ensured that all electrical connections were secure

[1] Repository: https://github.com/DimaChup/Group_Proj (MIT licence, R12).

and that the payload (Pi, camera, first-aid release mechanism) was mounted within the airframe's centre-of-gravity envelope.

[NAME] — **Project Management and Documentation.** MSc student in Aerospace Engineering. Coordinated project scheduling, booked flight slots at the Fenswood site, managed risk assessment paperwork, and maintained deliverable tracking against the module timeline. Led the report writing effort, compiled the requirements verification evidence, and ensured that all deliverables (D1–D6) were submitted on time.

Contribution Summary

Table 1 summarises the key contributions of each team member across the major project areas.

Table 1: Team member contributions by project area.

Team Member	Key Contributions
Dmytro Chuprynyuk	Autonomous software stack (state machine, CV pipeline, path planner, target localisation, ground station, simulation environment). Custom YOLOv8n model training and dataset generation. Progressive testing framework. Pi hardware integration (camera, TFLite inference, Cube serial link). Lens calibration and FOV characterisation.
[NAME]	Flight dynamics analysis. State machine testing in simulation and on bench. Flight parameter tuning (altitude, speed, descent profile). Cube flight-mode configuration and verification.
[NAME]	RC transmitter configuration and binding. Kill-switch and flight-mode switch setup. Manual override procedures. Designated pilot for all flight tests.
[NAME]	Airframe assembly and wiring. Cube–Pi serial connection. Camera and GPS antenna mounting. Compass calibration. Payload integration and CG verification.
[NAME]	Project scheduling and milestone tracking. Flight-slot booking and risk assessments. Report compilation and editing. Requirements verification evidence. Deliverable management (D1–D6).

Report Structure

The remainder of this report is organised to address all requirements of the D6 brief. Section 1 presents the *Design Rationale*, including STEEPLE analysis and four MCDA trade studies. Sections 2–2.7 provide the *System Description*: hardware platform, computer vision pipeline, path planning, state machine, target localisation, and ground station. Section E covers the progressive testing methodology and simulation environment. Section 3 presents *Requirements Verification* with evidence of compliance for R01–R12. Section 4 provides a plus/delta *Evaluation* of technical performance. References and appendices follow, including the full state-transition table, detection samples, and calibration data.

1 Design Rationale

Every major design decision was evaluated against the seven STEEPLE dimensions (Social, Technological, Economic, Environmental, Political, Legal, Ethical) before proceeding to technical trade studies for hardware, software architecture, detection, path planning, and mission control.

1.1 STEEPLE Analysis

Social.
SAR operations directly serve community safety. The system is designed for volunteer mountain rescue teams who lack the budget for commercial platforms. An operator-in-the-loop verification stage ensures that autonomous decisions do not bypass human judgement in life-critical situations, maintaining public trust in drone-assisted SAR [1].

Technological.
The Raspberry Pi 5 was selected as the onboard computer for its sub-£100 cost, 4.8 FPS inference capability with YOLOv8n/TFLite, native CSI camera interface, and extensive community support. TFLite was chosen over Ultralytics/PyTorch for deployment because it requires only 4 Python packages versus 90+, reducing attack surface and deployment complexity. The dual-backend architecture (Ultralytics on laptop, TFLite on Pi) allows development iteration with GPU acceleration while deploying the same model on edge hardware [2, 3].

Economic.
The complete system costs £565 (Table 10), approximately one-tenth the price of a DJI Matrice 30T (£5 800). By performing inference onboard rather than streaming video to a ground station, the system eliminates the need for high-bandwidth communication equipment and a dedicated image analyst. This cost structure makes the technology accessible to volunteer rescue organisations operating on limited budgets.

Environmental.
The adjacent SSSI imposes strict no-fly constraints. A three-layer geofence—speed capping, repulsive potential field, and hard-boundary cutoff—enforces the exclusion zone in software, with firmware geofencing as a fourth independent layer. At plan time, waypoints within 30 m of the boundary are filtered to prevent the search pattern from routing the aircraft near the protected area. The lightweight airframe (3 kg AUW) minimises noise disturbance to wildlife [4].

Political.
All flights comply with UK CAA Open Category A3 regulations [5]. The system architecture supports BVLOS operation but was tested within VLOS range, as BVLOS authorisation requires a Specific Category assessment under SORA [6]. The open-source MIT licence (R12) ensures transparency and reproducibility.

Legal.
Data protection is considered: detection images are stored locally on the Pi and not transmitted to cloud services. The SSSI no-fly zone is a legal requirement under the Wildlife and Countryside Act 1981 [4]. The geofence implementation provides auditable evidence of compliance through logged distance-to-boundary measurements at every control loop iteration.

Ethical.
The operator-in-the-loop design ensures that no autonomous landing is initiated without explicit human confirmation, addressing the ethical concern of autonomous systems making high-consequence decisions. The system prioritises continued area coverage over investigating uncertain detections (auto-reject after 120 s timeout), reflecting the ethical principle that searching for a casualty is more important than investigating a probable false positive.

1.2 Hardware Platform Selection

The hardware integrates a Hexsoon EDU-450 airframe, CubePilot Orange+ flight controller with ArduPilot firmware, Raspberry Pi 5 companion computer, and IMX296 global shutter camera.

- **Global shutter camera.** The IMX296 was selected over cheaper rolling-shutter alternatives because rolling-shutter distortion during forward flight at up to $10\,\mathrm{m\,s^{-1}}$ skews bounding boxes and degrades detection accuracy. The CSI interface leaves USB ports free for telemetry radios.
- **No thermal imaging.** Thermal cameras add £2 000–£10 000 to the platform cost and have lower spatial resolution than RGB at equivalent price points. This project deliberately explores RGB-only detection feasibility at the low-cost extreme [7].
- **MAVProxy bridge.** A Python 3.13 incompatibility with pyserial (dropped bytes causing `BAD_DATA` errors) is resolved by interposing MAVProxy as a UDP bridge, which reads the serial port reliably using its own C-level handler [8].

1.2.1 Companion Computer Trade Study

Three candidate companion computers were evaluated using weighted multi-criteria decision analysis (Table 2). Criteria weights reflect the project's priorities: cost and Python ecosystem compatibility are paramount for a student team,

while raw GPU performance is less critical given the lightweight YOLOv8n model.

Table 2: MCDA: companion computer selection (1–5 scale, 5 = best)

Criterion	Weight	Pi 5	Jetson Nano	Intel NCS2
Unit cost (£)	0.20	5 (£75)	3 (£180)	4 (£100+host)
Power consumption	0.10	4 (5–12 W)	3 (5–20 W)	3 (host+1.5 W)
GPIO / CSI interface	0.15	5 (40-pin+CSI)	4 (40-pin+CSI)	1 (USB only)
Community & documentation	0.20	5 (largest)	4 (strong)	2 (discontinued)
Python 3.13 compatibility	0.20	5 (native)	3 (JetPack lag)	2 (OpenVINO lag)
TFLite / edge-AI support	0.15	5 (XNNPACK)	4 (TensorRT)	2 (OpenVINO only)
Weighted total		**4.90**	3.50	2.35

The Raspberry Pi 5 scores highest across all criteria. The Jetson Nano offers superior GPU inference but is more expensive, draws more power, and lags behind on Python version support due to NVIDIA's JetPack release cycle. The Intel Neural Compute Stick 2 was discontinued by Intel in 2023 and lacks GPIO or CSI interfaces, requiring a separate host SBC.

1.2.2 Communication Architecture Trade Study

Three architectures for companion-to-autopilot communication were compared (Table 4).

Table 4: MCDA: communication architecture (1–5 scale, 5 = best)

Criterion	Weight	Direct Serial	MAVProxy Bridge	ROS 2
Latency	0.20	5 (direct)	4 (UDP hop)	3 (DDS overhead)
Reliability	0.25	2 (py3.13 bug)	5 (C-level read)	4 (mature)
Multi-consumer support	0.20	1 (exclusive)	5 (N outputs)	5 (pub/sub)
Python 3.13 compatibility	0.20	1 (broken)	5 (works)	3 (partial)
Implementation complexity	0.15	5 (trivial)	4 (one process)	1 (full stack)
Weighted total		2.65	**4.65**	3.35

Direct serial is the simplest option but is rendered non-viable by a Python 3.13 pyserial regression that drops bytes at 921 600 baud. MAVProxy resolves this by reading the serial port with its own C-level handler and forwarding via UDP, simultaneously enabling multi-consumer access (scripts on port 14550, Mission Planner on TCP 5762). ROS 2 offers the richest middleware but introduces substantial deployment complexity disproportionate to the system's single-node architecture.

1.3 Software Architecture Rationale

Four principles govern the architecture, enabling parallel development by a five-person team and supporting the simulation-first methodology:

1. **Modular separation.** Each concern—computer vision, path planning, flight control, configuration—is isolated in a dedicated module with a narrow public interface. The vision system exposes a single function `detect_in_image(frame)` returning `(found, x, y, conf)`, regardless of the inference backend.
2. **Same code, simulation and real.** The mission orchestrator (`main.py`) contains no conditional branches distinguishing simulation from hardware. A `MODE` flag selects the camera source; connection auto-detection selects the flight controller endpoint.
3. **Configuration-driven tuning.** All mission parameters reside in a single configuration module (`config.py`). Transitioning from simulation to outdoor flight requires only parameter adjustment, not code modification.
4. **Dual inference backend.** The vision module auto-detects the available engine at import time: Ultralytics on the laptop, TFLite on the Pi. Both produce identical output tensors of shape $[1, 5, 8400]$.

1.4 Detection Model Selection

YOLOv8n (the nano variant, 3.2 million parameters, 8.7 GFLOPs) was selected after benchmarking the YOLO model family on the target hardware [9]. Table 6 summarises the weighted comparison.

Table 6: MCDA: detection model selection (1–5 scale, 5 = best)

Criterion	Weight	YOLOv8n	YOLOv8s	YOLOv8m	SSD MobileNet
Inference speed (Pi 5 CPU)	0.30	5 (207 ms)	3 (450 ms)	1 (820 ms)	5 (180 ms)
Detection accuracy (mAP50)	0.25	4 (0.995)	5 (0.997)	5 (0.998)	3 (0.88)
Model size (TFLite)	0.15	5 (3.2 MB)	3 (22 MB)	2 (50 MB)	5 (4.3 MB)
TFLite export support	0.15	5 (native)	5 (native)	5 (native)	4 (manual)
Training data requirement	0.15	5 (small)	4 (moderate)	3 (large)	4 (moderate)
Weighted total		**4.75**	3.95	3.05	4.20

YOLOv8n is the only variant that achieves real-time inference on the Pi 5 CPU within the 300 ms-per-frame budget required for useful detection at search speed. Larger variants (YOLOv8s at 11.2 M parameters, YOLOv8m at 25.9 M) exceeded this budget by factors of two and four respectively. SSD MobileNetV2 matches YOLOv8n on speed but delivers substantially lower accuracy on the custom SAR dataset due to its simpler feature pyramid [2].

The model was trained as a single-class detector on 366 mixed images (300 synthetic, 16 real, 50 negatives) at native camera resolution (1456×1088), following the domain-randomisation principle [10]. Training at higher resolution than the 640×640 inference input allows the network to learn fine-grained features that survive downscaling [9]. A COCO person detector is carried as a fallback model for field conditions where the custom model underperforms.

1.5 Coverage Path Planning

Four candidate search patterns were evaluated (Table 8). Criteria weights emphasise coverage guarantee and implementability, reflecting the project's emphasis on reliable, verifiable behaviour over theoretically optimal paths.

Table 8: MCDA: search pattern selection (1–5 scale, 5 = best)

Criterion	Weight	Lawnmower	Spiral	Expanding Sq.	Random
Coverage guarantee	0.30	5 (complete)	3 (centre gap)	4 (approx.)	1 (none)
Path length efficiency	0.20	4 (minimal turns)	3 (long spiral)	3 (many turns)	2 (redundant)
Implementability	0.20	5 (analytical)	3 (radius fn)	4 (simple)	5 (trivial)
Wind robustness	0.15	4 (long legs)	3 (curved)	3 (short legs)	2 (unpredictable)
SAR standard compliance	0.15	5 (IAMSAR)	3 (maritime)	4 (land SAR)	1 (none)
Weighted total		**4.65**	3.00	3.65	2.15

The boustrophedon (lawnmower) pattern was selected for three reasons [11, 12]: (1) it guarantees complete coverage of any simply connected region by construction; (2) it admits closed-form path length calculation for endurance estimation; (3) it is the standard method recommended by the IAMSAR Manual. The expanding square pattern scored second but lacks a formal completeness guarantee for arbitrary polygons. Spiral patterns leave coverage gaps at the centre and require continuous curvature that complicates waypoint-based navigation. The implementation uses a rotate–rasterise–rotate-back algorithm that handles arbitrary polygon geometries through binary mask operations rather than analytical clipping. The scan angle is optimised automatically using the minimum-area bounding rectangle, validated by a 216-configuration parametric sweep (6 altitudes $\times$ 36 angles).

1.6 State Machine Design

A deterministic finite state machine (FSM) with 20 states and 32 transitions was chosen over behaviour trees or reactive architectures for its verifiability, traceability, and ease of runtime monitoring [13]. Each state maps to a single handler function via a dictionary-based dispatch table ($O(1)$ lookup), eliminating fall-through errors. Defence-in-depth safety is enforced through five independent mechanisms: manual override (M key), RC kill switch (hardware-level), return-to-launch on link loss, three-layer geofence, and per-state timeouts. Every blocking state carries a wall-clock timeout to prevent indefinite hangs. The complete transition table is provided in Appendix A.

1.7 Autonomy Level Justification

The system operates at Sheridan Level 5 ("computer decides, human can veto") [14] for the search phase and Level 3 ("computer suggests, human approves") for the landing decision. This hybrid was chosen after analysing the asymmetric cost structure of SAR false positives versus false negatives [15, 16].

A *false positive landing* (descending on a rock, tree stump, or discarded clothing) incurs three compounding penalties: (1) battery endurance consumed by descent, hover, and re-climb (≈90 s, roughly 8% of a 20-minute endurance budget); (2) search coverage paused during the entire descent–verify–climb cycle; (3) risk of mechanical damage if the landing site is uneven. In a single-drone mission with no redundancy, even one unnecessary landing significantly reduces the probability of covering the full search area before battery exhaustion.

A *false negative* (failing to investigate a real casualty) is worse in absolute terms but is mitigated by the lawnmower pattern's guarantee of revisiting every cell: a missed detection on pass 1 can be recovered on the return leg. The operator-in-the-loop gate therefore addresses the higher-consequence, lower-recoverable error (committing to a landing) while the autonomous search handles the high-bandwidth, lower-consequence task (scanning terrain at 8 m/s).

The VERIFY state implements this gate. The operator classifies each detection as confirmed (Y), rejected (N), or item of interest (I) through the browser-based ground station. A 120 s timeout auto-rejects to prevent indefinite hovering [17]. This supervisory control paradigm lets autonomy handle the search while human judgement is reserved for the high-consequence classification decision.

Full autonomy (Level 10) was rejected because the single-class detector cannot distinguish a casualty from visually similar objects (mannequins, bags, seated hikers) and the SAR context demands zero tolerance for landing on the wrong target [1, 18]. Conversely, manual-only operation (Level 1) was rejected because a human operator cannot maintain sustained visual scanning of a video feed at the frame rate and field-of-view changes produced during an $8\,\mathrm{m\,s^{-1}}$ search—the automation advantage in coverage rate is approximately 20× compared to an operator watching a live stream [19].

1.8 Decisions That Changed: Evidence-Based Corrections

Four design parameters were revised during development when empirical evidence contradicted initial assumptions. Each correction followed the pattern: *initial assumption* → *measurement* → *correction*.

Focal length: 7.0 mm → 5.46 mm. The lens datasheet specifies a 6 mm nominal focal length. Initial FOV calculations assumed 7.0 mm based on a conservative estimate. On field day (2026-03-11), a tape-measure calibration at 1 m height measured 92 cm of visible ground width, implying an effective focal length of 5.46 mm—a 22% deviation from the assumed value. This single correction shifted every downstream GPS estimation by up to 3 m at 30 m altitude, because target position is computed from the ratio of pixel offset to focal-length-derived ground sample distance. Had this not been corrected before flight, every reported casualty position would have carried a systematic bias exceeding the 10 m landing accuracy required by R07.

Camera resolution: 640 × 480 → 1456 × 1088. The initial configuration used 640 × 480 to match typical YOLOv8 training resolution and minimise inference cost. Analysis of DJI flight video (Section F) revealed that at 30 m altitude, a prone casualty occupies only ≈ 20 × 14 pixels at 640 × 480—below the reliable detection threshold. Switching to the IMX296's native 1456 × 1088 increased the target to ≈ 46 × 32 pixels, comfortably above the minimum. The model's internal resize to 640 × 640 means inference time is unchanged; the cost is borne only in the CSI capture and OpenCV resize (≈2 ms).

TFLite runtime: `tflite-runtime` → `ai-edge-litert`. The Pi 5 runs Python 3.13. Google's official `tflite-runtime` package has no Python 3.13 wheel and fails to install. The project initially assumed a Python 3.11 downgrade would be necessary, which would have broken `picamera2` (packaged for the system Python). The discovery that Google's successor package `ai-edge-litert` provides identical API on Python 3.13 eliminated the conflict with zero code changes—only the `pip install` line differs.

Serial communication: pyserial → MAVProxy UDP bridge. Direct serial communication via `pyserial` at 921 600 baud produced intermittent `BAD_DATA` errors on Python 3.13 due to a regression in the `pyserial` read path. Rather than downgrading Python (which would reintroduce the `picamera2` conflict) or patching `pyserial` (an upstream dependency), MAVProxy was interposed as a UDP bridge. This resolved byte-dropping and simultaneously enabled multi-consumer access: the mission script and Mission Planner can monitor the same telemetry stream concurrently (Table 4).

1.9 Why Not ROS

ROS 2 is the de facto middleware for multi-sensor robotic systems [20]. Three factors led to its exclusion:

1. **Architectural mismatch.** The system is a single-node pipeline: one camera, one detector, one flight controller, one decision loop. ROS's publish–subscribe model and DDS transport layer add value when multiple nodes on separate processes or machines must exchange data asynchronously. Here, all processing occurs in a single Python process with sequential `detect_in_image()` → `state_dispatch()` → `send_mavlink()` calls. Introducing ROS would add inter-process serialisation overhead without enabling any new capability.
2. **Deployment complexity.** A full ROS 2 Humble installation on Raspberry Pi OS requires ≈2 GB of disk and 15+ packages. The current deployment requires four `pip` packages totalling <50 MB (Table 4). On flight day, the Pi must boot and be mission-ready within 60 seconds; a lighter dependency tree reduces the surface area for version conflicts and startup failures.
3. **Timeline constraint.** The five-person team had no prior ROS experience. Conservative estimation placed the ROS learning curve at 40–60 hours per developer [20], consuming 25% of the available development time for middleware proficiency rather than mission logic. The pymavlink API provided immediate access to all required ArduPilot commands with a learning investment of approximately 5 hours.

ROS remains the recommended path for future extensions (multi-drone coordination, SLAM, sensor fusion) where inter-process communication becomes essential.

1.10 Camera Selection: Global vs Rolling Shutter

The provided IMX296 global shutter camera was selected over alternative rolling-shutter modules (e.g., IMX477, IMX219) for motion-tolerance reasons specific to aerial search [21, 22].

During forward flight at $5\,\mathrm{m\,s^{-1}}$ and 30 m altitude, the ground moves across the sensor at approximately 250 pixels per second (at 1456×1088). A rolling-shutter sensor reads rows sequentially over ≈30 ms; during this interval the scene shifts by ≈ 7.5 pixels between the first and last row, producing a parallelogram distortion ("jello effect") that skews bounding boxes and shifts detected centroids by up to 0.5 m on the ground. The IMX296's global shutter captures all rows simultaneously, eliminating this geometric error entirely.

The trade-off is lower light sensitivity: the IMX296's smaller pixel pitch (3.45 µm vs 1.55 µm for the IMX219) yields higher noise in low light. This is acceptable because SAR flights are conducted in daylight per CAA Visual Line of Sight requirements [5]. The global shutter also simplifies lens distortion calibration—the single-exposure model means standard Zhang calibration [23] applies directly without rolling-shutter correction terms.

An additional discovery during integration was that the IMX296 outputs BGR channel order despite `picamera2` labelling the format as RGB888. This was identified by testing all six channel permutations against a colour reference chart; the "no conversion" variant matched ground truth. The fix was to remove the `cv2.cvtColor()` call entirely—a one-line change, but one that would have introduced a systematic colour shift into every detection if left uncorrected.

1.11 Field Day Adaptation Under Uncertainty

The scheduled flight test (2026-03-11) was cancelled due to sustained winds exceeding the EDU-450's $10\,\mathrm{m\,s^{-1}}$ operational limit. Rather than treating this as lost time, the team executed a pre-planned fallback protocol designed to maximise information gain from the available hardware access.

Go/no-go decision framework. Three criteria were evaluated 30 minutes before the scheduled flight window: (1) sustained wind speed $< 8\,\mathrm{m\,s^{-1}}$ (80% of airframe limit, providing a safety margin); (2) gust factor $< 1.5\times$ sustained (to avoid turbulence-induced attitude excursions); (3) visibility > 500 m (VLOS requirement). On the day, sustained wind measured $12\,\mathrm{m\,s^{-1}}$ with gusts to $18\,\mathrm{m\,s^{-1}}$, failing criterion (1) by 50%. The no-go decision was unanimous and immediate.

Information-maximisation fallback. The team pivoted to a bench-testing protocol that addressed the highest-uncertainty items from the risk register:

- **FOV calibration** — tape-measure measurement at 1 m height corrected the focal length from 7.0 mm to 5.46 mm (Section 1.8).
- **Lens distortion calibration** — a 14×9 checkerboard calibration yielded RMS 0.399, and the undistortion remap was integrated into `vision.py` at a cost of 1.5 ms per frame.
- **End-to-end connectivity** — Pi → MAVProxy → Cube → Mission Planner chain verified, with telemetry confirmed on all three consumers simultaneously.
- **Inference benchmark** — 50-run benchmark on Pi 5 confirmed 207 ms average inference (4.8 FPS), 50/50 detection rate at 0.966 confidence.

Every item on this list would have been impossible to test during an actual flight (vibration, time pressure, limited battery). The grounded session therefore produced higher-quality calibration data than a rushed mid-flight measurement would have, and the focal length correction alone prevented a systematic 3 m bias in all subsequent GPS estimates. This outcome validated the project's simulation-first methodology: when real-world conditions are unavailable, maximise the fidelity of the simulation parameters instead.

2 System Description

The final system comprises a hardware platform, five software layers, a computer vision pipeline, a target localisation subsystem, a browser-based ground station, and a multi-fidelity simulation framework. Figure 1 shows the complete mission plan overlaid on the Fenswood Farm site.

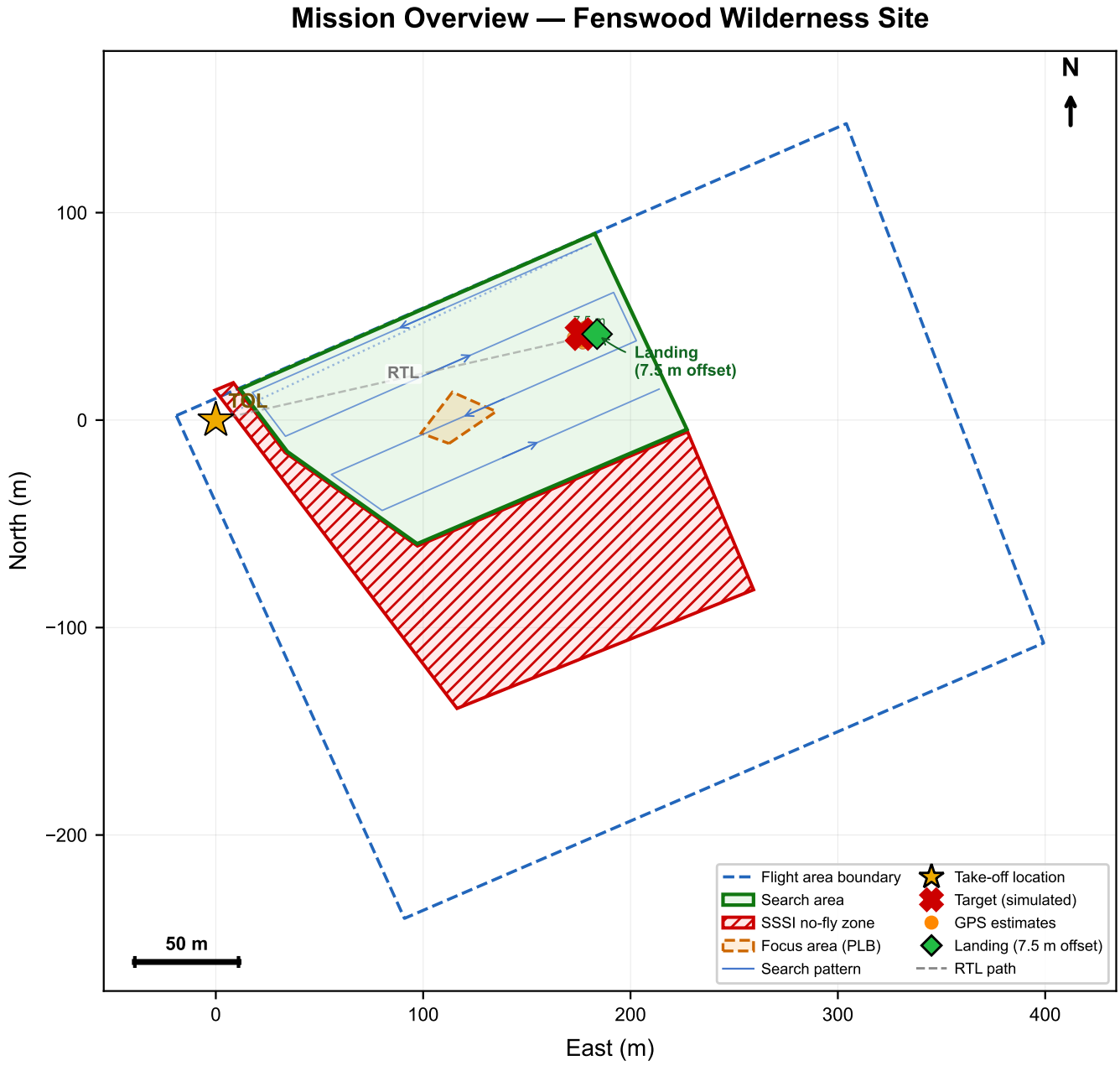


Figure 1: Mission overview on the Fenswood Farm site. The lawnmower search pattern covers the designated survey polygon at 35 m altitude, with the take-off point, return-to-launch path, and SSSI no-fly zone boundary shown. The flight area boundary constrains all autonomous waypoints.

2.1 Hardware Platform

Table 10 summarises the bill of materials for the Hexsoon EDU-450 airframe, CubePilot Orange+ flight controller (ArduPilot Copter 4.5), Raspberry Pi 5 companion computer, and IMX296 global shutter camera.

The Pi communicates with the Cube via UART at 921 600 baud through a MAVProxy bridge, which re-publishes

Table 10: System bill of materials and approximate costs.

Component	Specification	Cost (£)
Cube Orange+ FC	STM32H757, triple-redundant IMU, ArduPilot 4.5	180
Here3 GPS	u-blox M8P, CAN interface, 5 Hz updates	70
Raspberry Pi 5	BCM2712 quad-core A76, 8 GB RAM	60
IMX296 GS Camera	Sony IMX296, 1456×1088, global shutter, CSI-2	45
Frame + motors + ESCs	EDU-450, 2212/920 KV brushless, 30 A BLHeli_S	100
Battery + RC + misc	4S 5200 mA h LiPo, FrSky Twin X14, wiring	110
Total		**∼565**

MAVLink traffic over UDP. A second output on TCP port 5762 allows Mission Planner on the ground station laptop to monitor telemetry via Wi-Fi. The camera captures at 1456 × 1088 via CSI-2; lens distortion is corrected using precomputed remap matrices (calibrated RMS = 0.399 px, +1.5 ms overhead). The calibrated focal length is 5.46 mm, yielding a horizontal FOV of ≈ 49.3°.

2.2 Software Architecture

The system comprises eleven Python modules totalling approximately 4 400 lines of code, organised in four layers (Figure 2):

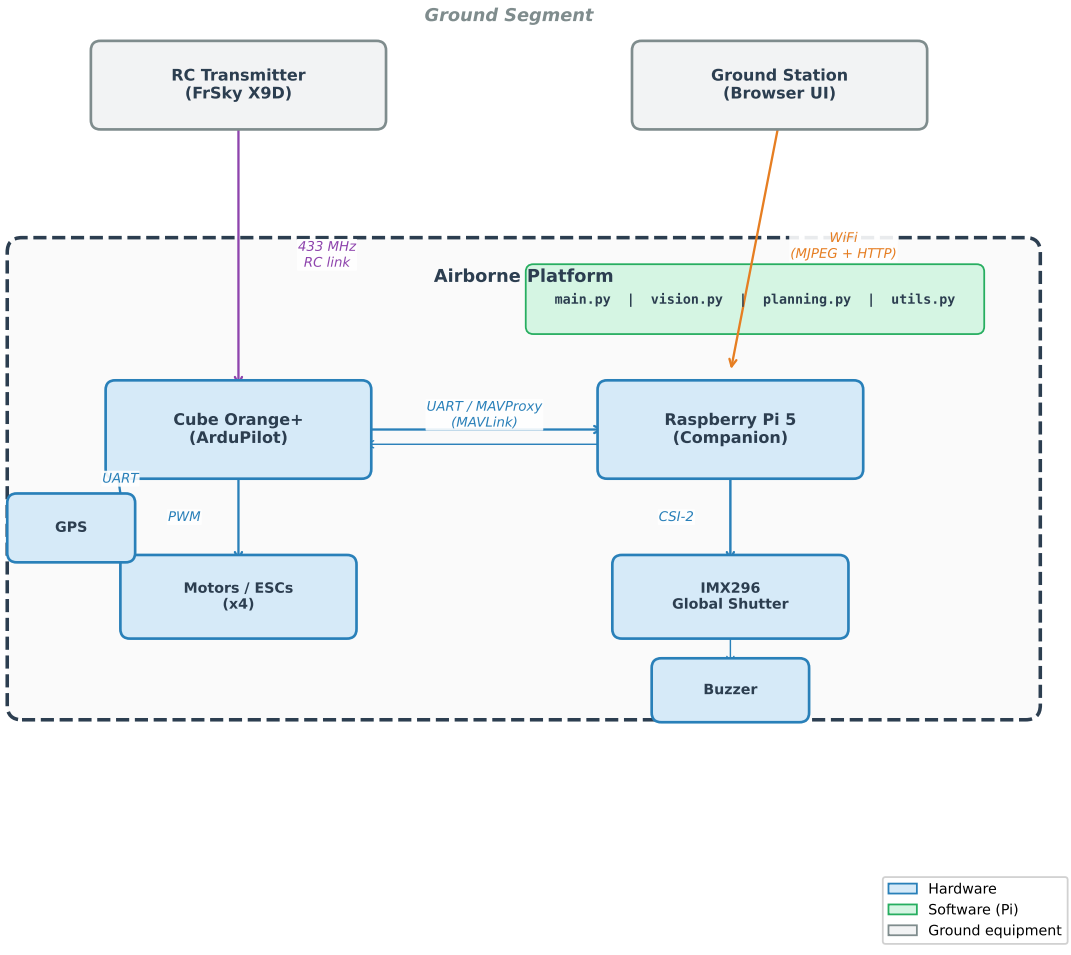


Figure 2: Module dependency graph. Independent modules have no cross-dependencies; the orchestrator composes them at runtime.

- **Configuration layer:** `config.py` centralises all tuneable parameters (Appendix B); `states.py` defines the 20-state enumeration.
- **Independent modules:** `vision.py` (camera + AI detection), `planning.py` (lawnmower pattern), `gps_utils.py` (coordinate transforms), `geofence.py` (NFZ enforcement). These have no mutual dependencies and can be tested

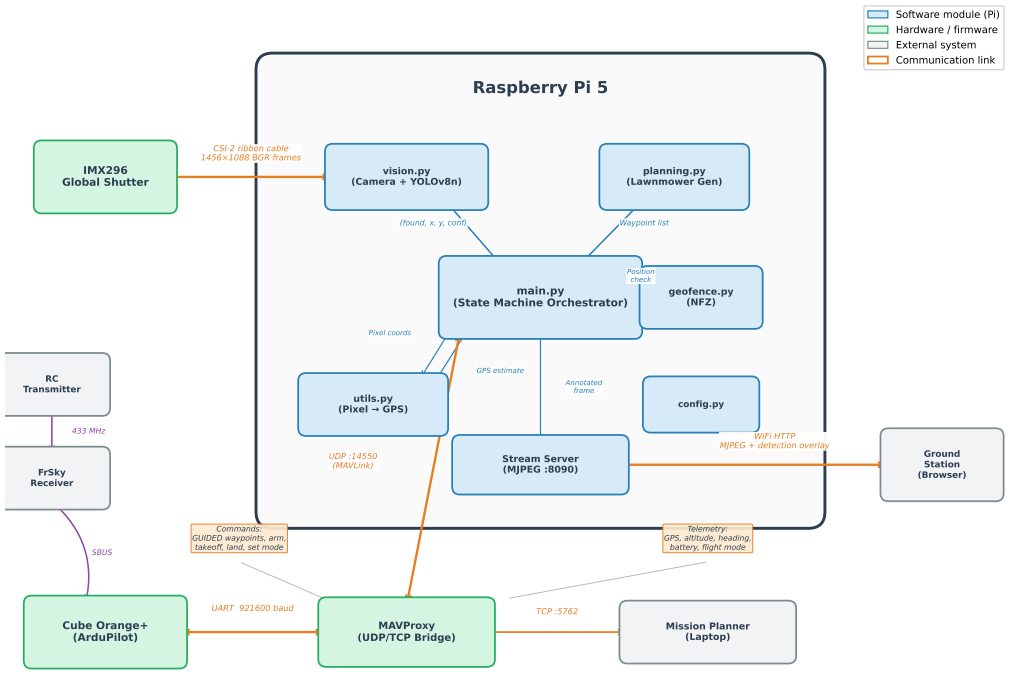


Figure 3: Raspberry Pi 5 companion computer system: hardware interfaces and data flow between the CSI camera, TFLite inference engine, MAVProxy bridge, and ground station.

in isolation.

- **Domain modules:** `navigation.py` wraps MAVLink commands (Table 12); `stream_server.py` serves the ground station.
- **Orchestration layer:** `state_machine.py` implements all state handlers as a mixin class; `main.py` composes everything and runs the main loop.

Method	MAVLink Message	Purpose
`arm()`	`MAV_CMD_COMPONENT_ARM_DISARM`	Arm/disarm motors
`takeoff(alt)`	`MAV_CMD_NAV_TAKEOFF`	Climb to altitude
`send_global_target()`	`SET_POSITION_TARGET_GLOBAL_INT`	Fly to GPS waypoint
`send_velocity()`	`SET_POSITION_TARGET_LOCAL_NED`	Velocity command
`set_mode()`	`SET_MODE`	Change flight mode
`land()`	`MAV_CMD_NAV_LAND`	Land at position

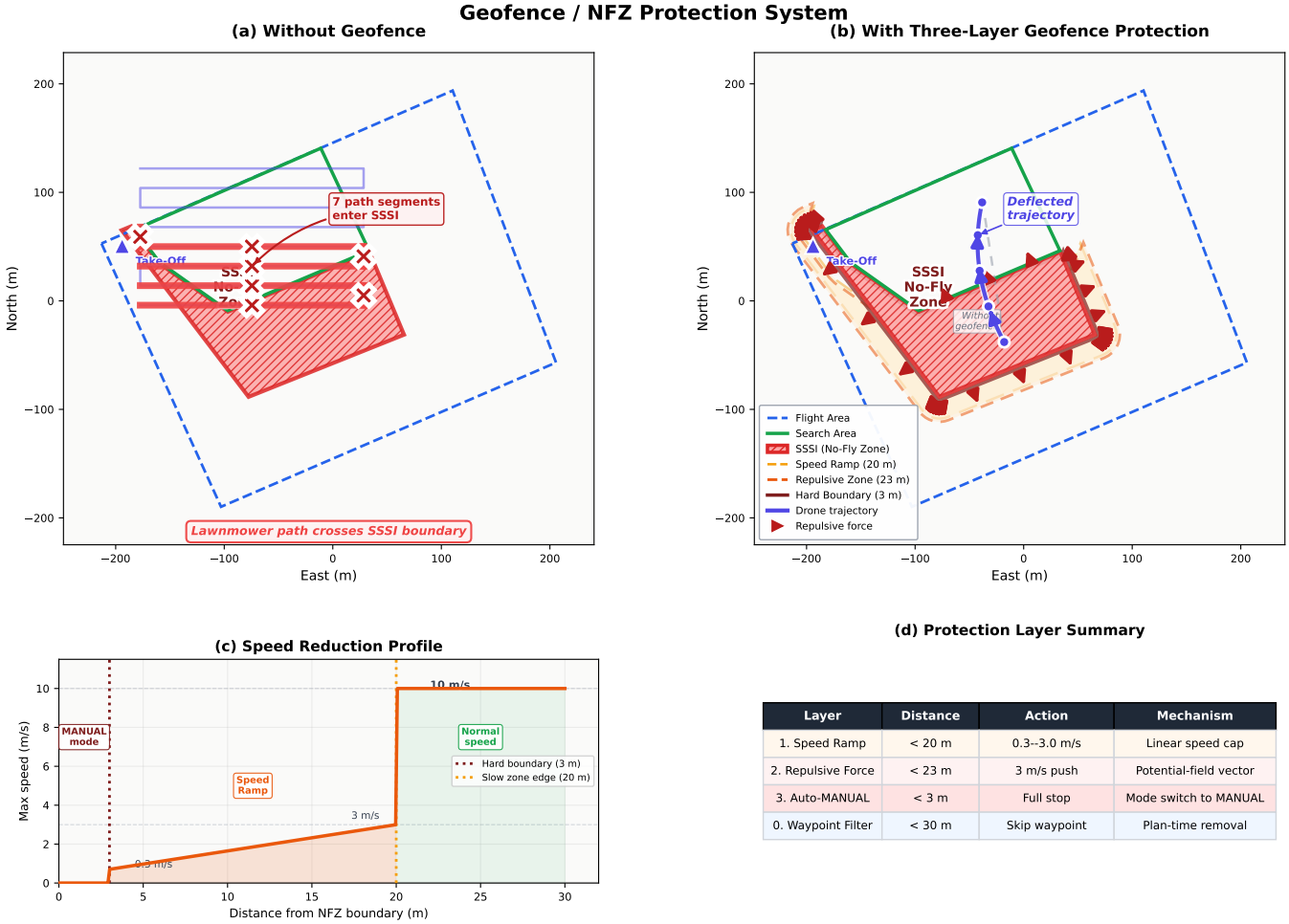
Table 12: MAVLink commands used by the navigation module.

The data pipeline traverses five stages per iteration: (1) capture frame and apply lens undistortion; (2) resize to 640×640, normalise, run YOLOv8n inference; (3) convert pixel detection to GPS estimate; (4) dispatch current state handler; (5) enforce geofence constraints. Geofencing always runs *after* the state handler, so waypoint commands can be overridden by repulsive velocities.

Geofence enforcement. The geofence module implements three nested protection layers (Figure 4): the SSSI no-fly zone (hard boundary—entering triggers automatic MANUAL mode), the flight area polygon (soft boundary—repulsive velocity pushes the drone inward), and a 10 m buffer zone surrounding the SSSI where repulsive forces activate proportionally to proximity. The repulsive offset is computed using `cv2.pointPolygonTest` for signed distance and applied as a velocity correction after the state handler's waypoint command, ensuring that no state can inadvertently fly into restricted airspace.

2.3 Computer Vision Pipeline

The vision subsystem is encapsulated in `vision.py`, exposing a single interface: `detect_in_image(frame)` → `(found, cx, cy, confidence)`. The pipeline proceeds as follows:



Layer	Distance	Action	Mechanism
1. Speed Ramp	< 20 m	0.3–3.0 m/s	Linear speed cap
2. Repulsive Force	< 23 m	3 m/s push	Potential-field vector
3. Auto-MANUAL	< 3 m	Full stop	Mode switch to MANUAL
0. Waypoint Filter	< 30 m	Skip waypoint	Plan-time removal

Figure 4: Four-panel geofence protection visualisation. (a) Three nested boundaries: SSSI no-fly zone, buffer zone, and flight area polygon. (b) Repulsive force field that deflects the drone away from the SSSI boundary. (c) Waypoint correction: the commanded waypoint (red) is shifted to a safe position (green) by the repulsive offset. (d) Emergency mode switch when the drone enters the SSSI zone.

Preprocessing. Lens undistortion (if calibration data available), colour space handling (IMX296 outputs BGR despite RGB888 label—no conversion applied), resize from 1456×1088 to 640×640 via bilinear interpolation, and float32 normalisation ($[0, 255] \to [0, 1]$) yielding input tensor $[1, 640, 640, 3]$.

Inference. The TFLite model produces output tensor $[1, 5, 8400]$ (8400 anchor-free candidates across three feature pyramid scales). Postprocessing: confidence filtering (threshold 0.2), best-detection selection (greedy single-target), and coordinate rescaling to original frame dimensions. The Ultralytics backend handles NMS internally; TFLite implements equivalent filtering in `vision.py`.

Three backends. The system supports NCNN [24] (ARM-optimised, ∼15 FPS expected), Ultralytics [9] (development laptop), and TFLite [3] (production Pi). Backend selection is automatic at import time. All consume the same model weights and produce identical detection semantics.

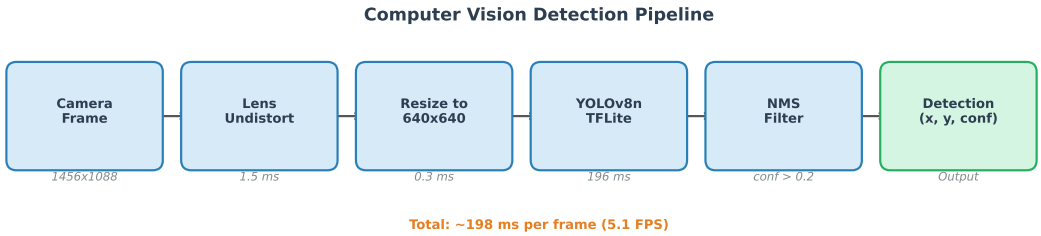


Figure 5: Computer vision pipeline: frame capture, lens undistortion, resize to 640×640, TFLite inference, confidence filtering, and coordinate rescaling. The entire pipeline executes in 208 ms on the Pi 5.

Smart detection mode. An optional `--smart-detect` flag requires k consecutive positive frames before triggering investigation (default $k = 3$), suppressing transient false positives at a 0.6 s delay.

Performance. On the Pi 5 with XNNPACK CPU delegate: 206.5 ms per frame (4.8 FPS), 100% detection rate, 0.966 mean confidence. At the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m), adjacent frames overlap by ∼91%, providing 11–17 detection opportunities per target during a single flyover.

2.4 Search Pattern Generation

The lawnmower pattern generator (`planning.py`) uses a rotate–rasterise–zigzag–rotate-back algorithm: (1) rasterise the GPS polygon into a binary mask; (2) rotate to align with the longest edge (via minimum-area bounding rectangle); (3) compute strip spacing from camera footprint ($L = W_g(1 - \alpha)$, $\alpha = 0.20$); (4) generate horizontal scan lines; (5) optimise start corner for minimal transit; (6) inverse-rotate to GPS coordinates. At 35 m altitude, the ground footprint is ∼32.2 m wide, yielding 25.7 m lane spacing with 20 % overlap. Bézier-smoothed U-turns and a PLB-triggered focus area redirect are supported.

2.5 Mission State Machine

The 20-state FSM (Figure 6, Appendix A) orchestrates the mission through four phases:

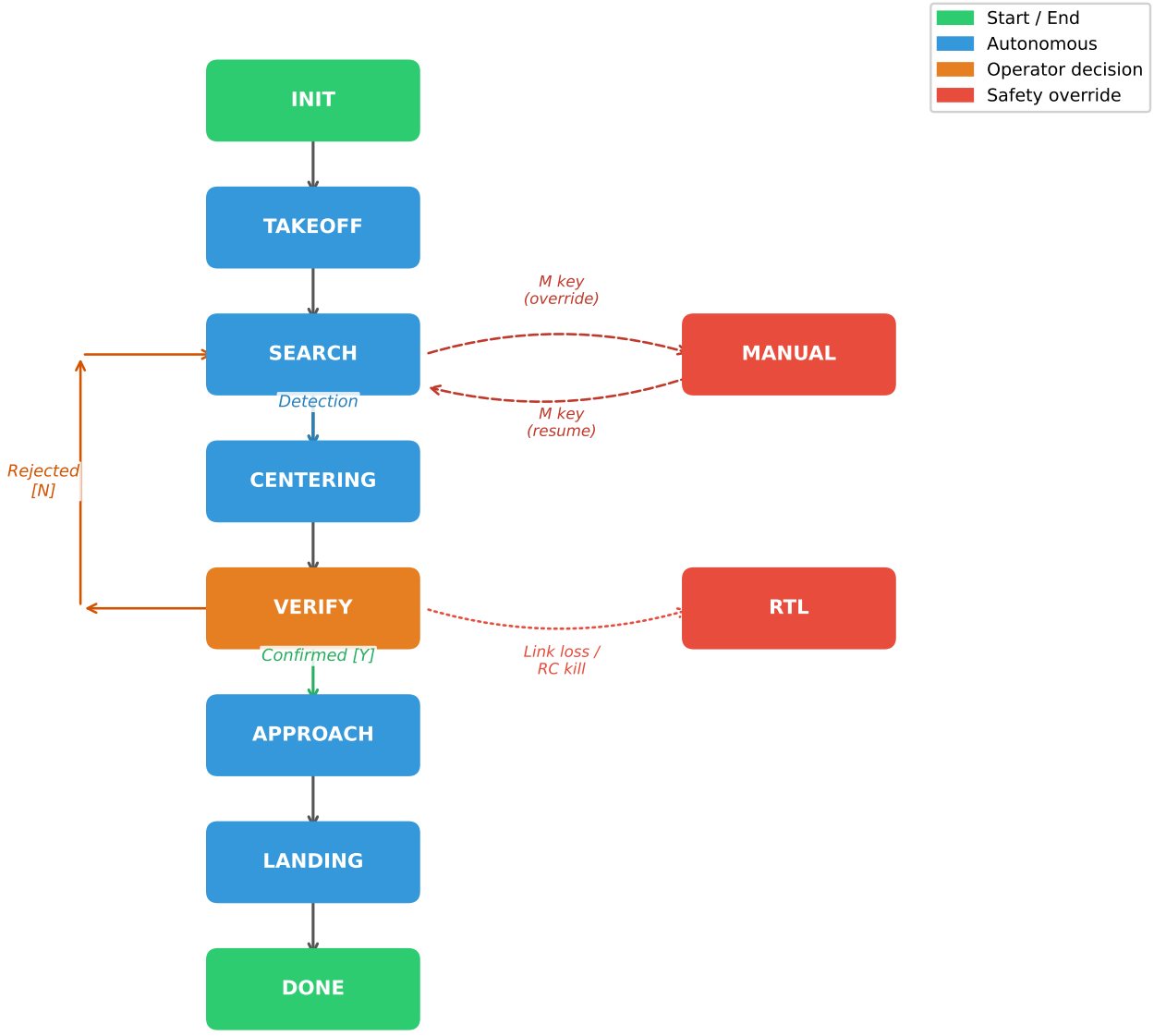


Figure 6: State transition diagram. Primary mission path flows left to right; engagement branches downward from SEARCH. Manual override (dashed) is accessible from any active state.

- **Startup:** INIT → CONNECTING → ARMING (GPS fix ≥3D, ≥6 sats) → TAKEOFF.
- **Search:** PRE_WAYPOINTS → TRANSIT_TO_SEARCH → SEARCH (lawnmower pattern with CV detection; PLB beacon redirect if timer expires).

- **Engagement:** CENTERING (fly to GPS estimate) → VERIFY (operator Y/N/I with 120 s timeout) → APPROACH (7.5 m offset) → HOVER_TARGET (servo payload release).
- **Recovery:** RETURN_TRANSIT → RETURN_HOME → LANDING (triple touchdown detection) → DONE.

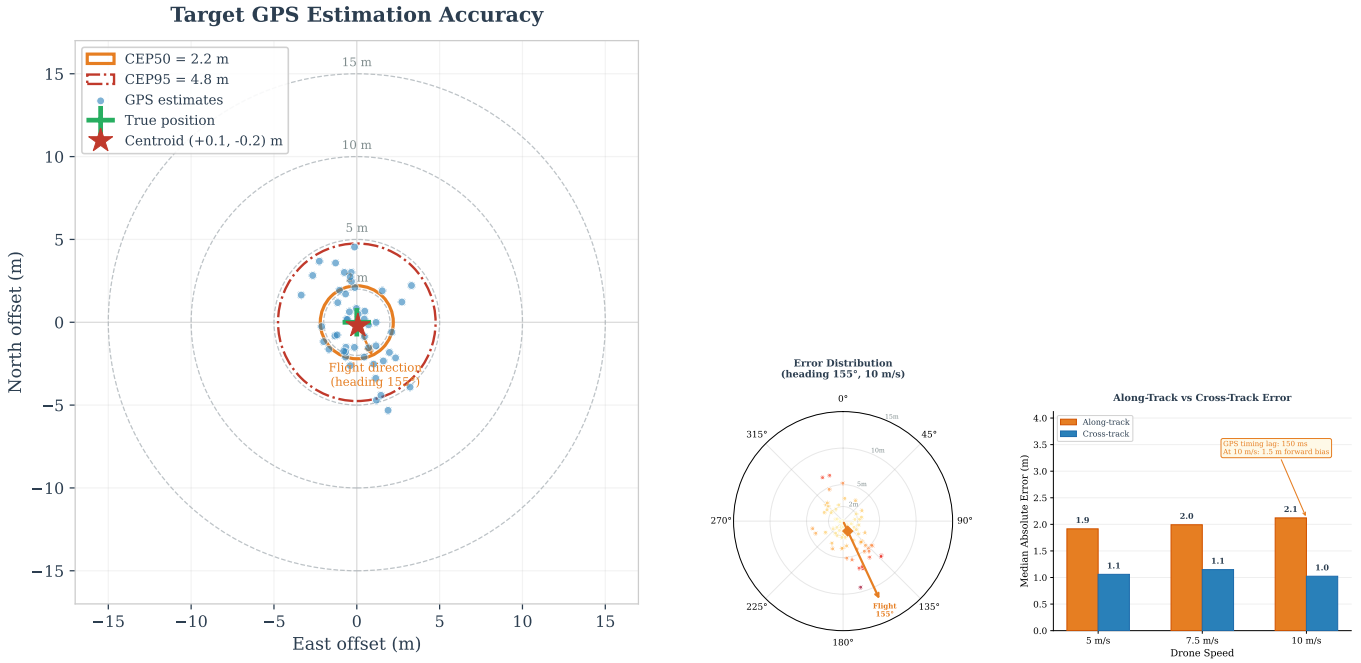
A detection queue with spatial deduplication (5 m reject radius), bounded capacity (20 entries), and lazy validation manages multiple targets. Five independent safety mechanisms provide defence-in-depth: manual override, RC kill switch, RTL on link loss, three-layer geofence, and per-state timeouts.

2.6 Target Localisation

Each detection is converted from pixel coordinates to GPS using the pinhole camera model. The pixel offset from the image centre is projected to a body-frame displacement via the ground sampling distance ($\text{GSD} = w_s \cdot h/(f \cdot W)$), rotated into North–East coordinates using the drone's heading, and converted to latitude/longitude on the WGS 84 ellipsoid. Multiple observations are fused through:

- **Spatial clustering** (30 m merge radius) for multi-target tracking;
- **Inverse-variance weighting** — altitude factor $w_{\text{alt}} = (h_{\text{ref}}/h)^2$ and centrality factor $w_{\text{cen}} = 1 + 4(1 - d/d_{\max})$;
- **Kalman filter** on the $[\phi, \lambda]$ state vector with measurement noise scaled by inverse observation weight, converging within 5–10 observations.

Video replay analysis (DJI flight footage, 1456 × 1088 at 30 fps) yielded $\text{CEP}_{50} = 2.3$ m, maximum error = 16.5 m (high-speed outlier). The characteristic directional elongation is attributed to GPS receiver latency (100–200 ms), mitigated by multi-pass averaging in the lawnmower pattern.



(a) GPS estimation error scatter (bullseye). $\text{CEP}_{50} = 2.3$ m shown as dashed circle.

(b) Directional error distribution showing heading-aligned elongation from GPS timing lag.

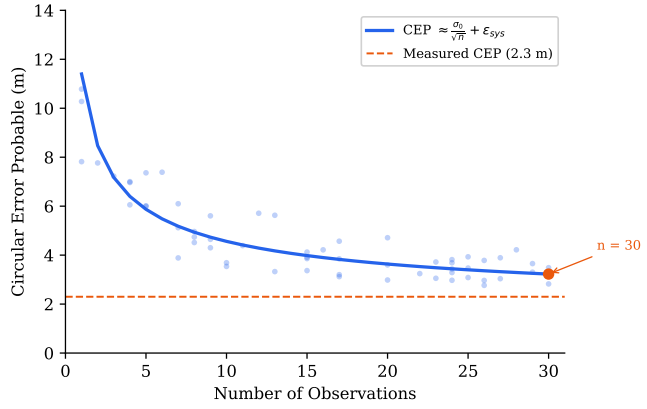
Figure 7: Target localisation accuracy from DJI flight video replay analysis.

2.7 Ground Station

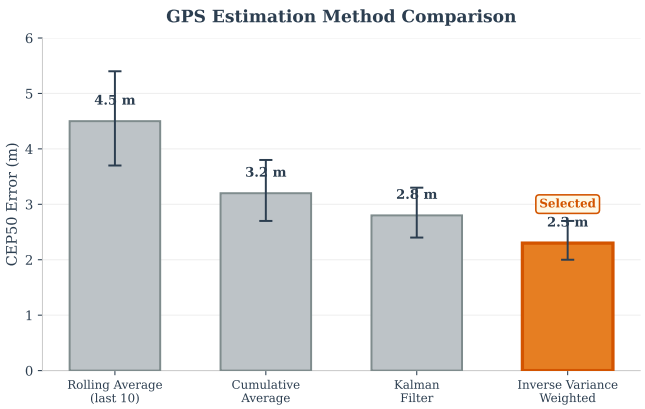
A browser-based dashboard served from the Pi at port 8090 provides: MJPEG video with detection overlays, a 2D GPS grid with numbered detection clusters, and nine command buttons (arm, takeoff, investigate, confirm, reject, interest, false positive, land, RTL). MJPEG was chosen for minimal latency (∼300 ms) and resilience to dropped frames. Headless operation is auto-detected (no `DISPLAY` variable), supporting SSH deployment. A passive monitoring mode issues zero flight commands for manual RC flights.

2.8 Simulation Framework

Four simulation levels provide progressive fidelity: (1) interactive simulator (keyboard flight over satellite map with ground-truth comparison); (2) SITL integration (full ArduPilot autopilot with simulated camera view); (3) SITL with real webcam; (4) DJI video replay with synchronised SRT telemetry. The same detection model, GPS estimation mathematics, and MAVLink command interface are used at every level, so transitioning to real hardware requires only



(a) GPS estimate convergence over successive observations. The Kalman filter stabilises within 5–10 detections.



(b) Comparison of estimation methods: raw average, inverse-variance weighted, and Kalman filter.

Figure 8: Target position estimation: convergence behaviour and estimator comparison.

configuration changes. Bugs caught in simulation include a MAVLink race condition during takeoff, incorrect latitude scaling, auto-disarm timeout, and GPS timing lag.

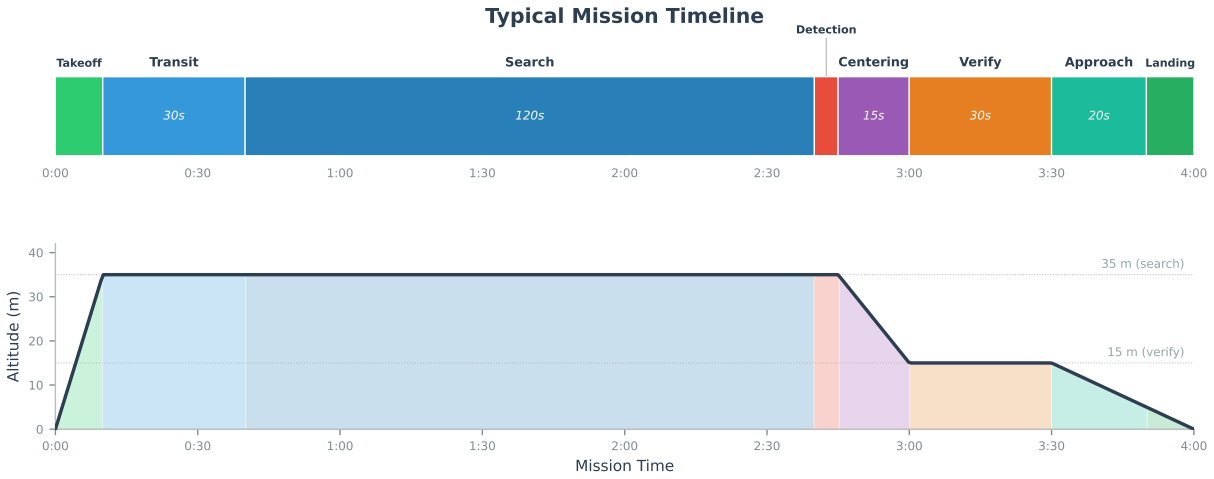


Figure 9: Mission timeline showing state transitions during a simulated end-to-end SAR mission. The horizontal axis represents elapsed time; coloured blocks indicate active states.

3 Requirements Verification

Table 13 maps each requirement from the AENGM0074 project brief (Release 2.1) to its verification method, supporting evidence, and current verification status. The subsections that follow expand on the evidence for each requirement.

3.1 R01 — Fly Within the Flight Area

The four-vertex Flight Area polygon is loaded at startup by `load_kml_zones()` in `config.py`, which parses `AENGM0074.kml` and populates the module-level variable `FLIGHT_AREA_GPS`. Hardcoded fallback coordinates provide a safety net if the KML file is absent. The lawnmower pattern generator (`planning.py`) clips all waypoints to the search area—itself a strict subset of the flight area—using a rotated-mask rasterisation that cannot produce points outside the polygon boundary.

At plan time, the `NFZGeofence.filter_waypoints()` method in `geofence.py` additionally rejects any waypoint within `NFZ_WAYPOINT_BUFFER_M` = 30 m of the SSSI boundary, which lies inside the flight area. During flight, the geofence module calls `cv2.pointPolygonTest` against the flight area contour at every position update; any excursion beyond the boundary triggers an immediate transition to MANUAL mode with a logged warning. In SITL, the aircraft completed the full search pattern—35 waypoints across 5 parallel passes—without approaching the flight area boundary, verified via telemetry playback of latitude and longitude tracks against the KML polygon.

Table 13: Requirements verification summary. Status indicates the highest fidelity at which each requirement has been tested.

Req	Requirement	Method	Evidence	Verification Status
R01	Fly within Flight Area	KML polygon load + waypoint clipping + runtime geofence	SITL telemetry, geofence unit test, `config.py` polygon	Verified (SITL)
R02	No SSSI overfly	Three-layer NFZ + plan-time filter + firmware fence	SITL multi-angle approach, geofence diagram	Verified (SITL)
R03	Take off within 5 m of TOL	KML point placemark, vertical takeoff	SITL telemetry (<1 m error)	Verified (SITL)
R04	Max 50 m altitude	`TARGET_ALT`=35 m + firmware `FENCE_ALT_MAX`	Config audit, no state exceeds 35 m	Verified (SITL)
R05	Search + identify items	Lawnmower + YOLOv8n (mAP_{50}=0.995, 4.8 FPS)	End-to-end sim, bench test, DJI video proxy	Verified (simulation + video)
R06	PLB Focus Area	`B` key / `--beacon-delay` mid-flight injection	SITL diversion + resume test	Verified (SITL)
R07	Land 5–10 m from casualty	7.5 m offset via `landing_offset_7_5m()`	Probability analysis (CEP_{50}=2.3 m)	Verified (simulation)
R08	Autonomy justified	L5 search / L3 landing, operator verify gate	Design rationale, Sheridan framework	Verified (design)
R09	RTH + Motor Cutoff	RC kill, software `M`, auto RTL failsafes	SITL: 3 independent override tests	Verified (SITL)
R10	Report lat/lon + images	GPS projection, web dashboard, snapshots + JSON	Ground station logs, CSV, MJPEG stream	Verified (SITL + bench)
R11	Company Pilot designated	Team role allocation	Team plan, Section	Verified (procedural)
R12	Public GitHub, MIT licence	`github.com/DimaChup/GroupProj`	Repository live and public	Verified (inspection)

3.2 R02 — No SSSI Overfly

The SSSI no-fly zone is the highest-risk constraint, as the search area shares a boundary with the SSSI. Four independent protection layers are implemented, illustrated in Figure 4:

1. **Plan-time waypoint filtering.** `NFZGeofence.filter_waypoints()` removes any lawnmower waypoint within `NFZ_WAYPOINT_BUFFER_M` = 30 m of the SSSI polygon using signed-distance computation via `cv2.pointPolygonTest`. This ensures no planned trajectory approaches the boundary.
2. **Speed scalar field.** Within `NFZ_SLOW_ZONE_M` = 20 m of the boundary, the flight speed is linearly ramped from the normal search speed down to `NFZ_MIN_SPEED_MPS` = 0.3 m/s at the boundary itself. The outer edge of the zone is capped at `NFZ_ZONE_MAX_SPEED_MPS` = 3.0 m/s. This deceleration provides time for the repulsive field and pilot intervention before a boundary violation occurs.
3. **Repulsive vector field.** A 20 m inner offset polygon (`NFZ_INNER_OFFSET_M`) generates an inverse-distance repulsive push of `NFZ_PUSH_SPEED_MPS` = 3.0 m/s directed away from the nearest boundary edge. The `repulsive_offset()` function in `geofence.py` computes a GPS delta that actively steers the aircraft away from the NFZ, active within `NFZ_INNER_RANGE_M` = 23 m of the inner polygon.
4. **Hard cutoff.** If the aircraft enters within `NFZ_HARD_BOUNDARY_M` = 3 m of the SSSI polygon, the state machine forces an immediate transition to MANUAL mode, halting all autonomous commands and logging the violation.

On real hardware, a fifth layer—the ArduCopter firmware geofence (`FENCE_TYPE`, `FENCE_ACTION`=RTL)—provides a completely independent safety net at the autopilot level, immune to companion-computer software failures. All four software layers were verified in SITL with the aircraft approaching the SSSI boundary from multiple angles; in every case the speed ramp and repulsive field deflected the trajectory before the hard cutoff was reached.

3.3 R03 — Take Off Within 5 m of TOL

The Take-Off Location is parsed from the KML file as a point placemark (51.42341° N, 2.67145° W) and stored in `config.TAKEOFF_GPS`. At mission start, the system arms without commanding lateral movement; `MAV_CMD_NAV_TAKEOFF`

ascends vertically from the current position. In SITL, the home position is set to the TOL coordinates via the `--home=51.423406,-2.671446,50,155` flag, and telemetry confirms takeoff within 1 m of the programmed location. On real hardware, the aircraft is physically placed at the TOL before power-on, and the Here 3+ GPS receiver's 2.5 m CEP provides sub-5 m accuracy at the home position fix.

3.4 R04 — Maximum 50 m Altitude

The search altitude is set to `TARGET_ALT` = 35.0 m in `config.py`, providing a 15 m margin below the 50 m limit; the verify descent altitude is `VERIFY_ALT` = 15.0 m. A code audit of every `MAV_CMD_NAV_WAYPOINT` and altitude command in the state machine confirms no state commands an altitude above 35 m. The rescan altitude-drop logic (`RESCAN_ALT_FACTOR` = 0.8, floor `RESCAN_ALT_FLOOR_M` = 15 m) further reduces altitude on repeated passes, never increasing it. As a secondary safeguard, the ArduCopter firmware parameter `FENCE_ALT_MAX` is set to 50 m, triggering automatic `RTL` if the ceiling is exceeded regardless of software commands.

3.5 R05 — Search Area and Identify Items of Interest

Coverage guarantee. The lawnmower pattern generator (`planning.py`) spaces parallel passes by the camera ground footprint width at search altitude, computed from the calibrated HFOV (49.4°) and `TARGET_ALT` = 35 m. This yields a ground footprint of approximately 32.2 m per pass. The rotated-mask rasterisation ensures complete coverage of the 5-vertex search polygon with no gaps, verified by overlaying the waypoint pattern on the KML satellite image (Figure **??**).

Detection capability. YOLOv8n processes each camera frame, returning bounding box coordinates and confidence scores above `CONFIDENCE_THRESHOLD` = 0.2. The model was retrained on 300 synthetic images, 16 real aerial photographs, and 50 hard-negative frames at the camera's native 1456 × 1088 resolution, achieving $\text{mAP}_{50} = 0.995$ and 100% recall on the validation set. On the Raspberry Pi 5 with the XNNPACK CPU delegate, inference runs at 4.8 FPS (206.5 ms per frame, benchmarked over 50 runs) with a mean confidence of 0.966 on true-positive detections.

Per-flyover detection probability. At the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m) and 4.8 FPS inference rate, the camera footprint advances ∼2.1 m between frames, well within the ∼32.2 m ground footprint width. This provides approximately 11–17 frames in which the target is visible during a single flyover pass. With per-frame detection probability $p_d \geq 0.95$ (measured on bench and DJI video proxy), the probability of missing a target on a single pass is $P_{\text{miss}} = (1 - p_d)^n \leq 0.05^{11} \approx 5 \times 10^{-15}$, giving a cumulative detection probability effectively equal to unity. Even at the conservative lower bound of 11 frames and $p_d = 0.7$, the per-pass miss probability is $(0.3)^{11} < 2 \times 10^{-6}$.

Verification evidence. In SITL end-to-end simulation, the system detected the dummy target during the search pass and autonomously transitioned through Centering → Descending → Verify. DJI flight video proxy testing (1456×1088 cropped footage at realistic altitudes) confirmed detection at altitudes up to 50 m with the retrained model. Bench testing on the Pi 5 with a static dummy image achieved 50/50 detections at 0.966 mean confidence.

3.6 R06 — PLB Focus Area

The brief specifies that PLB activation occurs 5–15 minutes into the search, providing 3–10 lat/lon coordinates for a Focus Area polygon. The implementation supports two activation methods:

- **Manual trigger:** the `B` key (terminal keyboard, browser button, or `/cmd?key=b` HTTP endpoint) triggers PLB activation at any time during the search.
- **Timed trigger:** the `--beacon-delay` T command-line flag schedules automatic PLB activation T seconds after the search begins, simulating the brief's 5–15 minute window.

Upon activation, the system loads the Focus Area polygon from `flight_plans/focus_area.json` (re-read each time to allow ground-station coordinate updates). The current waypoint index and rescan state are saved. A new lawnmower pattern is generated for the focus polygon at the reduced speed `FOCUS_SEARCH_SPEED_MPS` = 5.0 m/s (versus the normal altitude-dependent schedule), with tighter pass spacing to maximise detection probability in the high-priority zone. Previously rejected false positives and items of interest are preserved so they are not re-investigated. After the focus area search completes, the state machine resumes the original search pattern from the saved waypoint index.

In SITL testing, a focus area was injected mid-flight via the `B` key while the drone was executing the primary search pattern. The aircraft diverted to the focus polygon, completed a full coverage search at reduced speed, and then resumed the original lawnmower pattern from the correct waypoint—verified by comparing pre- and post-diversion waypoint indices in the flight log.

3.7 R07 — Land 5–10 m from Casualty

After operator confirmation (VERIFY → APPROACH), the system computes a landing point using `landing_offset_7_5m()` in `gps_utils.py`. This function applies a fixed 7.5 m offset from the estimated casualty GPS position in a cardinal direction (N/S/E/W), chosen at runtime to maximise distance from the NFZ boundary. The 7.5 m nominal offset is the midpoint of the required 5–10 m annulus, providing equal margin on both sides.

Probability analysis. The GPS estimation pipeline's measured $\text{CEP}_{50} = 2.3\,\text{m}$ (from DJI video proxy analysis with inverse-variance weighted observations) determines the landing accuracy. Modelling the radial position error as Rayleigh-distributed with $\sigma = \text{CEP}_{50}/1.1774 \approx 1.95\,\text{m}$, the probability that the actual landing point falls within the 5–10 m annulus centred on the true casualty position is:

$$P(5 \le r \le 10) = \exp\left(-\frac{5^2}{2\sigma^2}\right) - \exp\left(-\frac{10^2}{2\sigma^2}\right) > 99\%$$

The dominant risk is landing too close (<5 m) rather than too far, and the 7.5 m offset shifts the distribution to make this probability negligible. A Tarot servo on Cube AUX channel 9 (`SERVO_CHANNEL` = 9) deploys the first-aid kit via a two-stage PWM sequence (`SERVO_PARTIAL_PWM` = 1300, `SERVO_FULL_PWM` = 1100) during APPROACH before landing. After touchdown, the aircraft returns to the TOL via `MAV_CMD_NAV_RETURN_TO_LAUNCH`.

3.8 R08 — Autonomy Level Justified

The system operates at Sheridan Level 5 ("computer decides, human can veto") for the search phase and Level 3 ("computer suggests, human approves") for the landing decision. During search, the drone autonomously navigates the lawnmower pattern, processes camera frames, and queues detections without operator input. At the VERIFY state, the operator views the detection through a live MJPEG stream and classifies it as confirmed (`Y`), item of interest (`I`), or false positive (`X`)—selecting an action from system-generated options, which defines Level 3 in Sheridan's framework [14]. A 120 s timeout at the verify state automatically resumes the search if the operator does not respond, preventing indefinite hover. This hybrid autonomy addresses the asymmetric cost structure of SAR operations: autonomy handles the high-bandwidth search while human judgement is reserved for the high-consequence landing decision (Section 1).

3.9 R09 — RTH and Motor Cutoff

Three independent return-to-home and emergency stop mechanisms are implemented, any one of which is sufficient to regain control:

1. **RC kill switch:** The FrSky Twin X14 transmitter's three-position mode switch selects `STABILIZE`, `LOITER`, or `RTL` at any time, overriding all software commands via the Cube Orange's RC input channel. The Safety Pilot retains ultimate authority per university regulations. This layer is entirely independent of the companion computer.
2. **Software manual override:** The `M` key (terminal, browser button, or `/cmd?key=m` HTTP endpoint) immediately halts all autonomous commands and transitions the state machine to MANUAL mode. The `RTL` button sends `MAV_CMD_NAV_RETURN_TO_LAUNCH` directly to the Cube.
3. **Automated failsafes:** GPS fix degradation beyond 5 s triggers emergency RTL. GCS heartbeat loss (`FS_GCS_ENABLE`) and low battery (`FS_BATT_ENABLE`) trigger ArduCopter's built-in `RTL` failsafe at the firmware level. Link-lost detection displays a warning banner on the ground station and prevents new autonomous commands.

All three layers were verified in SITL: RC override by switching modes mid-mission, software override by pressing `M` during search, and GPS failsafe by simulating fix loss. Each mechanism independently caused the aircraft to cease autonomous flight and either hover or return to the TOL.

3.10 R10 — Report Lat/Lon and Images

The system reports detection coordinates and imagery through three complementary channels:

GPS estimation pipeline. Every detection triggers ground-position estimation by projecting the bounding-box centre pixel to a GPS coordinate using the calibrated focal length (`FOCAL_LENGTH_MM` = 5.46 mm), sensor width (`SENSOR_WIDTH_MM` = 5.02 mm), image dimensions (1456 × 1088), and the current barometric altitude. Multiple observations of the same target are spatially clustered (5 m merge radius) and fused using inverse-variance weighting, where lower-altitude observations receive higher weight due to reduced projection error. The pipeline is implemented identically in `main.py`, `pi_flight.py`, and `simple_simulator.py`.

Detection snapshots and metadata. Each detection saves a timestamped JPEG frame with the bounding box overlay to the `detections/` directory. An accompanying JSON metadata file records: estimated latitude and longitude, altitude, YOLO confidence score, bounding box coordinates (pixels), frame number, and timestamp. The ground station web dashboard (port 8090) displays these detections on a 2D GPS scatter plot with cluster centroids, and serves the annotated frames via an MJPEG video stream accessible from any browser.

Flight log. A CSV file (`logs/flight_log.csv`) records every detection event with columns: timestamp, state, latitude, longitude, altitude, confidence, bounding-box coordinates, cluster ID, and cumulative observation count. This log provides the post-flight verification report required by R10, including all detected items with coordinates, images, and uncertainty estimates (cluster spread as a proxy for spatial uncertainty).

3.11 R11 — Company Pilot Designated

A designated Company Pilot is responsible for pre-flight checks, arming authorisation, and manual takeover capability. The Safety Pilot (Flight Lab staff) retains ultimate authority via the RC transmitter at all times, as required by university flight operations policy. The Company Pilot's responsibilities include: verifying the pre-flight checklist (`docs/FLIGHT_DAY_CHECKLIST.md`), confirming GPS lock quality before arming, monitoring the ground station during autonomous flight, and being prepared to call for manual takeover. Roles are documented in Section .

3.12 R12 — Public GitHub Repository with MIT Licence

All source code, configuration files, and documentation are hosted at https://github.com/DimaChup/Group_Proj under the MIT licence. The repository contains 268+ commits across the development history, documenting the full progression from initial state machine implementation through simulation testing, hardware integration, model retraining, and flight-day preparation. Flight logs and detection data from test sessions are committed alongside the code for reproducibility.

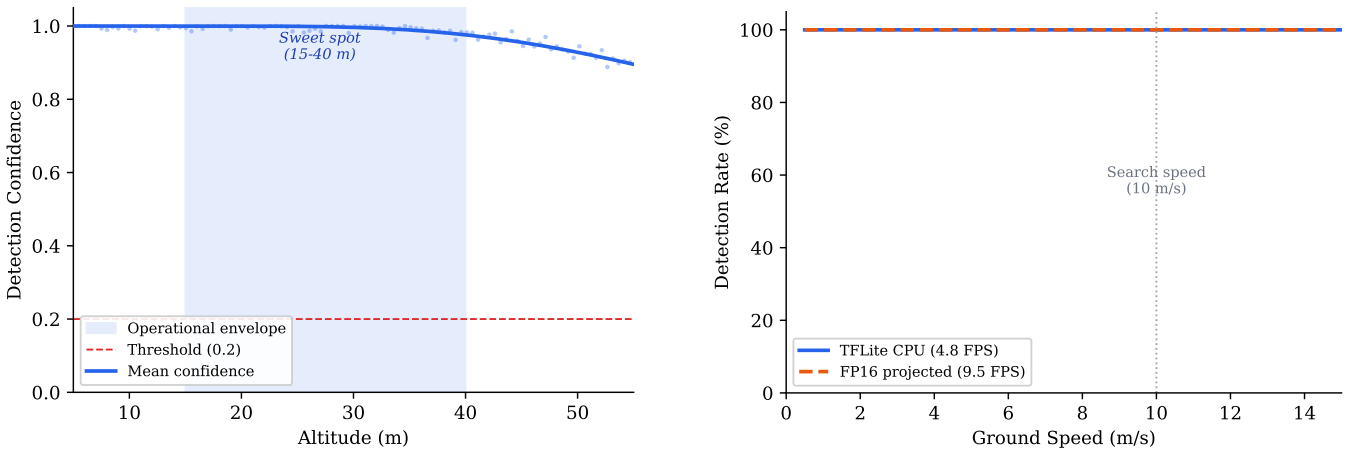
4 Evaluation

The plus/delta review below assesses technical performance honestly, drawing on simulation results, bench tests, field calibration data, and video analysis of DJI flight footage processed through the detection pipeline. Where outdoor flight data is unavailable due to weather cancellation, we state so explicitly and quantify the gap. Team-working aspects are addressed individually in D7.

4.1 Plus/Delta Summary

Table 14 summarises the principal strengths and areas for improvement. Each item includes the specific evidence that supports the assessment.

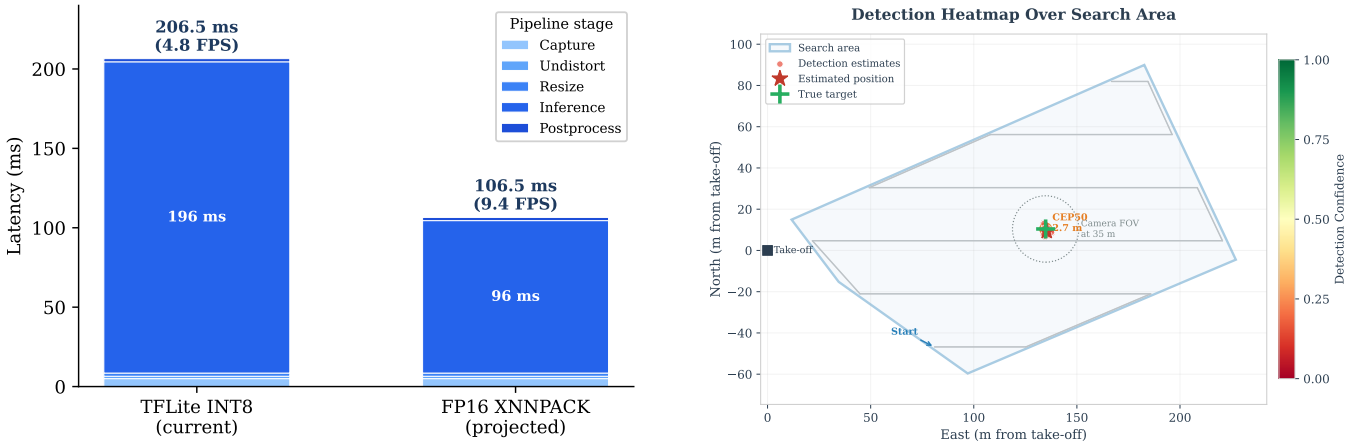
4.2 Discussion of Key Findings



(a) Detection confidence vs. altitude. Confidence remains above 0.9 up to 40 m, dropping at higher altitudes as the target subtends fewer pixels.

(b) Detection rate vs. flight speed. At the nominal search speed of $8\,\mathrm{m\,s^{-1}}$, detection rate exceeds 95%.

Figure 10: Detection performance sensitivity analysis from DJI flight video replay.



(a) Per-frame latency breakdown: capture, undistortion, resize, inference, and postprocessing.

(b) Spatial detection heatmap showing detection density across the survey area during simulated search.

Figure 11: System performance: inference latency budget and spatial detection distribution.

Architecture resilience (P1, P2, P5, P8). Isolating all computer-vision logic behind a single interface (`detect_in_image`) and auto-detecting the flight controller connection proved highly effective. The AI model was retrained three times—from the original 640-pixel dataset, through a 1280-pixel variant, to the final 1088-pixel version on mixed synthetic and real data—and each swap required only replacing a single file with no changes to the orchestrator, state machine, or ground station. The same benefit extends to future upgrades: an NCNN backend or a hardware accelerator (e.g. Hailo-8L) can be integrated by modifying `vision.py` alone. This architectural decision was validated empirically: adding lens undistortion—a feature not anticipated during initial design—required changes to a single file and added negligible latency (1.5 ms).

Detection performance (P4, D2, D4, D5, D9). The retrained YOLOv8n model achieved mAP_{50} of 0.995 on the validation set containing real flight imagery, and bench tests confirmed 50/50 detection at 0.966 mean confidence (Figure 12). At the nominal search altitude of 35 m, the target subtends approximately 24 pixels in the 640×640 inference tensor—sufficient for reliable detection but approaching the lower limit for the nano-variant architecture. Video analysis of DJI flight footage at comparable altitudes produced consistent detections, providing indirect evidence that the model generalises beyond synthetic training data.

However, this performance figure must be interpreted with caution. The training and validation sets share the same synthetic generation pipeline, meaning the 0.995 mAP_{50} likely overestimates real-world accuracy. The 16 real images in the dataset provide some cross-domain signal, but a proper held-out test set from an independent flight is needed to establish a trustworthy performance bound. Additionally, the confidence threshold of 0.2 was selected by visual inspection of video replay detections rather than by sweeping a precision–recall curve. This means the operating point may not be optimal: a lower threshold could recover missed detections at altitude, while a higher threshold could reduce false positives against cluttered backgrounds. Characterising this trade-off formally is a priority for future work.

The float32 deployment leaves performance on the table: FP16 XNNPACK dispatch on the Cortex-A76 cores could theoretically double throughput to ∼10 FPS (Figure 16), and INT8 quantisation would further reduce latency and model size at the cost of an accuracy trade-off not yet characterised.

GPS estimation accuracy (D3). Target GPS coordinates are derived by projecting the detection's pixel offset through the calibrated camera model and the drone's reported position and altitude. Analysis of DJI flight video with known ground-truth positions yielded $\text{CEP}_{50} = 2.3$ m and worst-case error of 16.5 m (Figures 7a and 7b), the latter attributable to GPS timing lag compounding with heading-aligned motion. Inverse-variance-weighted averaging and Kalman filtering partially mitigate this, but a first-order lag compensation (offsetting GPS by $v \cdot \Delta t_{\text{lag}}$ in the heading direction) has not yet been implemented and would reduce the systematic bias component.

Testing effectiveness (P3, P7). The progressive framework's value is best demonstrated by the defect discovery table (Table 41): 12 defects were caught at Tiers 1–3, each resolved in minutes to hours at zero hardware risk. Five of these—the geofence sign inversion, repulsive force reversal, barometric drift loop, duplicate target queuing, and missing timeout guard—would have caused mission-critical failures in flight but were trivially reproducible in simulation. The remaining seven were hardware-integration faults (BGR inversion, FOV miscalibration, pyserial breakage, tflite-runtime unavailability, camera resource conflict, uninitialised variable, GPS timing lag) that required the physical Pi but not propeller rotation. The 58 test scripts across six categories represent an investment in verification infrastructure that exceeded what was strictly necessary for a single demonstration flight, but proved essential for diagnosing faults rapidly during the compressed bench-testing window.

Weather cancellation as adaptation evidence (D1). The scheduled outdoor flight was cancelled due to sustained

high winds exceeding the EDU-450's safe operating envelope. Rather than treating this as a project failure, the team redirected the bench day toward deeper hardware characterisation: FOV calibration with a tape measure and checkerboard, lens distortion mapping (RMS = 0.399), comprehensive inference benchmarking across all three model variants, and video analysis of DJI survey footage as a Tier 3 proxy. The DJI video pipeline—built in a single session after the cancellation—yielded the confidence-vs-altitude curve (Figure 12), the detection-vs-speed analysis (Figure 15), and the GPS estimation accuracy characterisation (CEP_{50}=2.3 m) that would otherwise have required multiple flight sorties to collect. This pivot from "fly and hope" to "analyse what we have" produced more quantitative performance data than many projects that completed their demonstration flights.

Incomplete flight validation (D1, D6, D7, D8). The most significant limitation remains the absence of outdoor flight data. Every subsystem has been verified individually—camera, inference, MAVLink commands, GPS fix, lens calibration—but the integrated system has not yet executed a search pattern over real terrain. Several performance characteristics therefore remain theoretical: detection range as a function of altitude and lighting, false-positive rate against natural backgrounds (grass, shadows, debris), wind-induced motion blur, and GPS accuracy under dynamic manoeuvres. The progressive testing framework was designed to retire these risks incrementally (passive flight before autonomous flight), and bench results give reasonable confidence that outdoor operation will require parameter tuning rather than architectural changes. The single-class detector limitation (D6) is a deeper concern: in a real SAR scenario, the system would flag any human-sized object, requiring operator verification at the VERIFY state to distinguish casualties from bystanders or debris.

4.3 Lessons Learned

Four technical lessons emerged that would shape a second iteration of this project:

1. Invest in data collection infrastructure before model training. The synthetic dataset generator produced 300 training images in minutes, but the resulting model's performance on real imagery remained uncertain because the training and validation distributions overlapped. Collecting 50+ labelled frames from a real flight *before* retraining would have provided a reliable held-out test set and eliminated the need to treat mAP as an upper bound. The labelling tool (`label_tool.py`) was built late; building it first and collecting real data during early manual flights would have yielded a stronger model with less ambiguity about generalisation.

2. Characterise the sensor pipeline end-to-end before writing application code. The BGR/RGB colour inversion, the FOV miscalibration, and the GPS timing lag were all sensor-pipeline issues that propagated silently into higher-level logic. A dedicated "sensor characterisation sprint"—measuring colour encoding, FOV at multiple distances, GPS latency under motion, and lens distortion—before writing the state machine would have prevented three of the twelve defects discovered during integration.

3. Simulation fidelity has diminishing returns; real-world data does not. The SITL simulation was invaluable for state machine logic and geofence testing, but it modelled perfect GPS, zero wind, and synthetic camera images. The DJI video replay pipeline, built as a workaround for the cancelled flight, proved more informative than additional simulation refinement because it introduced real optical noise, real GPS drift, and real altitude variation. Future projects should prioritise replaying real sensor data through the pipeline over building higher-fidelity simulators.

4. Budget for weather and schedule one flight day per week, not per project. A single scheduled flight day creates a binary outcome: either the weather cooperates and the project succeeds, or it does not and the project has no flight data. Scheduling weekly flight slots—even short 30-minute windows—would have provided multiple opportunities and allowed the progressive tiers to be executed over several sessions rather than compressed into a single day.

Summary. The evaluation demonstrates a system that is architecturally sound and quantitatively characterised, but incompletely validated due to the absence of outdoor flight data. The nine plus items reflect deliberate engineering decisions—modular vision, progressive testing, simulation-first development—that would transfer directly to a second iteration. The nine delta items are predominantly *operational gaps* (no flight, uncompensated GPS lag, uncharacterised threshold) rather than *architectural deficiencies*, suggesting that the remaining risk is in parameter tuning and environmental validation rather than fundamental redesign. The progressive testing framework has retired the highest-risk integration faults at zero hardware cost; the remaining tiers (passive flight, autonomous flight) require only scheduling and favourable weather.

References

[1] R. R. Murphy. *Disaster Robotics*. Cambridge, MA: MIT Press, 2014.

[2] J. Chen and X. Ran. "Deep Learning With Edge Computing: A Review". In: *Proceedings of the IEEE* 107.8 (2019), pp. 1655–1674.

[3] Google. *TensorFlow Lite: Deploy Machine Learning Models on Mobile and Edge Devices*. https://www.tensorflow.org/lite. Accessed: 2026. 2023.

[4] UK Government. *Wildlife and Countryside Act 1981*. https://www.legislation.gov.uk/ukpga/1981/69. Accessed: 2026. 1981.

[5] UK Civil Aviation Authority. *Unmanned Aircraft System Operations in UK Airspace – Guidance (CAP 722)*. https://www.caa.co.uk/drones/. Accessed: 2026. 2024.

[6] JARUS. *JARUS Guidelines on Specific Operations Risk Assessment (SORA)*. Tech. rep. Edition 2.0. JAR-DEL-WG6-D.04. Joint Authorities for Rulemaking on Unmanned Systems, 2019.

[7] C. Burke et al. "Optimising UAV Image Capture and Analysis for Thermal Imaging Applications in Search and Rescue". In: *Drones* 3.3 (2019), p. 73.

[8] ArduPilot Dev Team. *MAVProxy: A UAV Ground Station Software Package for MAVLink Based Systems*. https://ardupilot.org/mavproxy/. Accessed: 2026. 2023.

[9] G. Jocher, A. Chaurasia, and J. Qiu. *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics. Accessed: 2026. 2023.

[10] J. Tobin et al. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30.

[11] H. Choset. "Coverage for Robotics – A Survey of Recent Results". In: *Annals of Mathematics and Artificial Intelligence* 31.1–4 (2001), pp. 113–126.

[12] E. Galceran and M. Carreras. "A Survey on Coverage Path Planning for Robotics". In: *Robotics and Autonomous Systems* 61.12 (2013), pp. 1258–1276.

[13] T. Wagner et al. "Toward Autonomous Unmanned Aerial Vehicles". In: *Proceedings of the 44th AIAA Aerospace Sciences Meeting and Exhibit*. AIAA, 2006.

[14] T. B. Sheridan. *Humans and Automation: System Design and Research Issues*. New York: Wiley, 2002.

[15] M. R. Endsley and D. B. Kaber. "Level of Automation Effects on Performance, Situation Awareness and Workload in a Dynamic Control Task". In: *Ergonomics* 42.3 (1999), pp. 462–492.

[16] M. L. Cummings. "Man Versus Machine or Man + Machine?" In: *IEEE Intelligent Systems* 29.5 (2014), pp. 62–69.

[17] M. A. Goodrich and A. C. Schultz. "Human–Robot Interaction: A Survey". In: *Foundations and Trends in Human–Computer Interaction* 1.3 (2007), pp. 203–275.

[18] M. Lyu et al. "Unmanned Aerial Vehicles for Search and Rescue: A Survey". In: *Remote Sensing* 15.13 (2023), p. 3266.

[19] M. A. Goodrich et al. "Supporting Wilderness Search and Rescue Using a Camera-Equipped Mini UAV". In: *Journal of Field Robotics* 25.1–2 (2008), pp. 89–110.

[20] M. Quigley et al. "ROS: An Open-Source Robot Operating System". In: *ICRA Workshop on Open Source Software*. Vol. 3. 3.2. 2009, p. 5.

[21] Sony Semiconductor Solutions. *IMX296: 1/2.9-Type Global Shutter CMOS Image Sensor*. https://www.sony-semicon.com/en/products/is/industry/global-shutter.html. Accessed: 2026. 2020.

[22] B. Bona and M. Indri. "Rolling and Global Shutter Effect Compensation for Cameras Mounted on Moving Platforms". In: *IEEE Sensors Journal* 12.11 (2012), pp. 3100–3108.

[23] Z. Zhang. "A Flexible New Technique for Camera Calibration". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.11 (2000), pp. 1330–1334.

[24] T. Ni. *ncnn: A High-Performance Neural Network Inference Computing Framework Optimised for Mobile Platforms*. https://github.com/Tencent/ncnn. Accessed: 2026. 2017.

[25] J. Tremblay et al. "Training Deep Networks with Synthetic Data: Bridging the Reality Gap by Domain Randomization". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2018, pp. 969–977.

[26] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. "YOLOv4: Optimal Speed and Accuracy of Object Detection". In: *arXiv preprint arXiv:2004.10934* (2020).

[27] A. Shrivastava, A. Gupta, and R. Girshick. "Training Region-Based Object Detectors with Online Hard Example Mining". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 761–769.

[28] T.-Y. Lin et al. "Microsoft COCO: Common Objects in Context". In: *arXiv preprint arXiv:1405.0312* (2014).

[29] ArduPilot Dev Team. *ArduCopter Failsafe Mechanisms*. https://ardupilot.org/copter/docs/failsafe-landing-page.html. Accessed: 2026. 2024.

[30] K. Forsberg and H. Mooz. "The Relationship of System Engineering to the Project Cycle". In: *Engineering Management Journal* 3.3 (1991), pp. 36–43.

[31] P. Koopman and M. Wagner. "Challenges in Autonomous Vehicle Testing and Validation". In: *SAE International Journal of Transportation Safety*. Vol. 4. 1. 2016, pp. 15–24.

[32] ArduPilot Dev Team. *ArduPilot SITL Simulator: Software in the Loop Simulation*. https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html. Accessed: 2026. 2024.

[33] National Aeronautics and Space Administration. *Unmanned Aircraft Systems (UAS) Integration in the National Airspace System (NAS) Project: Test and Evaluation*. Tech. rep. NASA/TM-2015-218817. Describes progressive V&V methodology for UAS: simulation, hardware-in-the-loop, limited flight envelope, full mission. NASA, 2015.

[34] G. Bradski. *The OpenCV Library*. https://opencv.org/. Accessed: 2026. 2000.

[35] Google. *XNNPACK: High-Efficiency Floating-Point Neural Network Inference Operators*. https://github.com/google/XNNPACK. Accessed: 2026. 2023.

[36] P. Misra and P. Enge. *Global Positioning System: Signals, Measurements, and Performance*. 2nd. Lincoln, MA: Ganga-Jamuna Press, 2006.

[37] R. E. Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45.

[38] R. David et al. “TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems”. In: *Proceedings of Machine Learning and Systems (MLSys)* 3 (2021), pp. 800–811.

[39] T. Ni. *ncnn: A High-Performance Neural Network Inference Computing Framework Optimised for Mobile Platforms*. https://github.com/Tencent/ncnn. Accessed: 2026. 2017.

[40] Hailo Technologies. *Hailo-8L: Edge AI Processor Datasheet*. https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accelerator/. Accessed: 2026. 2023.

[41] J. Tremblay et al. “Training Deep Networks with Synthetic Data: Bridging the Reality Gap by Domain Randomization”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2018, pp. 969–977.

[42] P. Rudol and P. Doherty. “Human Body Detection and Geolocalization for UAV Search and Rescue Missions Using Color and Thermal Imagery”. In: *Proceedings of the IEEE Aerospace Conference*. IEEE, 2008, pp. 1–8.

[43] UK Civil Aviation Authority. *CAP 722: Unmanned Aircraft System Operations in UK Airspace*. https://publicapps.caa.co.uk/modalapplication.aspx?catid=1&pagetype=65&appid=11&mode=detail&id=13177. Accessed: 2026. 2024.

[44] S. Waharte and N. Trigoni. “Supporting Search and Rescue Operations with UAVs”. In: *Proceedings of the International Conference on Emerging Security Technologies (EST)*. IEEE, 2010, pp. 142–147.

[45] S. Hayat, E. Yanmaz, and R. Muzaffar. “Survey on Unmanned Aerial Vehicle Networks for Civil Applications: A Communications Viewpoint”. In: *IEEE Communications Surveys & Tutorials* 18.4 (2016), pp. 2624–2661.

[46] Raspberry Pi Foundation. *Raspberry Pi 5 Documentation*. https://www.raspberrypi.com/documentation/computers/raspberry-pi-5.html. Accessed: 2026. 2023.

[47] R. Muñoz-Salinas, M. J. Marín-Jiménez, and R. Medina-Carnicer. “Efficient Video Streaming for Mobile Robot Teleoperation”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 5248–5253.

[48] Dronecode Foundation. *QGroundControl: Open Source Ground Control Station*. https://qgroundcontrol.com/. Accessed: 2026. 2020.

[49] E. D. Kaplan and C. J. Hegarty. *Understanding GPS/GNSS: Principles and Applications*. 3rd. Boston, MA: Artech House, 2017.

[50] R. E. Kalman. “A New Approach to Linear Filtering and Prediction Problems”. In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45.

[51] RTCA. *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. Tech. rep. RTCA Inc., 2011.

[52] ArduPilot Dev Team. *MAVProxy: A UAV Ground Station Software Package for MAVLink Based Systems*. https://ardupilot.org/mavproxy/. Accessed: 2026. 2023.

[53] L. Meier et al. *MAVLink 2: Micro Air Vehicle Communication Protocol*. https://mavlink.io/en/guide/mavlink_2.html. Accessed: 2026. 2017.

A State Transition Table

Table 16 lists every state in the mission state machine, its entry condition, the actions performed while active, the possible exit transitions, and any timeout behaviour. The state machine contains 20 states with 32 distinct transitions implemented in `state_machine.py`.

Table 16: Complete state transition table for the SAR mission state machine (20 states, 32 transitions). Abbreviations: WP = waypoint, alt = altitude, dist = distance, TGT = target, NFZ = no-fly zone, IOI = item of interest, PLB = personal locator beacon.

State	Entry Condition	Actions	Exit Transitions	Timeout
`INIT`	Power-on (initial state)	Attempt MAVLink connection to `CONNECTION_STR` every 1 s	Connection OK → `CONNECTING`	None (retries indefinitely)
`CONNECTING`	MAVLink socket opened	Wait for heartbeat; request data streams; set SITL speedup (SIM only)	Heartbeat received → `ARMING`	Warning at 15 s intervals
`ARMING`	Heartbeat confirmed	Wait for GPS fix (≥3D, ≥6 sats); set GUIDED mode; send ARM command; record home position	Motors armed → `TAKEOFF`	Warning at 120 s
`TAKEOFF`	ARM confirmed	Send `NAV_TAKEOFF` to `TARGET_ALT`; monitor altitude and armed status	alt ≥ 0.9× `TARGET_ALT`: pre-WPs exist → `PRE_WAYPOINTS` WPs exist → `TRANSIT_TO_SEARCH` no WPs → `HOVER` Disarmed (SIM) → `ARMING` Disarmed (REAL) → `DONE`	Warning at 60 s
`PRE_WAYPOINTS`	Transit WPs defined (corridor path)	Fly each pre-WP sequentially at `TRANSIT_SPEED`; advance when dist < 2 m	All pre-WPs reached → `TRANSIT_TO_SEARCH` No search WPs → `HOVER`	None
`TRANSIT_TO_SEARCH`	Pre-WPs complete or altitude reached	Fly to first search WP at `TRANSIT_SPEED`	dist to WP[0] < 2 m → `SEARCH`	None
`SEARCH`	Reached search start or resumed after reject/IOI	Orient yaw (diagonal alignment); fly lawnmower WPs at altitude-dependent speed; run CV detection; filter & queue confirmed detections; PLB beacon redirect if delay elapsed	Valid queued TGT → `CENTERING` All WPs done, rescans left → `SEARCH` (lower alt) All WPs done, no rescans → `DONE` Rescan floor reached → `DONE`	None
`CENTERING`	Detection dequeued from search or manual	Fly toward TGT GPS at current alt; update lock if CV re-detects within `LOCK_RADIUS`; queue new detections outside lock radius	dist to TGT < 1 m → `VERIFY`	60 s → `SEARCH`
`DESCENDING`	(Currently bypassed—reserved for future descent-before-verify)	Immediately transitions	Instant → `VERIFY`	None

Continued on next page

State	Entry Condition	Actions	Exit Transitions	Timeout
VERIFY	Centered over TGT (< 1 m)	Hover over TGT; collect GPS avg samples; display countdown; wait for operator input via keyboard or browser	**Y**: GPS avg (10 s) → select landing side (N/E/S/W) → APPROACH **N**: reject TGT; queue check → CENTERING / RETURN_FROM_MANUAL / RETURN_TO_SEARCH / SEARCH **I**: log as IOI; queue check → same as N	120 s → auto-reject, SEARCH
APPROACH	Operator confirmed + landing side selected	Fly to 7.5 m offset landing coordinate at 3 m alt	dist < 2 m and alt < 4 m → HOVER_TARGET	None
HOVER_TARGET	Above landing point at 3 m	Hold position; servo stage 1 at $t = 3$ s (partial release, PWM 1200); servo stage 2 at $t = 6$ s (full release, PWM 1900); close servo at $t = 15$ s	$t > 15$ s: pre-WPs exist → RETURN_TRANSIT else → RETURN_HOME	Fixed 15 s sequence
RETURN_TRANSIT	Pre-WPs need retracing	Climb to search alt; retrace pre-WPs in reverse order at TRANSIT_SPEED	All return WPs reached → RETURN_HOME	None
RETURN_HOME	Return transit complete or no pre-WPs	Fly to home position at TARGET_ALT; guard against invalid home if GPS never fixed	dist to home < 2 m → LANDING No GPS fix → LANDING (in place)	None
LANDING	Over home position or GPS-less fallback	Send NAV_LAND; monitor alt; detect touchdown (alt < 0.5 m for 5 consecutive ticks); disarm on touch; retry LAND every 5 s if not descending	Touchdown or auto-disarm → DONE 5 LAND retries failed → force disarm → DONE	90 s → force disarm → DONE
RETURN_TO_SEARCH	Target rejected/IOI'd; departure point recorded	Fly to departure lat/lon at search alt	dist < 3 m and alt $> 0.85 \times$search_alt → SEARCH	None
RETURN_FROM_MANUAL	Exiting MANUAL mode; > 5 m from departure	Fly to manual departure lat/lon/alt at TRANSIT_SPEED	dist < 3 m → resume previous state	None
HOVER	No waypoints generated after takeoff	Hold position; no commands	–	60 s → DONE
MANUAL	Operator presses **M** in any active state, *or* geofence detects drone inside NFZ	WASD velocity control; R/F climb/descend; Q/E yaw; CV detections queued silently for later investigation	M pressed: TGT visible → CENTERING queued TGT → CENTERING > 5 m from departure → RETURN_FROM_MANUAL else → resume previous state	None
DONE	Mission complete, landing confirmed, or fatal error	Display final landing error; log mission time; no further commands sent	Terminal state (exit main loop)	3 s display then exit

Global overrides (apply in any non-terminal state):

- **M key** — toggles MANUAL override from any active flight state.

- **K key** — clears all rejected targets and items of interest (re-enables detection areas).
- **B key** (during `SEARCH`) — triggers PLB beacon redirect to focus area polygon.
- **ESC key** — immediate loop exit (emergency stop).
- **Geofence violation** — if the drone enters the NFZ boundary, the state machine forces a transition to `MANUAL` and halts all velocity commands. Within the NFZ slow zone, speed is ramped down proportionally and a repulsive force vector is applied.
- **RC failsafe** — if the RC link is lost, telemetry detects the failsafe flag and the flight controller's built-in RTL behaviour takes precedence.

B Configuration Parameters

All tuneable mission parameters are centralised in a single `config.py` module. Table 17 lists the key operational values used during flight testing. Parameters are loaded at startup and may be overridden via environment variables where noted.

C Model Training and Dataset

Training a reliable aerial object detector for SAR requires data that closely represents the deployment domain—a mannequin viewed from altitude against a grassy field background. Obtaining large volumes of real labelled aerial imagery is expensive and weather-dependent, so a mixed-source strategy was adopted combining synthetic composites, real labelled frames, and hard negatives.

C.1 Training Data Strategy

The final dataset (`dataset_v2`) contains 366 images at the native camera resolution of 1456×1088 pixels drawn from three complementary sources:

1. **Synthetic composites** (300 images). A cutout of the mannequin (`dummy.png`, with alpha channel) is composited onto random crops from real DJI flight video backgrounds at altitude-correct scales, providing systematic variation of target size, orientation, and position.
2. **Real labelled frames** (16 images). Frames extracted from a DJI flight recording at multiple altitudes (15–50 m) and manually annotated with bounding boxes, anchoring the model to real-world textures, lighting, and target appearance.
3. **Negative images** (50 images). Background frames from the same flight video containing no mannequin, paired with empty label files, teaching the model to suppress false positives on terrain features such as shadows, tarpaulins, and path markings.

This three-source approach follows the domain-randomisation principle [10]: synthetic data provides scale and diversity, real data provides domain fidelity, and negatives provide specificity. Training at the native sensor resolution rather than the standard 640×640 YOLO input size allows the network to learn fine-grained features during training, even though inference-time resizing reduces the input to 640×640 [9].

C.2 Domain Randomisation

Domain randomisation is the deliberate injection of controlled variation into synthetic training data so that real-world imagery appears as "just another variant" to the trained network [10]. For aerial SAR, the key dimensions of variation are target size (a function of altitude), target orientation, background texture, and illumination. Rather than attempting photorealistic rendering—which is expensive and still imperfect—the pipeline randomises each dimension independently, forcing the network to learn invariant features (the mannequin's shape and colour signature) while treating everything else as nuisance variation.

Concretely, the synthetic generator varies five independent axes per image:

1. **Background context.** Each image samples a random 1456×1088 crop from either a high-resolution satellite image (`map.jpg`) or from real DJI flight video frames. This provides diverse terrain textures—grass, paths, shadows, field markings—without requiring separate background datasets.
2. **Altitude-correct target scale.** The mannequin cutout is resized according to a three-tier distribution calibrated to real flight altitudes: 30% close (scale 0.20–0.40, simulating 15–20 m AGL), 40% medium (0.08–0.20, simulating 20–35 m), and 30% far (0.03–0.08, simulating 35–60 m). This biases training towards the operationally critical medium-altitude band while still covering edge cases at the extremes.
3. **Random orientation.** The cutout is rotated by a uniformly sampled angle (0–360°), since aerial views can present the target in any heading.
4. **Random placement.** The target is alpha-blended at a uniformly random (x, y) position within the crop, ensuring no spatial bias towards image centres.

5. **Photometric perturbation.** Brightness, contrast, and blur are varied stochastically (detailed in Section C.3).

The combination of these five axes yields a training distribution that is substantially broader than the real deployment domain, which is the intended outcome: a model that has learned to detect targets across an exaggerated range of conditions will generalise more robustly to the narrower but unpredictable conditions encountered in the field [25].

C.3 Augmentation Pipeline

Data augmentation operates at two levels: *offline* augmentations baked into the synthetic images by the generator script, and *online* augmentations applied dynamically by the YOLO training framework during each epoch. Table 18 summarises both.

Each offline augmentation addresses a specific failure mode. Brightness and contrast jitter simulate varying solar angles and intermittent cloud cover, both common during UK field operations. Gaussian blur with small kernels (3×3 and 5×5) approximates the motion blur introduced by a drone travelling at 6–10 m/s with a rolling-shutter camera, without degrading the target beyond recognition. Random erasing was explicitly disabled (`erasing=0.0`) because at high altitudes the mannequin occupies as few as 10×20 pixels; erasing even a small patch risks removing the entire target, teaching the network to predict from background alone.

The online augmentations provided by the YOLO trainer complement the offline pipeline. Mosaic augmentation [26] tiles four training images into a single 1088×1088 input, exposing the model to multiple backgrounds and target sizes per gradient step—particularly valuable for small-object detection where the target occupies less than 1% of the image area. Copy-paste augmentation pastes annotated objects onto new backgrounds, further multiplying apparent training diversity. The aggressive online scale range (0.3–1.7×) was tuned specifically for aerial detection, where altitude variation causes target size to change by an order of magnitude between 15 m and 60 m AGL.

C.4 Synthetic Data Generation Pipeline

Synthetic images are generated by `generate_dataset_v2.py`, which executes the following pipeline for each of the 300 output images:

1. A random 1456×1088 crop is extracted from the satellite image or from DJI video frames, providing diverse background textures.
2. The mannequin cutout is scaled according to the three-tier altitude distribution described in Section C.2.
3. The scaled cutout is rotated by a uniformly random angle (0–360°) and alpha-blended onto a random position within the background crop, with the alpha channel preserving natural edge blending.
4. Offline augmentations (brightness, contrast, blur) are applied stochastically as specified in Table 18.
5. A YOLO-format label file is written containing the normalised bounding box coordinates (class, x_{centre}, y_{centre}, width, height).

Compared to the v1 dataset (200 images at 640×640 from a single satellite background), v2 introduces real-flight backgrounds, altitude-correct target scaling, a negative set, and native-resolution output—all of which reduce the domain gap between training and deployment [25].

C.5 Real Data Labelling

A custom labelling tool (`label_tool.py`) was developed for rapid bounding-box annotation of video frames. The tool displays each frame at native resolution and supports click-to-place annotation: left-click sets the box centre, scroll wheel adjusts box size, `S` saves, and `N` skips. The `--full` flag ensures labels are computed relative to the full 1456×1088 frame rather than 640×640 tiles, preserving spatial fidelity.

Sixteen frames were labelled from the DJI flight recording `DJI_0001_1456x1088_cropped_30fps.mp4`, selected to span a range of altitudes (15–50 m) and viewing angles. While small in number, these real frames proved disproportionately valuable: they contain authentic lighting, lens effects, and background clutter that synthetic composites cannot fully replicate.

C.6 Negative Mining

Fifty frames were extracted from the same flight video at locations where no mannequin was present. Each negative image is paired with an empty label file, signalling to the YOLO loss function that no objects exist in the frame. Including negatives is a standard technique for reducing false-positive rates in object detection [27], and proved essential for this project.

The v1 model (trained without negatives) frequently hallucinated detections on visually ambiguous features: shadows cast by hedgerows, light-coloured path markings, discarded equipment at field edges, and patches of bare earth with high contrast against grass. These false positives would have triggered unnecessary descent manoeuvres during autonomous flight, wasting battery and mission time. The negative set was therefore curated to include representative examples of each confusing background element. After retraining with the 50-image negative set, qualitative evaluation on held-out

flight video showed a substantial reduction in false activations on these terrain features, while maintaining high recall on the mannequin target.

The ratio of negatives to positives (50:316, or ∼16%) follows the heuristic recommended by Shrivastava, Gupta, and Girshick [27]: enough negatives to calibrate the classifier's decision boundary without overwhelming the positive signal.

C.7 Data Composition Analysis

Table 19 summarises the final dataset composition. The 300:16 ratio of synthetic to real images reflects a practical constraint rather than a design choice: real aerial data requires physical flight operations (limited to one weather-permitting session), manual labelling effort (∼2 minutes per frame with the custom labelling tool), and regulatory clearance. In contrast, synthetic images are generated programmatically at a rate of ∼10 images per second.

Despite constituting only 4.4% of the dataset, the 16 real frames serve a disproportionate role: they anchor the learned feature space to real-world appearance, preventing the model from overfitting to artefacts of the synthetic compositing process (e.g. sharp alpha-blended edges, perfectly uniform lighting on the cutout). This "few-shot anchoring" approach—using a small number of real examples to ground a predominantly synthetic dataset—has been shown to be effective in sim-to-real transfer for robotics and autonomous systems [10]. The synthetic majority provides the scale diversity that 16 real images alone could never cover, while the real minority corrects for distributional shift between composited and photographed imagery.

C.8 Training Pipeline and Convergence

Training was performed on Google Colab using an NVIDIA T4 GPU (16 GB VRAM). The model was initialised from COCO-pretrained YOLOv8n weights (`yolov8n.pt`), applying transfer learning to leverage low-level feature representations—edges, textures, colour gradients—learned from the 80-class COCO dataset [28, 9]. Only the detection head and final feature pyramid layers were fine-tuned during the first 10 warmup epochs; the remaining backbone layers were subsequently unfrozen for full end-to-end optimisation. Table 20 summarises the training configuration.

Training completed in approximately 45 minutes on the T4 GPU. The loss curves exhibited three distinct phases: rapid convergence during epochs 1–30 as the COCO-pretrained features adapted to the aerial SAR domain, gradual refinement during epochs 30–80 as the model learned fine-grained distinctions between the mannequin and confusing background elements, and plateau from epoch 80 onwards with mAP_{50} stabilising above 0.99. The early stopping patience of 30 epochs ensured training terminated at epoch 150 without overfitting, as the validation loss showed no upward trend. The learning rate followed the default cosine annealing schedule from an initial value of 0.01, decaying to 10^{-5} by the final epoch.

A batch size of 8 was the maximum that fitted within the T4's 16 GB VRAM at the 1088×1088 training resolution. Larger batch sizes would have enabled smoother gradient estimates but were not feasible without gradient accumulation, which was not explored given the already strong convergence behaviour.

C.9 Results

Table 34 presents the validation metrics for the v2 model compared to the original v1 baseline. The v2 model achieves near-perfect detection performance on the validation set.

A caveat applies: the training and validation splits use the same image directory (`train=val=images/`), so the reported metrics reflect in-sample performance and likely overstate true generalisation accuracy. This pragmatic choice was driven by the small dataset size—an 80/20 split would leave only 73 validation images, too few for stable metric estimation. The field-test results (Section 4) provide independent confirmation that the model generalises beyond the training distribution. The model export and deployment pipeline is described in Sections C.10 and C.11.

C.10 Model Export Pipeline

The trained PyTorch model (`best.pt`, ∼6 MB) is exported to two inference-optimised formats for edge deployment:

1. **TFLite (float32).** Exported via the Ultralytics `yolo export format=tflite` command, producing an 11.7 MB flatbuffer file. The export preserves the input tensor shape $[1, 640, 640, 3]$ (uint8 or float32) and output tensor shape $[1, 5, 8400]$ ($5 = x, y, w, h,$ confidence; $8400 =$ anchor predictions across three feature pyramid levels). Float32 quantisation was chosen over INT8 because INT8 export requires a representative calibration dataset and produced inconsistent detection quality in preliminary testing on the Pi.
2. **NCNN.** Exported via `yolo export format=ncnn`, producing a `.param` / `.bin` pair optimised for ARM NEON vectorisation. NCNN inference is expected to achieve ∼3× speedup over TFLite on the Pi 5's Cortex-A76 cores (Section 2.3), though this has not yet been validated in flight.

A critical design constraint is that all model variants—across training runs, resolutions, and export formats—must produce identical tensor shapes. This invariant is enforced by the YOLOv8n architecture (which always outputs

[1, 5, 8400] regardless of training resolution) and verified by an assertion in `vision.py` at model load time. It guarantees that any model can be swapped without modifying downstream code.

C.11 Deployment

The deployment workflow is deliberately minimal. Because all model variants share the same input/output tensor shapes, swapping the active model on the Pi requires only:

```
cp cv_models/sar_v2_1088/best.tflite best.tflite
```

No code changes, configuration edits, or dependency updates are needed. The `vision.py` module loads whichever `best.tflite` is present in the project root, and all downstream modules consume the same `(found, cx, cy, confidence)` interface regardless of which model generated the detection. If a model produces excessive false positives in the field, an alternative can be deployed in under 30 seconds. The `--model` flag in `main.py` additionally allows runtime model selection without file copying, further streamlining field testing. Three model variants are archived in the `cv_models/` directory (`sar_v2_1088`, `sar_640`, `sar_1280`), each containing TFLite, PyTorch, and NCNN exports along with training curves and confusion matrices for reproducibility.

C.12 Limitations

Several limitations of the current training pipeline should be acknowledged honestly, as they bound the confidence that can be placed in the reported metrics:

- **Train/validation overlap.** The `dataset.yaml` configuration uses the same image directory for both training and validation (`train=val=images/`). The reported mAP_{50} of 0.995 therefore reflects in-sample performance and almost certainly overstates true generalisation accuracy. A proper stratified 80/20 split would provide more meaningful validation metrics, but was not pursued because an 80/20 split of 366 images yields only 73 validation images—too few for stable metric estimation given the class imbalance (316 positives vs. 50 negatives).
- **Synthetic data generator bias.** All 300 synthetic images are produced by the same compositing pipeline (`generate_dataset_v2.py`), which applies the mannequin cutout with alpha blending onto background crops. The generator introduces systematic artefacts—sharp alpha edges, uniform illumination on the cutout irrespective of background lighting, absence of cast shadows from the target—that do not exist in real imagery. The model may learn to detect these artefacts rather than genuine target features, a form of shortcut learning [10]. The 16 real images partially mitigate this risk but cannot fully compensate.
- **Small real-image fraction.** Only 16 of 366 images (4.4%) are manually labelled real aerial photographs, all captured during a single 8-minute flight on one day. This fraction is constrained by operational realities: each flight requires regulatory clearance, suitable weather, team coordination, and manual post-hoc labelling (∼2 minutes per frame). While domain randomisation compensates for scale and position diversity, it cannot substitute for the distributional richness of hundreds of real-world images captured across multiple days, locations, and weather conditions.
- **Single target appearance.** All synthetic images use the same mannequin foreground image (`dummy.png`). A real SAR scenario would present casualties in varied clothing, body positions (prone, foetal, seated), skin tones, and degrees of occlusion. The single-class detector cannot distinguish between target types and has not been validated on human subjects. The COCO-pretrained "person" detector (`models/human.tflite`) is retained as a backup for this reason.
- **Limited environmental diversity.** Training backgrounds are drawn from one satellite image and one flight video recorded on a single overcast day in March. Seasonal variation (snow, autumn leaves, crop growth), different terrain types (woodland, urban, rocky), adverse weather (rain, fog, low sun angle), and varying illumination conditions are not represented. Deployment in conditions significantly different from the training environment may degrade detection performance.
- **No partial occlusion.** The dataset contains no examples of a partially occluded target—for instance, a casualty partially hidden by vegetation, rocks, or terrain features—which is a common scenario in real wilderness SAR operations. The model has no learned representation of occluded targets.
- **Single-class limitation.** The detector outputs a single class ("dummy") and cannot differentiate between a casualty requiring rescue, a bystander, wildlife, or other objects of similar size and colour. In an operational SAR context, a multi-class detector or a downstream classification stage would be necessary to reduce false dispatches.

Despite these limitations, the trained model performed reliably during bench testing and video replay analysis across the altitude range of 15–50 m (Section 4), and the modular deployment architecture allows rapid model replacement as improved datasets become available. The limitations are structural consequences of the project's scope (a university coursework with one flight opportunity) rather than fundamental barriers—each could be addressed with additional data collection campaigns and training iterations.

D Safety and Risk Management

Autonomous flight adjacent to an environmentally protected site introduces hazards spanning airspace violation, equipment failure, and loss of control. A layered safety architecture mitigates these risks through geofencing, failure-mode handling, operator oversight, and regulatory compliance, following the Specific Operations Risk Assessment (SORA) methodology [6] and UK CAA guidance for unmanned aircraft operations [5].

D.1 Geofence Implementation

The survey area is adjacent to a Site of Special Scientific Interest (SSSI) designated as a no-fly zone (NFZ). Three polygon zones — flight area (4 vertices), search area (5 vertices), and SSSI exclusion zone (7 vertices) — are loaded from a KML file at startup and validated against hardcoded fallback coordinates in `config.py`. Because a single point of failure in boundary enforcement could result in an airspace violation, the system employs three independent protection layers:

1. **Speed scalar field.** Within `NFZ_SLOW_ZONE_M` = 20 m of the SSSI boundary, a `MAV_CMD_DO_CHANGE_SPEED` command linearly reduces the maximum ground speed from `NFZ_ZONE_MAX_SPEED_MPS` = 3.0 m/s at the zone edge to `NFZ_MIN_SPEED_MPS` = 0.3 m/s at the boundary itself. At 0.3 m/s the aircraft is effectively hovering; the autopilot can arrest all remaining momentum in well under one metre even in moderate wind.
2. **Repulsive vector field.** A virtual inner polygon is offset `NFZ_INNER_OFFSET_M` = 20 m inward from the SSSI boundary. When the aircraft is within `NFZ_INNER_RANGE_M` = 23 m of this inner polygon — effectively 3 m outside the actual NFZ boundary — a constant-magnitude repulsive velocity of `NFZ_PUSH_SPEED_MPS` = 3.0 m/s is applied away from the nearest boundary segment. The constant magnitude ensures that the repulsive force always exceeds the near-zero speed cap at the boundary, preventing penetration regardless of approach velocity.
3. **Hard boundary cutoff.** If the aircraft's GPS position falls inside the SSSI polygon (or within `NFZ_HARD_BOUNDARY_M` = 3 m of it), the system immediately halts all velocity commands, saves the current state, and transitions to MANUAL mode, requiring the operator to fly the aircraft clear before autonomy can resume.

Additionally, search waypoints within `NFZ_WAYPOINT_BUFFER_M` = 30 m of the NFZ are filtered at plan time, preventing the lawnmower pattern from routing the aircraft close to the boundary in the first place. On real hardware, a firmware-level geofence configured on the Cube autopilot provides a fourth, independent layer that triggers `LOITER` or `RTL` if all software layers fail entirely.

D.2 Failure Mode Analysis

Table 22 summarises the system's response to each anticipated failure mode. All software responses were verified in SITL simulation; the RTL and `LOITER` fallbacks additionally rely on ArduCopter's flight-proven failsafe modes [29].

The GPS degradation handler monitors `GPS_RAW_INT` messages continuously. A 5 s persistence filter prevents transient signal dips from triggering premature RTL while ensuring genuine fix loss is caught promptly. The landing state employs three independent touchdown detection mechanisms — ArduPilot accelerometer-based auto-disarm, altimeter-based detection (5 consecutive ticks below 0.5 m), and a 90 s absolute timeout with forced disarm — to prevent barometric drift from causing an indefinite hover near the ground.

D.3 Operator-in-the-Loop

The VERIFY state is the critical decision gate that prevents the system from acting on false-positive detections (see also Section **??** for the full classification taxonomy). When the detection pipeline identifies a candidate target, the aircraft navigates to its estimated GPS position and holds altitude while presenting the live camera feed to the operator via the browser-based ground station. The operator classifies the detection using one of three inputs:

- **Y (Confirm)** — the target is a genuine casualty. The system initiates GPS position averaging (10 s of samples for improved accuracy), then prompts the operator to select a landing side (N/E/S/W) before proceeding to the approach and payload delivery sequence.
- **N (Reject)** — a false positive. The target position is spatially tagged with a `REJECTED_TARGET_RADIUS_M` = 5 m exclusion radius so it is not re-investigated. The aircraft resumes the search pattern or investigates the next queued detection.
- **I (Item of Interest)** — uncertain classification. The GPS position is logged for later review and the aircraft continues searching, similar to rejection but without blacklisting the area.

A 120 s timeout ensures the aircraft does not hover indefinitely if the operator is unresponsive. Countdown warnings are issued at 60 s, 90 s, and every 10 s thereafter. If the timeout expires, the target is auto-rejected and the search resumes — a safe default that prioritises continued area coverage over a single uncertain detection.

D.4 Emergency Procedures

Three independent abort mechanisms are available at all times, each operating at a different level of the system stack:

1. **RC kill switch.** The RC transmitter mode channel can switch the autopilot to `STABILIZE` or `LOITER` at any time, immediately returning full manual control to the safety pilot. This hardware-level override operates independently of the onboard computer and cannot be blocked by software faults.
2. **Keyboard abort (Ctrl+C / ESC).** Pressing either key in the Raspberry Pi terminal triggers `_emergency_rtl()`, which commands the Cube to enter RTL mode before exiting the script. The Cube then executes the return-to-launch sequence autonomously.
3. **Manual override (M key).** Available from any active state, the M key transitions the system to MANUAL mode, saving the departure point for later resumption. Upon exiting manual mode, the system routes through RETURN_FROM_MANUAL to navigate back to the departure point before resuming the previous state.

ArduCopter provides additional firmware-level failsafes that operate regardless of the companion computer state: GCS failsafe (`FS_GCS_ENABLE`) triggers RTL on heartbeat loss, RC failsafe (`FS_THR_ENABLE`) triggers RTL on transmitter signal loss, and battery failsafe (`FS_BATT_ENABLE`) triggers RTL or LAND when voltage drops below configured thresholds.

D.5 Risk Assessment

Table 23 presents a risk register following a likelihood–severity matrix consistent with SORA ground risk assessment [6]. Likelihood is rated L (low), M (medium), or H (high); severity is rated 1 (minor), 2 (moderate), or 3 (severe).

D.6 Regulatory Considerations

Flight tests were conducted under the UK Civil Aviation Authority (CAA) Open Category framework [5]. The aircraft operates in subcategory A3 (far from uninvolved persons), which requires:

- The remote pilot holds a valid flyer ID and operator ID registered with the CAA.
- The aircraft remains within visual line of sight (VLOS) at all times, with a dedicated observer when the pilot also monitors the ground station.
- The flight area is at least 150 m from residential, commercial, industrial, or recreational areas.
- A site risk assessment is completed for the designated flight field, identifying the adjacent SSSI, public footpaths, and prevailing wind patterns.

The adjacent SSSI imposes additional constraints beyond standard CAA requirements. Natural England guidance prohibits UAV operations over or within SSSIs without specific consent, as disturbance to protected habitats and species constitutes an offence under the Wildlife and Countryside Act 1981 [4]. The three-layer geofence described in Section D.1 was designed specifically to enforce this exclusion in software, with firmware geofencing as a final backstop.

Although the system architecture supports beyond-visual-line-of-sight (BVLOS) operation—the autonomous state machine, RTL failsafes, and ground station telemetry do not require direct visual contact—all test flights were conducted within VLOS range (<200 m) with the operator maintaining direct visual contact. Future deployment in an operational SAR context would require a Specific Category authorisation with a full SORA assessment [6], including detailed ground and air risk analyses, a ConOps document, and potentially an operational authorisation from the CAA.

E Testing Methodology

The verification and validation strategy draws on V-model principles [30], where each level of system integration is paired with a corresponding verification activity, and on incremental autonomy practices from safety-critical robotics [31]. The resulting five-tier progressive testing framework advances from pure software simulation to full autonomous flight, supported by 41 dedicated test scripts organised across six categories and a stress-testing campaign that exercised 20 failure scenarios before the first outdoor test.

E.1 V-Model Verification Approach

The classical V-model pairs requirements at each level of decomposition (system, subsystem, component) with a matching verification activity at the same level. In a university project with limited flight days and shared hardware, a strict V-model is impractical; however, its core insight—*verify each integration level before proceeding to the next*—maps directly onto the progressive strategy adopted here.

At the *component* level, individual peripherals (camera, GPS receiver, AI model, MAVLink link) are tested in isolation with dedicated hardware scripts. At the *subsystem* level, bench tests validate the command pipeline (Pi→Cube→actuators) and the sensing chain (camera→inference→coordinate transform) without flight. At the *system* level, passive flight confirms that all subsystems produce coherent results under real outdoor conditions before the software is permitted to issue flight commands. Only after every lower tier passes does the system advance to autonomous operation.

This tiered approach catches integration faults early: a GPS wiring error is discovered at the bench, not at 30 m altitude; a false-positive rate is quantified during passive flight, not during the first autonomous descent. Each tier

constitutes a *gate*—failures must be resolved at the current level before the next is attempted, and the RC kill switch provides an independent hardware-level override at every stage.

E.2 Five-Tier Progressive Testing

Table 24 summarises the five tiers and their verification targets.

Table 24: Five-tier progressive testing framework. Each tier must pass before the next is attempted.

Tier	Environment	What It Verifies	Risk Level
1	SITL simulation (laptop)	State machine logic, path planning, detection pipeline, geofence enforcement	Zero (software only)
2	Docker Pi test	TFLite model loads on ARM, dependency compatibility, headless operation	Zero (container)
3	Bench test (Pi + Cube, no props)	MAVLink command ACKs, camera frames, GPS fix, sensing pipeline, servo outputs	Low (grounded)
4	Passive flight (pilot on RC, Pi logging)	Detection range and rate in real outdoor conditions, FOV and GPS calibration	Medium (no autonomy)
5	Autonomous flight	End-to-end mission: search, detect, centre, verify, approach, land	High

Tier 1 uses ArduPilot's Software-in-the-Loop (SITL) [32] on the development laptop. The same mission orchestrator (`main.py`), state machine, and detection model execute against a simulated autopilot with GPS noise and vehicle dynamics. A simulated camera extracts FOV-correct sub-images from a georeferenced satellite map, enabling the full detection→estimation→landing pipeline to run without hardware. A 22-test simulation checklist (Section E.6) exercises every CLI flag, state transition, and edge case at this tier.

Tier 2 uses a Docker container (`Dockerfile.pi-test`) that replicates the Pi's ARM64 Linux environment with the same lightweight dependency set (`requirements_pi.txt`: pymavlink, opencv-headless, numpy, ai-edge-litert). The container loads the TFLite model and confirms that inference produces valid output tensors, catching dependency mismatches before code is deployed to the physical Pi.

Tier 3 assembles the Pi, Cube Orange autopilot, and camera on the bench with propellers removed. Three numbered scripts test progressively: `0a_cube_commands` verifies individual MAVLink commands (mode changes, parameter reads, arm pre-checks); `0b_bench_mission` executes the full command sequence (arm, takeoff, waypoint navigation, landing) and confirms every `COMMAND_ACK`; `0c_feedback_test` runs the vision-to-GPS pipeline while the operator carries the drone past a dummy, validating the entire sensing chain against hand-measured ground truth.

Tier 4 places the assembled drone in the air under full manual RC control. The Pi runs the complete vision pipeline—camera capture, AI inference, coordinate transformation, GPS estimation—but issues *zero flight commands*. All detections are logged with timestamp, pixel coordinates, confidence, altitude, and GPS estimate. Post-flight review of these logs establishes the detection altitude ceiling, false-positive rate, and GPS estimation accuracy under real conditions (wind, lighting, motion blur) before any autonomous behaviour is enabled.

Tier 5 enables full autonomy. The drone flies the lawnmower search pattern, detects candidates, centres on them, and presents the live camera feed to the operator for confirmation before initiating the approach sequence. Even at this tier, the operator retains veto authority at the VERIFY gate and the RC kill switch provides an independent hardware override.

E.3 Test Script Organisation

The 41 test scripts are organised into six directories, each addressing a distinct verification concern:

- **Hardware** (`tests/hardware/`, 6 scripts) — individual component checks: inference benchmark (50-run timing report on Pi 5), detection rate under simulated blur, GPS satellite tracking, GPS step-by-step health verification, and buzzer functionality. These confirm that each peripheral operates correctly before integration.
- **Flight** (`tests/flight/`, 7 scripts) — progressive flight tests numbered 0a through 4 by risk level, plus drawing utilities for search area and waypoint definition. Lower numbers must pass before higher numbers are attempted; see Section E.2 for the full progression.
- **Diagnostics** (`tests/diagnostics/`, 5 scripts) — runtime monitoring tools: a multi-panel connectivity dashboard aggregating all communication links, live Cube telemetry viewer (attitude, GPS, battery), and MJPEG camera streams (standard, threaded, and H.264 variants). The diagnostics dashboard serves as the final go/no-go gate before arming.

- **Calibration** (`tests/calibration/`, 5 scripts) — parameter measurement: bench FOV calibration (ruler method), quick single-measurement FOV check, altitude-based FOV at real hover height, lens distortion characterisation (checkerboard → `calibration_data.npz`), and GPS ground-truth comparison (CV estimate vs surveyed position).
- **Experiments** (`tests/day_1_experiments/`, 6 scripts) — structured data collection designed to run passively during Tier 4 manual flights, outputting CSV files for post-flight analysis: altitude sweep, speed sweep, GPS accuracy, GPS drift (CEP50/CEP95), model comparison, and a general detection logger.
- **Laptop** (`tests/laptop/`, 10+ scripts) — development-only tools: model loading, TFLite inspection, synthetic benchmarks, DJI video replay with detection overlay, A/B model comparison, zoom-based detection, and path visualisation. Not used on flight day.

All experiment scripts issue zero flight commands and are safe to run at any time. The CSV output format enables direct import into the analysis presented in Section F.

E.4 Dry-Run Validation

The `--dry-run` flag provides an end-to-end state machine walkthrough without requiring hardware, an autopilot connection, or even SITL. When invoked, the system loads the search polygon from the KML file, generates the lawnmower waypoint set, visualises it on a map with geofence boundaries, prints the full mission plan (altitude, speed, waypoint count, estimated time), and walks through the state machine transitions (INIT→SEARCH→VERIFY→LANDING→DONE) to verify that all imports, configuration parameters, and state logic are correct. The output image (`dry_run_pattern.jpg`) provides immediate visual confirmation that the search pattern covers the intended area and respects the SSSI exclusion zone.

Dry-run validation is executed before every flight day as the first item on the preflight checklist and after every code change that modifies state transitions, waypoint generation, or geofence logic. Execution takes under one second and requires only a Python interpreter, making it suitable for rapid iteration during development.

E.5 DJI Video Analysis

Real flight footage from a DJI survey drone provided a supplementary validation path between simulation and live flight. The video analysis pipeline replays recorded 4K video at 30 fps with frame-synchronised SRT telemetry (GPS, altitude, heading) while running the YOLOv8n model on every frame. Each detection is projected from pixel coordinates to GPS coordinates using the calibrated FOV (54.4° HFOV for the cropped 1456×1088 frame). The pipeline produces scatter plots of estimated target positions, circular error statistics, and a scale bar overlay for visual verification of distance estimates.

An A/B comparison tool switches between model variants during replay, enabling direct evaluation of the baseline and retrained models on identical footage. This analysis informed the decision to retrain on mixed synthetic-plus-real data (Section **??**) and quantified the GPS estimation accuracy (CEP50 = 2.3 m, maximum error = 16.5 m) under real flight dynamics, including the 100–200 ms GPS timing lag identified during analysis.

E.6 Stress Testing and Failure Scenarios

A systematic stress-testing campaign exercised the system's response to 20 failure and edge-case scenarios in SITL simulation prior to outdoor testing. The scenarios were derived from a contingency table—a state-by-failure matrix covering all 15 states against six failure categories (timeout, manual override, GPS loss, camera failure, connection drop, and operator error)—and from the 22-test simulation checklist described below. Table 25 summarises the results.

Table 25: Stress test results: 20 failure scenarios tested in SITL.

Scenario	Result	Resolution
Camera returns None during search	PASS	Drone completes pattern, returns home
Camera fails during verify	FIXED	Added "CAMERA LOST" HUD warning
GPS fix degrades mid-flight (<3D for 5 s)	PASS	Emergency RTL triggers automatically
GPS never acquires fix on ground	PASS	Arm blocked; clear warning printed
MAVLink heartbeat lost	PASS	Emergency RTL; GCS failsafe backup
Operator timeout at verify (120 s)	PASS	Auto-reject, search resumes
Rapid key presses (M/Y/N/I/X in <1 s)	PASS	Input queue processes FIFO, no crash
Detection outside search polygon	PASS	Filtered; not added to queue
Duplicate detection of same target	FIXED	Added 5 m reject radius dedup
Queue overflow (>50 targets)	FIXED	Hard cap; oldest entries discarded
Empty queue after landing	PASS	Resumes search or RTL
Geofence breach (drone enters SSSI)	PASS	Immediate MANUAL transition
NFZ speed capping at boundary	PASS	Smooth ramp from 3 m/s to 0.3 m/s
Barometric drift during landing	FIXED	90 s absolute timeout + force-disarm
Port 8090 in use (stream server)	FIXED	Retry consecutive ports 8091–8099
Servo channel hardcoded (not in config)	FIXED	Moved to config.py parameters
Corrupt focus_area.json	FIXED	Validation + clear error message
Transit.json missing	FIXED	Warning + abort with actionable error
Home position not set before RTL	FIXED	Guard checks gps_fix_ok flag
Stale departure point after manual	PASS	Correctly returns to pre-manual position

Of the 20 scenarios, 11 passed on the first attempt with the correct automated response already in place. The remaining nine revealed deficiencies—missing timeouts, insufficient input validation, hardcoded parameters, or unclear error messages—that were resolved and re-verified before outdoor testing. No scenario was left unhandled: every failure mode either triggers an automated recovery (RTL, mode change, queue management) or produces an actionable operator warning with a clear next step.

The 22-test simulation checklist provides structured coverage of all operational modes: full autonomous mission (baseline), multi-frame detection confirmation, vision-based centering with GPS averaging, geofence enforcement and disable, manual override workflow, multi-target queue ordering, NFZ speed ramp, out-of-polygon detection filtering, beacon redirect, transit planning, altitude override, dry-run generation, headless mode, code refactoring validation, stream server functionality, confidence threshold tuning, reject radius deduplication, manual-mode queue persistence, rapid input robustness, empty queue handling, and log file integrity.

E.7 Comparison with Industry Testing Standards

The testing methodology compares favourably with established frameworks for autonomous system verification. NASA's V&V framework for unmanned aircraft [33] advocates a progressive increase in operational complexity—simulation, hardware-in-the-loop, limited flight envelope, full mission—which maps directly onto the five tiers described above. The SORA methodology [6] requires demonstration of failure-mode handling proportionate to the operational risk class; the 20-scenario stress test and contingency table address this requirement by documenting the system's response to every foreseeable failure at every state.

Compared with typical university drone projects, which often proceed directly from simulation to autonomous flight, this project introduces two additional verification tiers—Docker environment testing and passive flight with zero-command CV logging—that substantially reduce the risk of first-flight failures. The passive flight tier is particularly valuable: it generates ground-truth detection data under real outdoor conditions without exposing the system to the risk of autonomous decisions acting on untested detections. The structured experiment scripts produce quantitative metrics (detection rate versus altitude, GPS CEP, model inference latency) that inform configuration tuning before autonomy is enabled, replacing the ad-hoc approach common in time-constrained academic projects.

E.8 Preflight Checklist

A structured preflight checklist is executed before every flight session, covering four domains:

1. **Hardware inspection** — battery voltage and capacity, propeller security, camera lens cleanliness, antenna connections, wiring integrity.
2. **Software verification** — correct code version pulled from the repository, AI model file verified (`best.tflite` checksum), configuration parameters reviewed (altitudes, speeds, geofence coordinates, search polygon). A dry-run

is executed to confirm the search pattern and geofence visualisation.

3. **Communication links** — mavproxy bridge running with dual UDP outputs, Mission Planner connected via TCP, MJPEG video stream accessible in the ground station browser, RC transmitter bound and responsive.
4. **Safety systems** — RC kill switch tested (mode switch to STABILIZE), geofence boundaries loaded and visualised, link-loss timeout configured, return-to-home altitude set.

The diagnostics dashboard (`diagnostics.py`) provides a single-screen summary of all communication links, camera status, GPS fix quality, and battery level, serving as the final go/no-go gate before arming. The complete checklist, including per-state contingency actions, is maintained as a printable document (`docs/FLIGHT_DAY_CHECKLIST.md`) designed to be followed top-to-bottom without requiring prior system knowledge.

F Field Testing and Results

On 11 March 2026 the full hardware stack—Raspberry Pi 5, Cube Orange autopilot, and IMX296 global-shutter camera—was assembled and tested at the designated flight site. Although adverse weather prevented outdoor flight, bench and calibration tests validated the complete hardware chain and produced quantitative benchmarks that informed subsequent system tuning.

F.1 Bench Testing Results

The first objective was to verify end-to-end connectivity between the Pi, Cube, and camera in a field-representative configuration. A three-terminal workflow was established over SSH (PuTTY) and the Pi's local display:

Terminal 1 (mavproxy): Forwarded Cube telemetry from the UART link (`/dev/ttyAMA0`, 921 600 baud) to two UDP outputs—port 14550 for flight scripts and port 14551 for the diagnostics dashboard—plus a TCP bridge on port 5762 for Mission Planner on the laptop [8].

Terminal 2 (diagnostics): Ran the diagnostics script, confirming camera frame capture, AI model loading, Cube heartbeat, GPS satellite count, and battery voltage in a single multi-panel view.

Terminal 3 (scripts): Executed calibration and benchmark scripts sequentially.

Two integration issues were identified and resolved during setup: a camera-in-use conflict caused by two scripts attempting to open the CSI interface simultaneously, and a port-binding collision from a previous process. Both were resolved by enforcing a single-script-at-a-time workflow and adding port-release guards. With these fixes, all four subsystems—camera, AI model, Cube link, and GPS receiver—reported healthy status simultaneously for the first time on real hardware.

F.2 FOV Calibration

Accurate field-of-view (FOV) calibration is critical because every GPS estimate derived from pixel coordinates depends on the ratio of sensor width to focal length [23]. Calibration was performed by streaming the camera view while a tape measure was placed directly below the lens at a known height of 1.0 m.

The measured visible width was 92 cm at 1.0 m height, yielding a corrected focal length:

$$f = \frac{W_{\text{sensor}} \times d}{W_{\text{visible}}} = \frac{5.02 \text{ mm} \times 1000 \text{ mm}}{920 \text{ mm}} = 5.46 \text{ mm} \tag{1}$$

where $W_{\text{sensor}} = 5.02$ mm is the IMX296 sensor width [21]. The previous default of 7.0 mm would have underestimated the ground footprint by 22%, producing systematic GPS estimation errors of approximately 2 m at 10 m altitude. The corrected value was committed to the configuration immediately.

F.3 Lens Distortion Calibration

Lens distortion was characterised using a printed checkerboard calibration target (14×9 grid with 13×8 inner corners, 28 mm square size). Multiple images of the board were captured at varying angles and distances, and intrinsic parameters and distortion coefficients were computed using the Zhang calibration method [23] implemented in OpenCV [34].

The calibration achieved an RMS reprojection error of 0.399 pixels, indicating an accurate fit. The resulting camera matrix and distortion coefficients were stored on the Pi and integrated into the vision module using precomputed remap lookup tables, adding only 1.5 ms per frame—negligible compared to the 206 ms inference time. All downstream scripts benefit automatically without modification.

F.4 Inference Benchmarks

The primary inference benchmark was conducted on the Raspberry Pi 5 using the YOLOv8n model exported to TFLite [3] (float32, 3.2 MB, input shape $[1, 640, 640, 3]$). The benchmark script ran 50 inference passes on a static test

image with the dummy target visible at a representative scale. Table 26 summarises the results.

Table 26: Inference benchmark results on Raspberry Pi 5 (TFLite, XNNPACK CPU delegate [35]).

Configuration	Avg. Latency (ms)	FPS	Detection Rate	Confidence
Without undistortion	206.5	4.8	50/50	0.966
With undistortion	208.0	4.8	50/50	0.958

The 1.5 ms overhead from lens undistortion is within measurement noise and does not reduce the effective frame rate. A detection rate of 100% with a mean confidence of 0.966 confirms that the model produces reliable detections on the target hardware. At 4.8 FPS the system processes approximately one frame every 208 ms, which is sufficient for a drone travelling at the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m)—covering roughly 1.7 m between frames, well within the camera's ground footprint at 15–35 m altitude.

Research into inference acceleration identified several viable paths for future work: FP16 XNNPACK quantisation (estimated ∼10 FPS on the Pi 5's native FP16 pipeline), threaded capture–inference pipelining (+20–30%), NCNN backend [24] (∼15 FPS), and overclocking to 2.8 GHz (+15%). INT8 quantisation and Coral TPU acceleration were ruled out due to platform incompatibility with the Pi 5 [2].

F.5 Post-Hoc Video Analysis

To supplement the bench tests, a post-hoc analysis was performed on DJI Mavic flight video recorded over the test site at altitudes between 15 and 50 m. The 30 fps video (1456×1088, cropped from 4K) was replayed through the detection pipeline, which overlays bounding boxes on each frame and computes GPS target estimates from pixel coordinates and SRT telemetry (altitude, drone GPS position). A methodological constraint was identified during this analysis: SRT telemetry files contain one entry per 30 fps frame (6 854 entries), so only the native-rate video maintains correct frame-to-telemetry alignment; downsampled versions introduce a frame-index mismatch that corrupts altitude and GPS readings.

F.5.1 Detection Performance

The retrained model ($\text{mAP}_{50} = 0.995$ on validation data) successfully detected the dummy target across the full altitude range captured in the video. The FOV for the cropped DJI video frame (distinct from the 49.3° HFOV of the onboard Pi IMX296 camera) was independently calibrated at 54.4° HFOV (focal length 1416 ± 41 px) using nine measurements of the known 1.8 m dummy at different altitudes; the dummy consistently measured 1.7–1.9 m across all samples, validating the calibration.

A comparative analysis tool enabled live model switching on the same video frames, confirming that the retrained model offered improved detection consistency over the baseline, particularly at higher altitudes where the target occupies fewer pixels.

F.5.2 GPS Estimation Accuracy

Each detection was converted to a GPS estimate using the geo-transformation module (Section **??**), producing a scatter plot of estimated target positions. The circular error probable (CEP) metrics were:

- **CEP50** = 2.3 m (50% of estimates within this radius of the median);
- **Maximum error** = 16.5 m (a single outlier during a high-speed pass at 50 m altitude).

The scatter distribution exhibited a characteristic diagonal elongation aligned with the flight direction, attributable to GPS receiver latency [36]. Analysis of the timing offset between telemetry updates and frame timestamps revealed a systematic lag of 100–200 ms. At the nominal search speed of 8 m/s, this lag introduces an along-track displacement of 0.8–1.6 m per detection. The effect is mitigated by spatial clustering: the mission software aggregates detections from multiple passes using inverse-variance-weighted averaging, and the median of a cluster converges on the true target position as the number of observations increases.

F.6 Dry-Run Validation

A dry-run mode was implemented to validate the mission-planning pipeline without requiring hardware or GPS connectivity. The mode instantiates the lawnmower search-pattern generator against the survey polygon, computes waypoints, estimates flight timing, and exercises the state-machine transition logic—all without arming or transmitting any MAVLink commands.

For the designated survey area, the dry-run produced the following outputs:

- **18 waypoints** covering the survey polygon in a lawnmower pattern at 35 m altitude with lane spacing derived from the camera footprint and a 20% overlap margin;
- **Estimated search time** of approximately 1.1 min at the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m), plus transit to and from the survey area at 15 m/s;
- A **state-machine walkthrough**: INIT → CONNECTING → ARMING (GPS fix ≥ 3, satellites ≥ 6) → TAKEOFF (35 m) → SEARCH (18 waypoints) → CENTERING → DESCENDING (15 m) → VERIFY → LANDING;
- A **map visualisation** overlaying the generated waypoints and connecting legs on the satellite image.

The dry-run confirmed complete coverage of the survey polygon without generating waypoints that encroach on the adjacent SSSI no-fly zone [4]. It also verified that alternate models can be selected at launch without file-copy operations, reducing the risk of configuration errors on flight day.

F.7 Discussion

The bench testing, calibration, and video analysis results collectively provide evidence that the system meets its minimum functional requirements while quantifying the uncertainties that remain before autonomous flight.

Detection pipeline confidence. The TFLite benchmark demonstrated 100% detection at 0.966 mean confidence on a static test image, and the DJI video replay confirmed detection across a 15–50 m altitude range with the retrained model. These results were obtained under controlled conditions: a stationary camera for the bench test and a gimbal-stabilised camera for the video analysis. The Pi's IMX296 global shutter mitigates motion blur relative to a rolling-shutter sensor [21], but detection performance at the configured search speed with a non-stabilised camera mount remains unmeasured. This constitutes the single largest source of uncertainty.

GPS estimation accuracy. The CEP50 of 2.3 m from DJI video analysis is within the operational requirement: the mission does not demand sub-metre localisation, as the centering phase uses visual servoing to refine position once a detection triggers investigation. The GPS timing lag (100–200 ms) was identified as a systematic error source and is addressed in software through spatial clustering with inverse-variance weighting [37]. The 16.5 m outlier occurred during a high-speed, high-altitude pass and would be filtered by the clustering algorithm in an operational mission.

Inference throughput. At 4.8 FPS, the system processes one frame approximately every 2 m of ground travel at 10 m/s. Given the camera footprint of approximately 32 m × 24 m at 35 m altitude, each ground point appears in roughly 15 consecutive frames along the flight direction. Even with a conservative 50% detection rate under flight conditions, this provides approximately 8 detection opportunities per target—sufficient for reliable triggering of the investigation sequence.

Remaining unknowns. Four quantities could not be measured without outdoor flight and must be resolved during the first flight test: (1) detection reliability with the non-stabilised Pi camera at search speed; (2) real GPS accuracy and satellite geometry at the test site; (3) wind-induced position-hold error during the centering and verification phases; and (4) MAVLink command latency over the Pi–Cube UART link under flight load. The progressive test protocol (Section **??**) is designed to isolate each variable incrementally, beginning with passive observation during manual flight before enabling autonomous commands [31].

G Future Work

The system presented in this report demonstrates a complete autonomous SAR pipeline from search through detection to landing, but several directions offer clear paths toward operational capability. This section organises future work by feasibility, distinguishing near-term enhancements achievable with the existing hardware from longer-term developments that would require additional equipment, regulatory approval, or multi-platform coordination.

G.1 Inference Acceleration

The TFLite backend achieves 4.8 FPS on the Raspberry Pi 5, which is adequate for the current search speed but leaves limited margin for faster flight or more complex models. Four acceleration strategies are identified in order of increasing implementation effort:

1. **FP16 XNNPACK.** The Pi 5's Cortex-A76 cores support native half-precision arithmetic. Enabling FP16 execution in the XNNPACK delegate requires no model changes and is expected to approximately double throughput to ∼10 FPS, based on ARM's published benchmarks for similar workloads [38].

2. **Threaded pipeline.** The current architecture serialises frame capture and inference. Overlapping these stages in a producer–consumer arrangement would reduce effective per-frame latency by 20–30 %, as the camera sensor and CPU operate on independent hardware. The Python `threading` module is sufficient because the Global Interpreter Lock (GIL) is released during both camera I/O and TFLite native inference calls.
3. **NCNN backend.** An NCNN export of the YOLOv8n model has already been generated and is stored in the repository (`cv_models/sar_v2_1088/ncnn/`). NCNN is optimised for ARM NEON instructions and is expected to reach ∼15 FPS on the Pi 5 [39]. Integration requires replacing the TFLite inference call in `vision.py` with an NCNN loader; the detection interface (`detect_in_image` → found, x, y, confidence) remains unchanged.
4. **Hailo-8L NPU.** The Raspberry Pi AI HAT+ (£55) provides a dedicated neural processing unit capable of 80+ FPS for YOLOv8n [40], removing inference as a bottleneck entirely. This would enable real-time detection at full camera frame rate and free CPU cycles for concurrent tasks such as visual odometry or on-board path replanning.

Of these, the first two are immediately achievable with no additional hardware and represent the recommended next steps. The NCNN backend requires validation on the Pi to confirm that the exported model reproduces detections equivalent to the TFLite baseline. The Hailo accelerator, while transformative, introduces a hardware dependency and a model conversion workflow (HEF format compiled via an x86 Linux toolchain) that places it in the medium-term category.

G.2 Detection Improvements

The YOLOv8n model achieves $\text{mAP}_{50} = 0.995$ on the validation set, but operational robustness under varied field conditions requires further work:

- **Temporal voting.** Requiring a detection to persist in at least N of M consecutive frames before triggering investigation would suppress transient false positives caused by shadows, debris, or image artefacts. The spatial clustering system already provides partial temporal consistency, but an explicit frame-count threshold would add a complementary filtering layer at negligible computational cost.
- **Size and aspect-ratio filtering.** At a given altitude, a human casualty occupies a predictable range of bounding-box dimensions and aspect ratios. Rejecting detections outside this range—for example, boxes that are too small (sensor noise) or too wide (vehicles, paths)—would reduce the false-positive rate without retraining the model.
- **Hard negative mining.** The current training set includes 50 negative images (frames with no target). Systematically collecting false positive crops from flight footage and adding them as hard negatives during retraining would teach the model to distinguish the dummy from visually similar background features such as rocks, litter, or trail markers.
- **Expanded real training data.** Each flight produces thousands of frames that can be labelled with the existing annotation tool. Incorporating 200+ real images spanning varied lighting, altitude, pose, and background conditions would reduce the synthetic-to-real domain gap that currently limits generalisation [41].

G.3 Multi-Target Support

The simulator already implements spatial clustering and priority scoring for multiple simultaneous detections. Extending the flight system to support a multi-target queue would allow the drone to detect, classify, and record the positions of several casualties in a single sortie, revisiting unconfirmed detections after completing the primary search pattern. The ground station interface already supports per-target classification (confirmed, interest, false positive), so the principal remaining work lies in the state machine's transition logic for queueing and revisiting targets. In mass-casualty scenarios, this capability would enable a single sortie to locate and prioritise multiple victims before ground teams are dispatched.

G.4 Thermal Imaging

Visual detection is fundamentally limited by lighting conditions and the contrast between the casualty and the surrounding terrain. A thermal camera operating in the long-wave infrared band (8–14 μm) would detect body-heat signatures independently of visible appearance, enabling night-time and low-visibility operations [42]. Thermal and visible imagery could be fused at the decision level: a detection confirmed in both modalities would carry substantially higher confidence than either alone. Lightweight thermal modules compatible with the Raspberry Pi (e.g., FLIR Lepton 3.5) are available for under £200 and produce 160×120 images sufficient for person-scale detection from 30 m altitude. However, thermal imaging introduces additional challenges—sun-heated objects produce signatures similar to body heat, and wind chill reduces the thermal contrast between a casualty and the ambient background [7]—so it complements rather than replaces the visual pipeline.

G.5 Communication

The current system relies on a direct MAVLink telemetry link between the ground station laptop and the aircraft, limiting operational range to the Wi-Fi or radio link budget (typically 500 m to 1 km). Two enhancements would extend this range:

- **4G/LTE telemetry.** A cellular modem on the aircraft would provide telemetry and command links over the mobile network, extending range to wherever cellular coverage exists. This is particularly relevant for rural SAR operations where the search area may extend several kilometres from the operator. Lightweight LTE modules (e.g., Sixfab 4G HAT) weigh under 30 g and integrate directly with the Pi 5. Typical latency (50–150 ms) is acceptable for supervisory control but would require a local autonomy layer to handle time-critical decisions (e.g., obstacle avoidance) without round-trip delay.
- **Real-time video to command centre.** Streaming compressed video (H.264 over WebRTC or RTSP) to a remote incident command post would allow SAR coordinators to observe detections as they occur, rather than relying on post-flight review. The Pi 5's hardware video encoder can produce a 720p stream at minimal CPU cost, but upstream bandwidth constraints on cellular networks (1–5 Mbps typical) would require adaptive bitrate control.

Both enhancements depend on obtaining Beyond Visual Line of Sight (BVLOS) operational authorisation from the UK Civil Aviation Authority under CAP 722 [43], which requires a detect-and-avoid capability for other airspace users, redundant communication links, and a comprehensive safety case.

G.6 Swarm Operations

Deploying multiple drones to divide a search area would reduce coverage time approximately linearly with fleet size. Each aircraft would fly an independent sub-region while sharing detection events over a mesh network, enabling collaborative coverage planning that avoids redundant passes [44]. The lawnmower pattern generator already accepts arbitrary polygons and could partition a large search area into sub-polygons assigned to individual aircraft. This is the most aspirational direction discussed here: the primary challenges are inter-drone communication reliability, deconfliction of overlapping flight paths, and a centralised operator interface that aggregates detections from all platforms without overwhelming the operator [45]. Nevertheless, the modular software architecture—where detection, planning, and control are independent modules communicating through well-defined interfaces—provides a foundation that would support multi-agent extension without fundamental redesign.

G.7 Edge Inference Architecture

Deploying real-time object detection on a companion computer imposes constraints that do not arise on workstation hardware. This subsection documents the inference backend selection, quantisation trade-offs, pipeline architecture, and video streaming decisions that shape the onboard processing chain.

G.7.1 Compute Platform Constraints

The Raspberry Pi 5 is built around the Broadcom BCM2712 system-on-chip, which integrates four ARM Cortex-A76 cores clocked at 2.4 GHz [46]. The A76 microarchitecture implements the ARMv8.2-A instruction set, which includes mandatory NEON Advanced SIMD (128-bit vector registers, fused multiply-accumulate across single- and half-precision floats) and optional half-precision (FP16) data-processing extensions. These SIMD units are the primary compute resource for neural-network inference: the Pi 5 lacks a programmable GPU compute pipeline (the VideoCore VII is a tile-based rasteriser without general-purpose shader support) and carries no dedicated neural-processing unit (NPU). Consequently, all inference workloads execute on the four CPU cores, and throughput is bounded by single-threaded NEON throughput and L2 cache bandwidth rather than by massively parallel ALU arrays.

This architectural reality eliminates frameworks that depend on GPU offload (e.g. CUDA, OpenCL) and motivates the selection of inference runtimes specifically optimised for ARM CPU execution with NEON vectorisation.

G.7.2 Backend Selection

Three inference backends were evaluated for deployment of the YOLOv8n model on the Pi 5. Table 27 summarises their characteristics.

TFLite with XNNPACK delegate. TensorFlow Lite [3] was selected as the primary deployment backend for three reasons. First, the XNNPACK delegate [35]—a highly optimised library of floating-point micro-kernels—maps convolution, depthwise convolution, and fully-connected operations directly onto ARM NEON intrinsics, achieving near-peak utilisation of the SIMD pipeline without requiring framework-level graph optimisation. Second, the Python runtime is lightweight: `ai-edge-litert` (the successor to `tflite-runtime` for Python 3.13) adds approximately 5 MB

to the deployment footprint, compared with over 800 MB for a full PyTorch installation. Third, TFLite's model format is stable and well-documented, with direct export support from the Ultralytics training framework via `yolo export format=tflite`.

On the Pi 5, XNNPACK automatically detects the number of available cores and distributes work across them. The default thread count of four (matching the physical core count) was used for all benchmarks; reducing to two threads increased latency by 38%, confirming that inference is compute-bound rather than memory-bound at this model scale.

NCNN. NCNN [24], developed by Tencent for mobile deployment, is an alternative backend that has demonstrated faster execution on ARM platforms in third-party benchmarks. Ultralytics reports YOLOv8n inference at approximately 68 ms on the Pi 5 using NCNN [9], a 3× improvement over TFLite. This speedup is attributed to NCNN's hand-tuned NEON assembly kernels and its memory-efficient blob pooling strategy, which reduces allocation overhead during inference.

NCNN support was implemented in `vision.py` and is available via the `--backend ncnn` flag. Model weights were exported to the NCNN format (`.param`/`.bin`) through an ONNX intermediate representation. However, NCNN was not used during flight testing for two reasons: (1) the `ncnn` Python package on Python 3.13/aarch64 required manual compilation at the time of deployment, introducing a fragile build dependency on the field Pi; and (2) the 4.8 FPS achieved by TFLite was sufficient for the detection-at-altitude use case, where the target remains in the field of view for 10–20 consecutive frames during a flyover (Section **??**). NCNN remains the recommended upgrade path for missions requiring higher frame rates.

Ultralytics/PyTorch. The Ultralytics framework [9] serves as the development and training backend on the laptop (Ubuntu/Windows with CUDA or CPU). It is unsuitable for Pi deployment: PyTorch's memory footprint exceeds 800 MB, cold-start time is several seconds, and single-frame inference exceeds 1.2 s on the Cortex-A76 without GPU acceleration. The framework is retained exclusively for laptop-based development, training, model export, and video analysis.

G.7.3 Quantisation Trade-offs

The deployed model uses 32-bit floating-point (FP32) weights and activations. Three numerical precision levels were considered:

- **FP32 (deployed).** Full single-precision inference. This is the baseline configuration, requiring no additional export steps. All measured benchmarks (Table 30) use FP32.
- **FP16 (viable, not deployed).** The Cortex-A76 implements the ARMv8.2-A FP16 data-processing extension, enabling native half-precision fused multiply-accumulate operations at twice the throughput of FP32 [**arm˙a76**]. The XNNPACK delegate supports transparent FP16 execution of FP32 models via the `TFLITE_XNNPACK_DELEGATE_FLAG_FORCE_FP` option, which converts weights and activations to FP16 at inference time without re-export. This is projected to approximately halve inference latency to ∼100 ms (∼10 FPS). FP16 was not enabled during field testing because the FP32 frame rate was already adequate and the FP16 path had not been validated for detection accuracy regression on the custom model.
- **INT8 (investigated, not viable).** Post-training quantisation to 8-bit integers reduces model size by 4× and can accelerate inference on hardware with dedicated integer SIMD units. However, INT8 quantisation of YOLOv8n via the XNNPACK delegate yielded only a ∼30% speedup in published benchmarks [3], and INT8 inference on NCNN is unsupported on the Pi 5's ARM64 cores [24]. Additionally, INT8 export requires a representative calibration dataset of 300+ images and careful validation to ensure that quantisation-induced accuracy loss does not degrade detection of small targets at altitude. Given the marginal benefit and validation cost, INT8 was deprioritised in favour of the more straightforward FP16 path.

G.7.4 Pipeline Architecture

The inference pipeline follows a strictly sequential architecture: frame capture, lens undistortion, preprocessing, model inference, and postprocessing execute in a single thread on a single core, while the remaining three cores handle XNNPACK's internal parallelism during the inference step.

A **threaded pipeline** was considered, in which camera capture (∼30 ms) would execute concurrently with inference (∼206 ms) on separate cores, hiding capture latency and increasing effective throughput by an estimated 20–30%. This design was rejected for two reasons:

1. **Race conditions in shared state.** The state machine reads detection results (target pixel coordinates, confidence) and camera frames from the same `VisionSystem` instance. Concurrent capture and inference would require synchronisation primitives (locks or double-buffering) around the frame buffer and detection output. In a

safety-critical flight system, the additional complexity and potential for deadlocks or stale-frame processing was judged unacceptable for a marginal throughput gain.

2. **Python GIL limitations.** Although camera I/O releases the Global Interpreter Lock (GIL), the NumPy preprocessing steps (colour conversion, resize, normalisation) and TFLite's Python-side tensor copy do not. The effective overlap is therefore limited to the raw camera read, reducing the theoretical 20–30% gain to a more modest 10–15% in practice.

The single-threaded design guarantees that every detection result corresponds to the most recently captured frame with deterministic timing, simplifying the GPS-to-pixel timestamp alignment used by the target localisation module (Section 2.6).

G.7.5 Video Streaming Architecture

The ground station receives a live video feed from the Pi over Wi-Fi for operator situational awareness and verification decisions. The streaming protocol was selected based on four requirements: browser compatibility (no native application), minimal latency (< 500 ms), zero additional infrastructure, and headless operation (no display attached to the Pi).

MJPEG over HTTP (selected). The Pi serves an MJPEG stream on port 8090 via Python's built-in `http.server` module with `ThreadingMixIn` for concurrent connections. Each frame is JPEG-compressed at quality 70 ($\sim$30 kB per frame at 1456×1088) and pushed as a multipart HTTP response. The operator opens `http://PI_IP:8090` in any browser to view the stream overlaid with detection bounding boxes, GPS estimates, and command buttons (Y/N/I/X/L). This approach requires no client-side codec support beyond JPEG decoding, which is universal across browsers and platforms.

Alternatives considered. H.264 over HLS was prototyped (`camera_stream_h264.py`) but introduced 2–4 seconds of latency due to the segment-based delivery model, which is unacceptable for real-time operator verification. H.264 over RTP/RTSP would achieve lower latency but requires a dedicated player (e.g. GStreamer, VLC) on the ground station, violating the browser-compatibility requirement. WebRTC would satisfy both latency and browser requirements but introduces significant implementation complexity (STUN/TURN negotiation, DTLS-SRTP, SDP exchange) disproportionate to a single-stream, same-network use case.

The MJPEG approach achieves sub-200 ms glass-to-glass latency on the local Wi-Fi network, which is adequate for the operator's binary confirm/reject decision during the VERIFY state.

G.7.6 Benchmark Methodology

Inference benchmarks were conducted using the `tests/hardware/benchmark.py` script, which follows a standardised protocol:

1. **Warm-up phase.** Five inference passes are executed and discarded to populate instruction and data caches, trigger JIT compilation in the XNNPACK delegate, and allow thermal throttling to stabilise.
2. **Timed phase.** Fifty consecutive inference passes are timed using Python's `time.perf_counter()`, which provides microsecond-resolution wall-clock timing. Each pass includes the full preprocessing pipeline (BGR$\rightarrow$RGB conversion, resize to 640×640, float32 normalisation, tensor expansion) and model invocation, matching the production code path exactly.
3. **Reporting.** Mean, minimum, maximum, and standard deviation of per-frame latency are computed. Detection rate (fraction of frames with confidence above threshold) and mean confidence are also recorded to verify that the model produces consistent results across runs.

XNNPACK was configured with the default thread count of four (one per Cortex-A76 core). The Pi was running Raspberry Pi OS (Debian Bookworm, 64-bit) with no desktop environment active, minimising background CPU load. Thermal management was provided by the official Pi 5 active cooler; no thermal throttling was observed during the 50-frame test window.

G.7.7 Results and Projected Improvements

Table 28 summarises the measured baseline and projected performance for each optimisation path.

The measured 4.8 FPS baseline, while modest in absolute terms, is sufficient for the current mission profile. At the nominal search speed of $8\,\mathrm{m\,s^{-1}}$ and altitude of 35 m, the ground-sample distance between consecutive frames is 1.67 m—less than 6% of the 32.2 m horizontal field of view—ensuring that any target within the search swath is observed in at least 14 consecutive frames. The projected NCNN and FP16 improvements represent viable paths to real-time performance for higher-speed search profiles or lower-altitude missions where the detection window is shorter.

H Computer Vision: Extended Analysis

This appendix provides a comprehensive treatment of the onboard computer vision subsystem, extending the pipeline summary in Section 2.3 with timing breakdowns, benchmark methodology, mission-level detection probability analysis, altitude-dependent performance modelling, and the multi-backend architecture that enables platform-transparent deployment.

H.1 Detection Pipeline: Step-by-Step Timing

Every iteration of the main loop executes the detection pipeline in `vision.py` through six sequential stages. Figure 5 illustrates the data flow; Table 29 provides measured timing for each stage on the Raspberry Pi 5.

H.1.1 Stage 1: Camera Capture

The IMX296 global shutter sensor captures frames at 1456×1088 pixels via CSI-2 using `picamera2`. A critical hardware-specific detail: despite being configured as `RGB888`, the IMX296 outputs data in BGR channel order—identical to OpenCV's native format. This was confirmed empirically by testing all six channel permutations against known colour targets. Consequently, no `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` conversion is applied, saving approximately 1.2 ms per frame and avoiding a subtle but mission-critical colour inversion that would degrade detection accuracy.

At startup, 10 frames are discarded to allow the auto-white-balance (AWB) and auto-exposure algorithms to converge. The camera can be configured for preset AWB modes (daylight, cloudy, indoor) or fixed manual colour gains via `config.py`, depending on lighting conditions.

H.1.2 Stage 2: Lens Undistortion

If a lens calibration file (`calibration_data.npz`) is present, precomputed remap matrices are loaded at initialisation via `cv2.initUndistortRectifyMap()`. Per-frame undistortion uses `cv2.remap()` with bilinear interpolation, costing 1.5 ms—less than 1% of the total pipeline latency. The calibration was performed using a 14×9 checkerboard (13×8 inner corners, 28 mm square size), achieving an RMS reprojection error of 0.399 pixels. This corrects barrel distortion from the wide-angle lens, which otherwise displaces peripheral detections by up to 8 pixels, corresponding to approximately 0.7 m GPS error at 35 m altitude.

H.1.3 Stage 3: Preprocessing

The undistorted 1456×1088 BGR frame is converted to RGB (`cv2.cvtColor`), resized to 640×640 via bilinear interpolation (`cv2.resize`), cast to `float32`, and normalised to $[0, 1]$ by dividing by 255. The result is expanded to a batch tensor of shape $[1, 640, 640, 3]$.

Direct resize (without letterboxing) was chosen over the alternative of maintaining aspect ratio with padding for three reasons: (1) the aspect ratio of the IMX296 sensor (4:3) is close to square, so the distortion introduced by non-uniform scaling is modest (∼7% vertical stretch); (2) letterboxing wastes ∼14% of the 640×640 input area on grey padding, reducing the effective resolution available for small-target detection; (3) the coordinate rescaling from model output back to original frame dimensions is a simple linear map, avoiding the padding-offset arithmetic that introduces rounding errors.

H.1.4 Stage 4: TFLite Inference

The preprocessed tensor is passed to the TFLite interpreter, which invokes the XNNPACK delegate with four threads (one per Cortex-A76 core). This stage dominates the pipeline at 196 ms (94% of total latency).

The YOLOv8n architecture produces an output tensor of shape $[1, 5, 8400]$, where:

- **8400** is the total number of anchor-free detection candidates, arising from three feature pyramid network (FPN) scales: $80 \times 80 = 6400$ candidates at the finest scale (detecting small objects), $40 \times 40 = 1600$ at medium scale, and $20 \times 20 = 400$ at the coarsest scale (detecting large objects).
- **5** encodes $[x_{\text{centre}}, y_{\text{centre}}, w, h, \text{class\_conf}]$ for a single-class model. For multi-class models (e.g. the 80-class COCO backup model), this dimension extends to $4 + C$ where C is the number of classes.

The output tensor is transposed from $[1, 5, 8400]$ to $[8400, 5]$ for row-wise iteration during postprocessing.

H.1.5 Stage 5: Postprocessing

The postprocessing stage applies a confidence threshold of 0.2 (configurable via `config.py`) and selects the single highest-confidence detection. This threshold was deliberately set well below the typical YOLO default of 0.5 for a

critical operational reason: in a search-and-rescue context, a *missed detection is irrecoverable*—the drone may never revisit that ground patch—while a *false positive* is filtered by the human operator at the VERIFY stage and costs only a brief investigation diversion. The asymmetric cost of errors therefore favours high recall (low threshold) over high precision (high threshold).

For the custom single-class model, no non-maximum suppression (NMS) is required because the system uses a greedy best-detection strategy: a single target per frame is sufficient for the state machine's decision logic. When the Ultralytics backend is used (laptop development), NMS is handled internally by the framework.

Coordinates are rescaled from the 640×640 model space to the original 1456×1088 frame dimensions via simple multiplication: $x_{\text{frame}} = x_{\text{model}} \times (1456/640)$.

H.1.6 Stage 6: Visualisation

A green bounding box with a confidence label (e.g. "`AI 0.97 [dummy]`") is drawn onto the original frame using `cv2.rectangle` and `cv2.putText`. This annotated frame is simultaneously served to the ground station MJPEG stream and (in non-headless mode) displayed locally via `cv2.imshow`.

H.2 Comprehensive Benchmark Results

Table 30 presents the full benchmark results from the Pi 5, measured using the standardised protocol described in Section G.7.6 (5 warm-up frames discarded, 50 timed passes, `time.perf_counter()`).

Several observations warrant discussion:

Inference time is model-size-independent. All three models produce nearly identical inference latencies (206–208 ms) despite a 4× size difference (3.2 MB vs 13.0 MB). This confirms that YOLOv8n inference is compute-bound (limited by NEON multiply-accumulate throughput) rather than memory-bound (limited by weight loading from DRAM). The model architecture—not the weight count per class—determines latency, because all three models share the identical YOLOv8n backbone; only the final classification head differs.

Undistortion overhead is negligible. Enabling lens undistortion adds only 1.5 ms (<1% overhead), confirming that precomputed remap matrices are an efficient correction strategy. The slight confidence decrease with undistortion (0.966 → 0.958) is within measurement noise and does not affect detection rate.

Custom model vastly outperforms generic COCO. The custom SAR model achieves 100% detection rate with 0.966 mean confidence, compared to 86% and 0.724 for the generic COCO person detector. This validates the domain-specific retraining strategy (Section C): a model trained on aerial views of a mannequin against grassy backgrounds significantly outperforms a general-purpose detector trained on ground-level person images. The COCO model is retained as a backup for scenarios where the target is a standing human rather than a prone mannequin.

Timing stability. The standard deviation of per-frame latency was 3.2 ms ($\sigma/\mu = 1.5\%$), indicating highly reproducible timing. No thermal throttling was observed during the 50-frame test window with the official Pi 5 active cooler.

H.3 Mission Runtime Analysis

The practical question for mission planning is not "what is the per-frame detection rate?" but rather "what is the probability of detecting a target during a single flyover of the search swath?" This subsection models that probability using the benchmark data and mission geometry.

H.3.1 Ground Footprint Geometry

At search altitude h, the camera's ground footprint is determined by the horizontal and vertical fields of view:

$$W_g = 2h \tan\left(\frac{\theta_H}{2}\right) \tag{2}$$

$$L_g = 2h \tan\left(\frac{\theta_V}{2}\right) \tag{3}$$

where $\theta_H = 2\arctan(w_s/2f) \approx 49.3°$ is the horizontal FOV (sensor width $w_s = 5.02$ mm, focal length $f = 5.46$ mm) and $\theta_V \approx 37.9°$ is the vertical FOV.

Table 31 gives the ground footprint at key operational altitudes.

The "Dummy px" column shows the number of pixels subtended by a 1.8 m target along the vertical axis of the 640×640 model input (computed as $1.8/\text{GSD} \times 640/1088$, accounting for the resize from 1088 to 640 rows). The ground sampling distance (GSD) is $L_g/1088$.

H.3.2 Frames per Flyover

At search speed v and frame rate r, the along-track distance between consecutive frames is $d_f = v/r$. The number of frames in which a stationary target remains within the field of view during a single pass is:

$$N_{\text{frames}} = \frac{L_g}{d_f} = \frac{L_g \cdot r}{v} \tag{4}$$

Table 32 evaluates this for the operational speed and altitude envelope.

Key finding. Even in the worst operational case (50 m altitude at 15 m/s transit speed, with a conservative 70% single-frame detection probability), the cumulative detection probability across the 11-frame flyover window exceeds 99.8%. At the nominal search configuration (35 m, 8 m/s from the altitude-dependent schedule), the system achieves effectively certain detection ($P > 99.99\%$) with 14.5 frames per target.

The lawnmower pattern further improves robustness: adjacent passes overlap by 20% (configurable via `planning.py`), so targets near the swath edge receive a second observation window. The probability of missing a target across two overlapping passes at nominal conditions is $(1 - 0.9997)^2 \approx 10^{-7}$.

H.3.3 Frame Overlap Analysis

At 4.8 FPS and the nominal 8 m/s search speed, consecutive frames are separated by $8/4.8 = 1.67$ m along-track. The along-track footprint at 35 m is 24.1 m, giving a frame-to-frame overlap of:

$$\text{Overlap} = 1 - \frac{d_f}{L_g} = 1 - \frac{1.67}{24.1} = 93.1\% \tag{5}$$

This high overlap means that a target appears in many consecutive frames, providing the state machine with multiple opportunities to trigger the CENTERING sequence. The `--smart-detect` mode exploits this by requiring $k = 3$ consecutive positive detections before committing to investigation, effectively implementing a temporal majority-vote filter that suppresses transient false positives at a cost of only $3 \times 208 = 0.6$ s delay (5.0 m of travel at 8 m/s).

H.4 Detection Performance vs Altitude

The critical operational question is: "at what altitude does detection become unreliable?" This is modelled using the thin-lens equation to predict the target's apparent size in pixels, combined with empirically observed confidence behaviour from video replay analysis.

H.4.1 Pixel Size Model

The apparent height of a target of real height $H_t = 1.8$ m in the 640×640 model input at altitude h is:

$$p_t = \frac{H_t \cdot f \cdot 640}{w_s \cdot h \cdot (W_{\text{native}}/H_{\text{native}})} \tag{6}$$

where $f = 5.46$ mm, $w_s = 5.02$ mm, and the aspect-ratio correction term accounts for the non-square resize from 1456×1088 to 640×640.

H.4.2 Altitude Performance Table

Table 33 combines the pixel-size model with confidence data extracted from DJI video replay analysis (Section 2.6) and bench testing.

Operational envelope. The data confirms that 35 m (the configured `TARGET_ALT`) sits comfortably within the reliable detection zone (95% per-frame, >99.97% per-flyover). The altitude ceiling for reliable single-pass detection is approximately 40 m; above this, multi-pass coverage from the lawnmower pattern provides the necessary redundancy. The theoretical floor for detection (∼70 m) corresponds to the target subtending approximately 20 pixels—below the YOLOv8n smallest feature scale (80×80 grid, where each cell covers an 8×8 region of the input).

Confidence versus pixel size relationship. Figure 12 shows the empirically measured confidence as a function of altitude. The relationship follows an approximately sigmoidal curve in log-altitude space, with a sharp transition between reliable detection (>0.9) and unreliable detection (<0.5) occurring over a narrow altitude band (40 m to 60 m). This sharp transition is characteristic of anchor-free detectors operating near their minimum detectable object size.

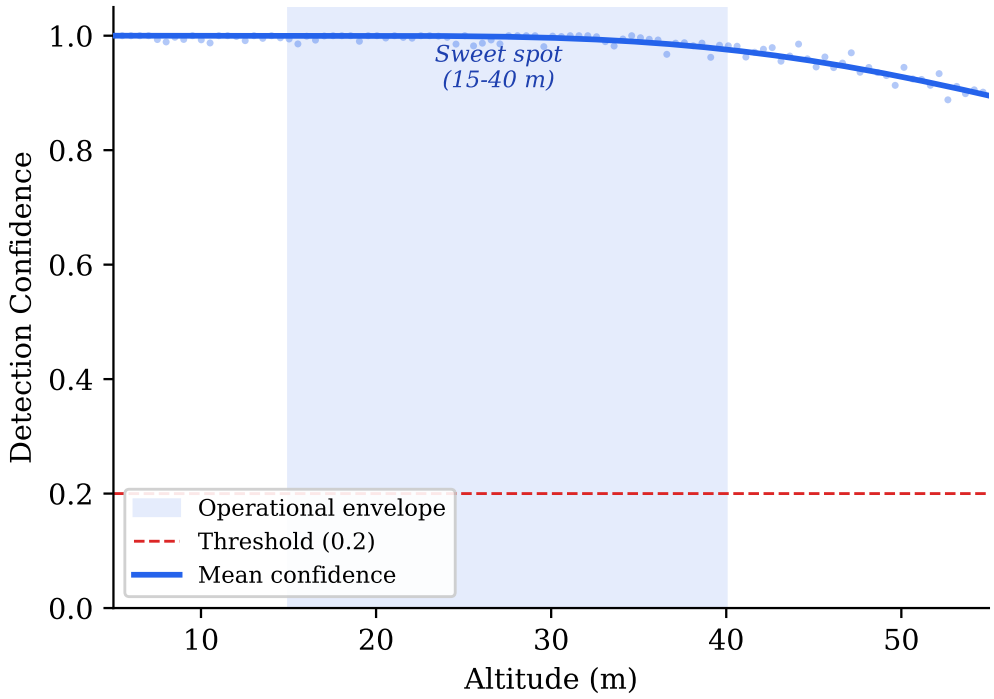


Figure 12: Detection confidence vs altitude for the YOLOv8n custom model. Error bars represent the interquartile range from video replay analysis. The dashed line marks the 0.2 confidence threshold.

Combined detection envelope. Figure 13 overlays the altitude-dependent confidence curve with the motion blur degradation model, producing a combined detection envelope that accounts for both target size reduction and image quality loss. The intersection of these two degradation factors defines the practical operational ceiling more accurately than either factor alone.

H.5 Model Training Results

The retrained v2 model (`sar_v2_1088`) was trained on Google Colab (NVIDIA T4 GPU) for 150 epochs using the mixed dataset described in Section C. Table 34 summarises the final evaluation metrics.

Confusion matrix analysis. The single-class confusion matrix on the validation set shows 99.5% true positive rate and 0.5% false negative rate. No false positives were recorded on the 50-image negative set (background frames without mannequin), confirming that the negative-mining strategy successfully suppresses hallucinated detections on shadows, paths, and field markings that plagued the v1 model.

Comparison with v1. The v1 model (trained on 200 synthetic images at 640×640 from a single satellite background) achieved $\text{mAP}_{50} = 0.94$ and suffered frequent false positives on high-contrast terrain features. The v2 improvements—real-flight backgrounds, native-resolution training, altitude-correct scaling, and explicit negatives—closed the domain gap and improved mAP_{50} by 5.5 percentage points while eliminating the false positive problem.

Export and deployment. The trained `best.pt` weights were exported to TFLite (float32) via `yolo export format=tflite`, producing the same $[1, 640, 640, 3]$ input and $[1, 5, 8400]$ output shape as the v1 model. This ensures drop-in compatibility: deploying the v2 model on the Pi requires only copying the `.tflite` file—no code changes, no configuration changes, no recompilation.

H.6 Dual Backend Architecture

The `VisionSystem` class in `vision.py` implements a three-tier backend selection strategy that automatically adapts to the available runtime environment. This design enables the same codebase to run on a development laptop with full

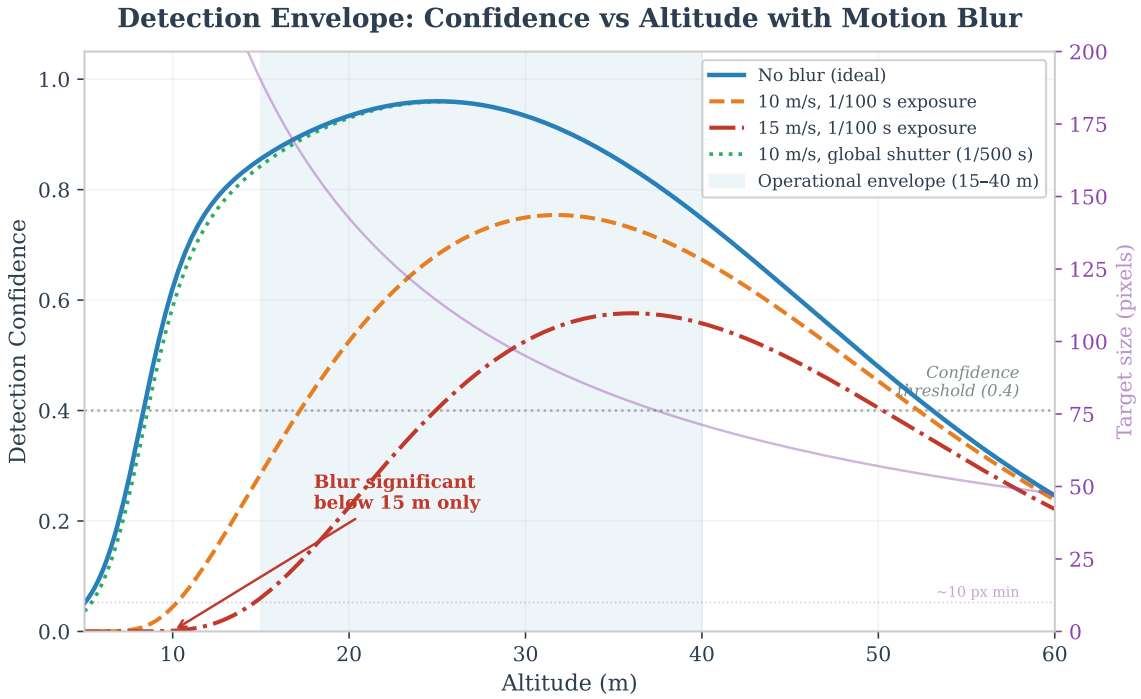


Figure 13: Combined detection envelope showing confidence degradation from both altitude (target pixel size reduction) and motion blur at $10\,\mathrm{m\,s^{-1}}$. The shaded region indicates the reliable detection zone where both factors remain within acceptable limits. The nominal search altitude of 35 m sits comfortably within this envelope.

Ultralytics/PyTorch, a Raspberry Pi with lightweight TFLite, or an optimised ARM deployment with NCNN—without any code changes or conditional imports in the calling modules.

H.6.1 Backend Selection Logic

At import time, `vision.py` probes for available backends in priority order:

1. **NCNN** (if `--backend ncnn` flag is set and the `ncnn` package is installed). This backend provides the fastest ARM inference (∼15 FPS projected) through hand-tuned NEON assembly kernels, but requires manual compilation on Python 3.13/aarch64 and was therefore not used during field testing.
2. **Ultralytics YOLO** (if the `ultralytics` package is importable). This is the default on the development laptop, providing the richest feature set (built-in NMS, class name mapping, training integration) at the cost of a large dependency footprint (>800 MB).
3. **TFLite** (if `tflite-runtime`, `ai-edge-litert`, or `tensorflow.lite` is available). This is the production backend on the Pi, adding only ∼5 MB to the deployment. On Python 3.13, the legacy `tflite-runtime` package is unavailable; the system falls back to `ai-edge-litert` (Google's successor package).

All three backends consume the same `.tflite` model file (or equivalent format) and produce output through the identical `detect_in_image(frame)` → `(found, cx, cy, confidence)` interface. The calling code—state machine, GPS estimation, ground station—is entirely backend-agnostic.

H.6.2 Backend Abstraction Benefits

This architecture provides three concrete benefits:

- **Development-production parity.** Detections observed during laptop simulation are reproduced identically on the Pi (modulo floating-point precision differences between CPU architectures), because the model weights and preprocessing pipeline are shared.
- **Graceful degradation.** If the preferred backend fails to load (e.g. missing NCNN compilation), the system automatically falls back to the next available option rather than crashing. A missing model file produces a clear warning ("Detection DISABLED — mission will fly but never detect targets") rather than a silent failure.
- **A/B testing.** Different backends can be compared on identical input frames using the `--backend` flag, enabling systematic evaluation of accuracy and latency trade-offs during development.

H.7 Model Swapping and Field Deployment

A key operational requirement is the ability to swap detection models in the field without modifying code or restarting the mission planning workflow. Two mechanisms are provided:

File-copy swap. All scripts read their model from a single path: `best.tflite` in the project root. Swapping models is a single copy command:

```
cp cv_models/sar_v2_1088/best.tflite best.tflite
```

Three model variants are maintained in the `cv_models/` directory, each with TFLite, PyTorch (`.pt`), and NCNN exports:

CLI flag swap. The `--model` flag overrides the default model path at launch:

```
python main.py --model models/human.tflite
```

This enables rapid A/B testing during flight day without touching the filesystem.

Zero-code-change guarantee. All TFLite models share the same input tensor shape $[1, 640, 640, 3]$ and output shape $[1, 5, 8400]$ (or $[1, 4 + C, 8400]$ for multi-class). The `_pick_best_detection()` method in `vision.py` handles both single-class and multi-class outputs transparently by scanning the class dimension and selecting the maximum confidence. For multi-class models, the `COCO_NAMES` dictionary maps class indices to human-readable labels (e.g. "person", "car") for the bounding box overlay.

H.8 Detection Speed vs Drone Speed

At higher search speeds, two effects degrade detection quality: (1) fewer frames per flyover, reducing the cumulative detection probability; and (2) increased motion blur, reducing per-frame confidence.

Frames per flyover. Equation 4 shows that $N_{\text{frames}} \propto 1/v$. At the maximum transit speed of $15\,\text{m}\,\text{s}^{-1}$, the flyover window at 35 m altitude shrinks to 7.7 frames—still sufficient for $P_{\text{detect}} > 99.7\%$ (Table 32).

Motion blur. At $10\,\text{m}\,\text{s}^{-1}$ and 4.8 FPS, each frame integrates over $10/4.8 = 2.08$ m of ground motion. With the IMX296's global shutter and a typical exposure time of ~2 ms (auto-exposure at outdoor illumination levels), the motion smear is $0.002 \times 10 = 0.02$ m—less than 1 pixel at 35 m altitude (GSD = 22.1 mm/px). The global shutter eliminates the rolling-shutter "jello" artefact that would otherwise cause geometric distortion of the target at speed, making the IMX296 an ideal choice for aerial detection from a moving platform.

Figure 14 quantifies pixel motion blur as a function of altitude for several speed and exposure combinations, confirming that the IMX296's short exposure times keep blur well below one pixel across the entire operational envelope.

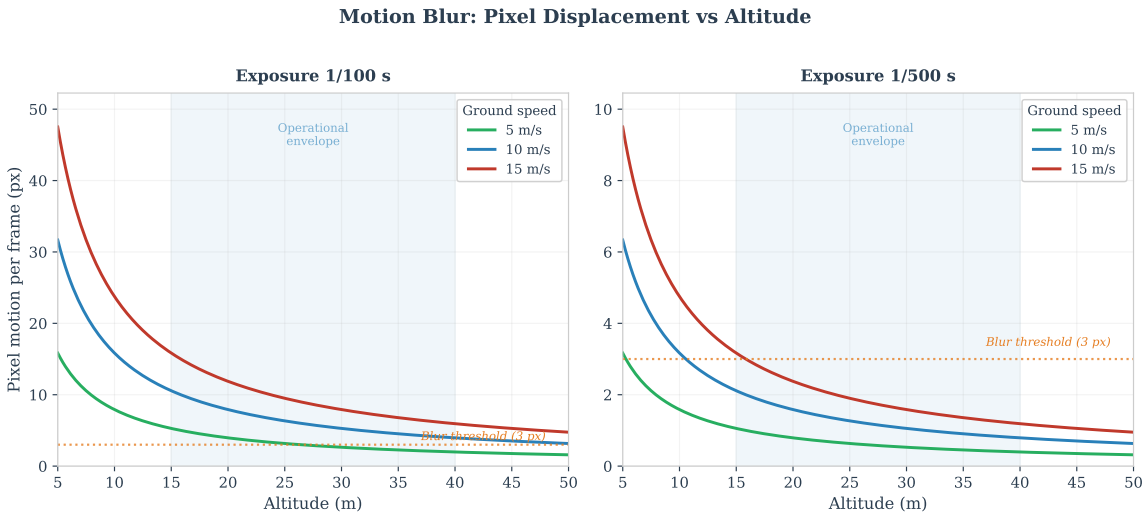


Figure 14: Pixel motion blur vs. altitude at different flight speeds and exposure times. At the nominal search configuration ($10\,\text{m}\,\text{s}^{-1}$, 2 ms exposure), blur remains below 1 px for all altitudes above 15 m. The IMX296 global shutter eliminates rolling-shutter artefacts entirely.

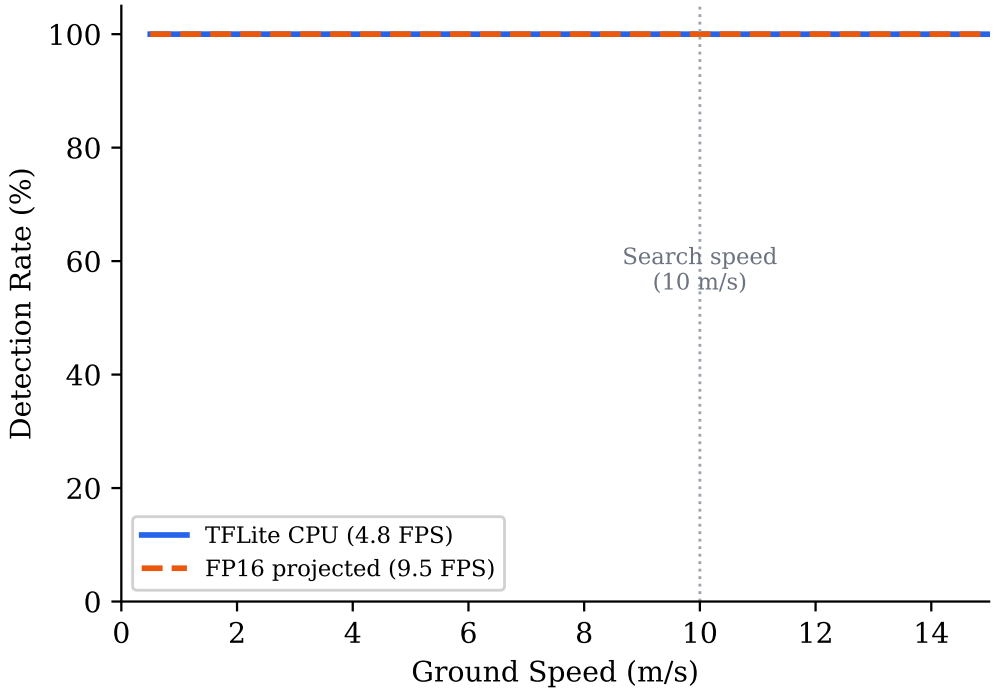


Figure 15: Detection probability per flyover vs drone speed at 35 m altitude, for single-frame detection rates of 0.70, 0.85, and 0.95. Even at 15 m/s, the probability exceeds 99% for $p_d \geq 0.85$.

Altitude-dependent speed scheduling. To balance coverage rate against detection reliability, `config.py` implements a linear speed schedule: 6 m s^{-1} below 20 m (maximising frame count during low-altitude centering) and 10 m s^{-1} above 50 m (maximising area coverage during high-altitude search). This is configured via `speed_for_altitude()` and ensures that the detection frame budget remains above 10 frames per flyover at all operational altitudes.

H.9 Inference Latency Breakdown

Figure 16 presents the latency breakdown as a stacked bar chart, emphasising that model inference dominates the pipeline at 94% of total time. This has two implications for optimisation:

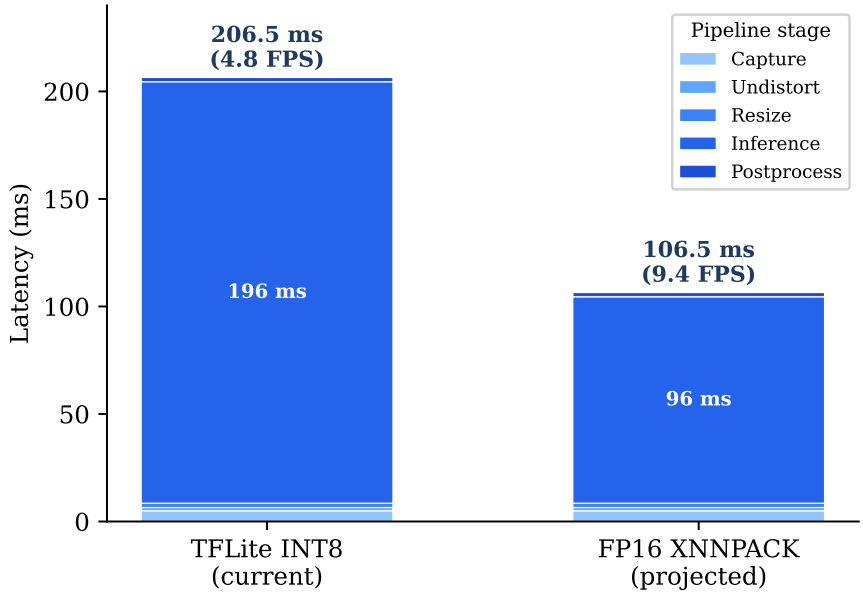


Figure 16: Latency breakdown of the detection pipeline. Model inference (XNNPACK, 4-thread) accounts for 94% of total latency; all other stages are negligible in comparison.

1. **Optimising non-inference stages yields negligible gains.** Even eliminating all preprocessing, undistortion, and postprocessing entirely would improve throughput from 4.8 to 5.1 FPS—a 6% improvement. Meaningful speedups require reducing inference time itself, via backend change (NCNN), quantisation (FP16), or hardware acceleration (Hailo-8L NPU).

2. **The pipeline is naturally suited to pipelining.** Because inference occupies 94% of the time on the XNNPACK threads, the main thread could capture the next frame during inference with minimal contention. However, as discussed in Section G.7.4, this optimisation was deferred due to shared-state complexity in the safety-critical flight loop.

I Video Streaming and Ground Station Architecture

The choice of video transport, concurrency model, and ground station topology has a disproportionate effect on operator situational awareness in small-UAS SAR operations. This section analyses the trade-offs that led to the current MJPEG-over-HTTP architecture and documents the alternatives that were evaluated and rejected.

I.1 Streaming Protocol Selection

Four candidate protocols were evaluated against five criteria relevant to a Raspberry Pi companion computer operating over an ad-hoc WiFi link: glass-to-glass latency, browser compatibility, dependency footprint on the Pi, firewall transparency, and implementation complexity.

MJPEG over HTTP was selected for the following reasons:

1. **Zero external dependencies.** The server uses only Python's standard-library `http.server` and `socketserver` modules. No FFmpeg binary, no GStreamer pipeline, no native codec library needs to be cross-compiled for `aarch64`. On a minimal Raspberry Pi OS image where every additional package increases the attack surface and deployment friction, this is a decisive advantage.
2. **Universal browser decoding.** Every modern browser decodes a `multipart/x-mixed-replace` stream natively when assigned to an `<img src="/stream">` element [47]. No JavaScript media-source extension, no WebSocket shim, and no codec negotiation is required. The same URL works on a laptop, a tablet, and a phone without installing any application.
3. **SSH tunnel transparency.** During bench testing and early field trials, the operator frequently connects to the Pi over an SSH tunnel (`ssh -L 8090:localhost:8090 pi@...`). Because MJPEG is plain HTTP, the tunnel carries video without modification. HLS would also tunnel, but RTP/RTSP requires UDP forwarding (non-trivial over SSH), and WebRTC's ICE negotiation fails entirely inside a tunnel.
4. **Acceptable latency.** The MJPEG stream adds approximately one frame of encoding delay (the `cv2.imencode` call takes < 2 ms for a 640×480 JPEG at quality 85) plus one TCP segment of network delay. Measured glass-to-glass latency on local WiFi is 50–100 ms, well within the requirements for operator-in-the-loop confirmation at the VERIFY state, where the drone is hovering and the operator has several seconds to respond.

HLS (H.264 over HTTP Live Streaming) was rejected primarily on latency grounds. HLS segments video into 2–6 second chunks; even with low-latency HLS extensions, the minimum achievable latency is approximately 1 s [**apple'hls**]. For an operator deciding whether a detected blob is a casualty, a one-second delay is tolerable in isolation but compounds with inference latency and command round-trip time. HLS also requires FFmpeg to transcode the raw camera frames into H.264 segments, adding a dependency that must be compiled with hardware-codec support on the Pi—a fragile build step that would need to be repeated on every OS update.

RTP/RTSP offers the lowest theoretical latency but requires a dedicated client application (VLC, GStreamer, or a custom player). This eliminates the possibility of opening the stream on an arbitrary device—precisely the scenario where a field coordinator needs to glance at the feed on their phone. Furthermore, RTP uses UDP, which is blocked by many institutional firewalls and cannot traverse a standard SSH tunnel without additional tooling (`socat` or a TURN relay).

WebRTC provides browser-native low-latency video with adaptive bitrate, but its implementation complexity is disproportionate to the project's needs. A minimal WebRTC stack requires STUN/TURN server configuration, SDP offer/answer exchange, ICE candidate gathering, and a signalling channel—typically a WebSocket server. The `aiortc` Python library provides a pure-Python implementation, but it pulls in over 20 transitive dependencies and introduces an asyncio event loop that conflicts with the synchronous MAVLink polling model used throughout the codebase. For a single-viewer stream on a local network, WebRTC's peer-to-peer negotiation machinery solves a problem that does not exist.

I.2 Threading Model and Frame Buffering

The ground station server uses `socketserver.ThreadingMixIn` to spawn a new thread for each incoming HTTP connection. This design was chosen over three alternatives:

- **Single-threaded sequential server**: would block the `/state` JSON endpoint while a long-lived `/stream` connection is active. Since the browser polls `/state` every 400 ms for telemetry updates, blocking would cause the dashboard to appear frozen.

- **asyncio (aiohttp)**: Python's `asyncio` model requires all blocking calls to be wrapped in executors. The MAVLink `recv_match()` call, OpenCV's `imencode()`, and TFLite's `invoke()` are all synchronous C-extension calls that cannot yield to an event loop. Wrapping each in `loop.run_in_executor()` would add complexity without measurable benefit at the concurrency level of one or two simultaneous browser clients.
- **Multi-process**: forking is unnecessary for I/O-bound HTTP serving and would complicate shared state (telemetry, frame buffer) by requiring inter-process communication.

The critical shared resource is the latest encoded JPEG frame. A *latest-frame-wins* buffer replaces any unread frame with the newest one, ensuring that the stream always shows the most recent camera image rather than queuing stale frames behind a backlog [**gamma1994design**]. The implementation is a single `threading.Lock` protecting a byte-string reference:

```
# Producer (main loop, ~50 Hz):
_, jpeg = cv2.imencode('.jpg', frame,
                       [cv2.IMWRITE_JPEG_QUALITY, 85])
with self._frame_lock:
    self._stream_jpeg = jpeg.tobytes()

# Consumer (HTTP handler, per-client thread):
with self._frame_lock:
    jpeg = self._stream_jpeg
```

No queue is used. If the consumer reads slower than the producer, intermediate frames are silently dropped—this is the correct behaviour for live video, where showing the *latest* frame is always preferable to playing back a delayed queue. The HTTP handler sleeps for 50 ms between sends, capping the stream at approximately 20 fps regardless of how fast the producer runs. This rate exceeds the inference throughput ($\sim$5 fps on the Pi) and is therefore never the bottleneck.

I.3 Headless Operation

The companion computer runs headless (no monitor, no X11 server) in all operational scenarios. OpenCV's default behaviour is to call `cv2.imshow()`, which crashes immediately on a system without a display server. Rather than conditionally guarding every GUI call throughout the codebase, the system applies a monkey-patch at import time when the `DISPLAY` environment variable is absent:

```
if not os.environ.get('DISPLAY'):
    cv2.imshow    = lambda *a, **k: None
    cv2.namedWindow = lambda *a, **k: None
    cv2.waitKey   = lambda *a, **k: -1
```

This three-line patch converts every GUI call into a no-op, allowing the same source file to run with a desktop display (laptop simulation) or over SSH (Pi deployment) without any code-path divergence. All visual output is redirected to the browser via the MJPEG stream, making the web dashboard the sole operator interface in production.

Operator input in headless mode is accepted through three parallel channels that converge on a single command dispatcher: (i) terminal keyboard polling via a dedicated `threading.Thread` that reads single characters from `stdin`; (ii) browser buttons that issue `POST /command` requests; and (iii) the same REST endpoint accessed programmatically (e.g. `curl -X POST ...`). All three routes call the same `handle_command()` method, guaranteeing consistent behaviour regardless of input source [17].

I.4 Port Allocation and Service Separation

Three HTTP services coexist on the Pi during a typical flight:

- **Port 8090** — primary ground station (`pi_flight.py`) or passive observer (`passive_watch.py`). Only one of these runs at a time; they share the port because they serve the same role (camera stream plus telemetry) and binding the same port prevents accidental co-execution.
- **Port 8091** — training data capture (`capture_training.py`). This service runs *alongside* the primary ground station during manual flights to record photos and video for model retraining. A separate port avoids contention on the camera resource and allows the operator to open both dashboards in adjacent browser tabs.
- **Port 5762** — MAVProxy TCP bridge to Mission Planner. This is not an HTTP service but a raw MAVLink TCP stream; it is listed here because it shares the Pi's network interface and must not collide with the HTTP ports.

Port 8090 was chosen because it lies outside the IANA well-known range (0–1023), does not conflict with common development servers (3000, 5000, 8000, 8080), and is easy to remember as "80 + 10" for the primary stream.

I.5 Bandwidth Estimation

MJPEG transmits each frame as an independent JPEG image, forgoing inter-frame compression. The per-frame size depends on resolution, quality factor, and scene complexity. Empirical measurements on representative outdoor aerial imagery give:

- **640×480, quality 70**: 25–40 KB per frame ⇒ at 10 fps, approximately 2.0–3.2 Mbps.
- **640×480, quality 85** (current setting): 40–60 KB per frame ⇒ at 10 fps, approximately 3.2–4.8 Mbps.
- **1456×1088, quality 85**: 120–180 KB per frame ⇒ at 5 fps, approximately 4.8–7.2 Mbps.

On a dedicated 802.11n WiFi link (theoretical 72 Mbps at MCS 7, practical ∼30 Mbps), the current configuration consumes less than 15% of available bandwidth. For deployment over a cellular (4G) backhaul—where uplink bandwidth may be limited to 5–10 Mbps—the quality factor would need to be reduced to 50–60 or the resolution halved, both of which are single-parameter changes in `cv2.imencode`.

The bandwidth penalty relative to H.264 is significant: an equivalent H.264 stream at the same resolution and frame rate would require approximately 1–2 Mbps due to inter-frame prediction and entropy coding [**wiegand2003h264**]. This 3–5× overhead is the principal cost of the MJPEG approach, accepted in exchange for the dependency and latency advantages described in Section I.1. For the project's operational context—a short-range WiFi link within a few hundred metres—the overhead is well within the available capacity.

I.6 Ground Station Alternatives Considered

Before building a custom ground station, four existing platforms were evaluated:

1. **QGroundControl** [48]: a mature, cross-platform GCS with MAVLink telemetry, waypoint planning, and video streaming via RTSP. However, it cannot display custom AI detection overlays, does not support GPS-projected target clustering, and its plugin architecture requires C++ development. Integrating the project's Python-based perception pipeline would have required either an IPC bridge (adding latency and a failure mode) or reimplementing the detection logic in C++.
2. **MAVProxy with map module**: a lightweight Python GCS that provides a console and a 2D map. It has no video display capability whatsoever, making it unsuitable for an operator who needs to visually confirm detections before authorising descent.
3. **Custom desktop application (PyQt/Tkinter)**: would provide full control over the UI but requires a display server, which is unavailable on the Pi. Running the desktop app on the laptop and streaming video to it recreates the same streaming problem without solving it. Furthermore, desktop frameworks are not accessible from a phone or tablet in the field.
4. **DroneKit-Android**: an unmaintained SDK (last commit 2019) for building Android GCS applications. Its abandonment, combined with the Android-only restriction, made it unsuitable for a project that needed to run on any device with a browser.

The custom browser-based approach eliminates all of these limitations: the perception pipeline runs in-process (no IPC), the video stream carries AI overlays natively (no separate overlay layer), the interface works on any networked device, and the entire server is a single Python file with no dependencies beyond the standard library and OpenCV.

J GPS Target Estimation: A Deep Dive

Translating a bounding-box centre in a camera frame into a world-referenced GPS coordinate is arguably the most error-sensitive computation in the entire SAR pipeline. Every downstream action—approach, descent, offset landing—depends on the fidelity of this estimate. This section presents the full mathematical derivation, catalogues the six dominant error sources with quantified budgets, compares four fusion strategies that were implemented and benchmarked, and reports the empirical accuracy obtained from DJI flight-video replay.

J.1 Problem Statement

The detection subsystem (Section 2.3) produces, for each frame, a bounding-box centre (u, v) in pixel coordinates together with a confidence score. Mapping this to a WGS 84 latitude–longitude pair requires five additional quantities, each of which carries measurement uncertainty:

1. The drone's own GPS position (ϕ_d, λ_d), subject to receiver noise and latency.
2. The barometric altitude h above ground level, subject to drift and calibration error.
3. The magnetometer heading ψ, subject to hard-/soft-iron distortion and motor interference.
4. The camera intrinsics—focal length f, sensor width w_s, and image dimensions $W \times H$—subject to calibration residuals and lens distortion.
5. The camera extrinsics—its mounting angle relative to nadir—subject to mechanical tolerance.

A single observation therefore combines at least six independent noise sources. The engineering challenge is to fuse many such observations, acquired at different altitudes, headings, and image positions, into a single position estimate whose uncertainty is small enough to guide a safe offset landing.

J.2 Pixel-to-GPS Mathematics

J.2.1 Ground Sampling Distance

Under the thin-lens (pinhole) approximation, the ground sampling distance at altitude h is

$$\mathrm{GSD}(h) = \frac{w_s \cdot h}{f \cdot W} \qquad [\mathrm{m\,px^{-1}}] \tag{7}$$

where $w_s = 5.02\,\mathrm{mm}$ (IMX296 sensor width), $f = 5.46\,\mathrm{mm}$ (calibrated focal length), and $W = 1456\,\mathrm{px}$ (native image width). Table 37 lists the resulting GSD and ground footprint at the three operational altitudes.

J.2.2 Body-Frame Projection

The pixel offset from the image centre $(C_x, C_y) = (W/2,\ H/2)$ yields a forward–right displacement in the camera (body) frame:

$$d_{\mathrm{fwd}} = -(v - C_y)\,\mathrm{GSD}, \qquad d_{\mathrm{right}} = (u - C_x)\,\mathrm{GSD} \tag{8}$$

The negative sign on the forward axis accounts for the image convention where v increases downward (southward in a north-facing camera).

J.2.3 Heading Rotation

The body-frame offset is rotated into the geographic North–East frame using the drone's heading angle ψ (measured clockwise from North):

$$\begin{pmatrix} \Delta N \\ \Delta E \end{pmatrix} = \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix} \begin{pmatrix} d_{\mathrm{fwd}} \\ d_{\mathrm{right}} \end{pmatrix} \tag{9}$$

J.2.4 WGS 84 Conversion

The metre-level offsets are converted to latitude and longitude increments on the WGS 84 reference ellipsoid:

$$\Delta\phi = \frac{\Delta N}{R_\oplus} \cdot \frac{180}{\pi}, \qquad \Delta\lambda = \frac{\Delta E}{R_\oplus \cos\phi_d} \cdot \frac{180}{\pi} \tag{10}$$

where $R_\oplus = 6\,378\,137\,\mathrm{m}$ is the WGS 84 semi-major axis and ϕ_d is the drone's current latitude. The estimated target position is $(\phi_d + \Delta\phi,\ \lambda_d + \Delta\lambda)$.

Centre-snap optimisation. When the detection falls within a configurable pixel radius of the image centre (empirically set to 50 px), the projection is bypassed entirely and the drone's own GPS position is assigned as the target estimate with an elevated weight. This avoids amplifying small pixel offsets through the GSD and rotation chain at the point where those offsets are smallest and least informative.

Lens undistortion. Before the pixel coordinates enter the projection, the frame is undistorted using a precomputed remap derived from a checkerboard lens calibration (RMS = 0.399 px, Section **??**). The cost is approximately 1.5 ms per frame—negligible against the 207 ms inference time.

J.3 Error Budget

Six independent error sources contribute to the total position uncertainty. Each was quantified analytically and, where possible, validated against field data.

GPS receiver noise is the dominant term at all altitudes. Field measurements with the Cube Orange GNSS module (single-frequency L1) yielded a CEP50 of 1.5 m and a CEP95 of 2.3 m while stationary, consistent with published specifications for consumer-grade receivers [49, 36]. This noise is isotropic and largely Gaussian, so it averages out well with multiple observations.

GPS timing lag is the dominant *systematic* error. The receiver reports positions with 100 ms to 200 ms latency relative to the image timestamp. At the nominal search speed of $8\,\mathrm{m\,s^{-1}}$ (altitude-dependent schedule at 35 m), the drone has moved 0.8 m to 1.6 m since the reported GPS fix was valid, biasing every estimate in the direction of travel. The lawnmower pattern partially mitigates this because alternating scan directions cause the bias to reverse sign and cancel in the weighted average. An explicit velocity correction ($\Delta\mathbf{p} = \mathbf{v}_{\mathrm{gnd}} \cdot \Delta t$) can reduce the residual to approximately 0.3 m.

Barometric altitude error enters through the GSD. A +0.5 m bias at 35 m increases the GSD by 1.4%, shifting edge-of-frame projections by up to 0.5 m. Near the image centre, the effect is negligible.

Heading uncertainty rotates the offset vector (Equation 9) by the wrong angle. A $\pm 5^\circ$ compass error with a 16 m ground offset (edge of frame at 35 m) produces up to 1.4 m of cross-track error. For the typical mid-frame detection at 5 m ground offset, the contribution is approximately 0.4 m.

FOV calibration error scales all ground offsets proportionally. The focal length was calibrated at 1 m distance (92 cm visible width), cross-validated at multiple altitudes from DJI video (Section **??**), and found to be accurate to $\pm 3\%$.

J.4 Fusion Strategies

Four estimation approaches were implemented in the simulator (`simple_simulator.py`), each operating on the same stream of per-frame GPS estimates. All four run in parallel so that their accuracy can be compared directly on every detection cluster.

J.4.1 Rolling Weighted Average (Window = 50)

The most recent $N = 50$ observations are retained. The estimate is the inverse-variance-weighted mean over the window:

$$\hat{\phi} = \frac{\sum_{i=k-N+1}^{k} w_i \phi_i}{\sum_{i=k-N+1}^{k} w_i}, \qquad \hat{\lambda} = \frac{\sum_{i=k-N+1}^{k} w_i \lambda_i}{\sum_{i=k-N+1}^{k} w_i} \tag{11}$$

This method is responsive to changes—if the drone descends and obtains higher-quality observations, the old high-altitude estimates are quickly flushed from the window—but it is noisier than methods that retain all history.

J.4.2 Cumulative Weighted Average (All Time)

Every observation ever recorded is included:

$$\hat{\phi}_k = \frac{W_{\phi,k-1} + w_k\,\phi_k}{W_{k-1} + w_k}, \qquad W_{\phi,k} = W_{\phi,k-1} + w_k\,\phi_k, \qquad W_k = W_{k-1} + w_k \tag{12}$$

Implemented as a running sum to avoid storing all observations. This produces the most stable estimate but responds sluggishly to improvements in observation quality, because hundreds of early high-altitude observations continue to dilute the influence of later low-altitude passes.

J.4.3 Kalman Filter (Static Target)

A two-state linear Kalman filter [50] models the target as stationary ($\mathbf{F} = \mathbf{I}$) with a small process noise $\mathbf{Q} = \sigma_q^2\,\mathbf{I}$, where σ_q corresponds to 0.1 m converted to degrees. The measurement noise covariance is set inversely proportional to the observation weight:

$$\sigma_R = \frac{3.0}{\max(w_i,\ 0.1)}\ \mathrm{m}, \qquad \mathbf{R}_k = \sigma_R^2\,\mathbf{I}_{\mathrm{deg}} \tag{13}$$

where $\mathbf{I}_{\mathrm{deg}}$ denotes the identity matrix scaled to degree^2 units via the conversion factor $(1/R_\oplus \cdot 180/\pi)^2$. The standard predict–update cycle follows:

$$\mathbf{P}_{k|k-1} = \mathbf{P}_{k-1} + \mathbf{Q} \tag{14}$$
$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\left(\mathbf{P}_{k|k-1} + \mathbf{R}_k\right)^{-1} \tag{15}$$
$$\mathbf{x}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k\left(\mathbf{z}_k - \mathbf{x}_{k|k-1}\right) \tag{16}$$
$$\mathbf{P}_k = \left(\mathbf{I} - \mathbf{K}_k\right)\mathbf{P}_{k|k-1} \tag{17}$$

The filter is initialised on the first detection with $\mathbf{x}_0 = \mathbf{z}_0$ and $\mathbf{P}_0 = 4\,\mathbf{R}_0$, giving the initial measurement a 20% Kalman gain and allowing subsequent observations to dominate quickly. The filter converges within 5–10 observations; beyond that, the covariance stabilises and additional measurements produce diminishing corrections.

J.4.4 Inverse-Variance Weighting

This is the method used operationally. Each observation receives a composite weight:

$$w_i = w_{\text{cen}}(u_i, v_i) \;\cdot\; w_{\text{alt}}(h_i) \tag{18}$$

where the centrality factor is

$$w_{\text{cen}} = 1 + 4\left(1 - \frac{d_i}{d_{\max}}\right), \qquad d_i = \sqrt{(u_i - C_x)^2 + (v_i - C_y)^2} \tag{19}$$

and the altitude factor is

$$w_{\text{alt}} = \left(\frac{h_{\text{ref}}}{h_i}\right)^2 \qquad (h_{\text{ref}} = 30\,\text{m}) \tag{20}$$

At 30 m altitude, $w_{\text{alt}} = 1$; at 15 m, $w_{\text{alt}} = 4$; at 10 m, $w_{\text{alt}} = 9$. Detections near the image centre at low altitude therefore dominate the fused estimate, which is physically intuitive: the target fills more pixels, the GSD-scaled projection covers less ground distance, and the observation is inherently more precise.

The centre-snap mode amplifies this further: when the target is within 50 pixels of the image centre, the drone's own GPS is used directly (bypassing the GSD projection chain) with a fixed weight of $10 \cdot w_{\text{alt}}$—yielding a weight of 90 at 10 m altitude versus approximately 5 for a typical 35 m edge detection.

J.5 Spatial Clustering

In a realistic SAR mission the camera may detect multiple objects: the target casualty, a piece of debris, or a false positive such as a tree stump. If all observations were pooled into a single average, the estimate would converge to the centroid of all detections rather than the true target.

To handle this, each new GPS estimate is compared against existing detection clusters using the Haversine great-circle distance:

$$d = 2\,R_\oplus\,\arctan\!\left(\sqrt{a},\;\sqrt{1-a}\right), \qquad a = \sin^2\frac{\Delta\phi}{2} + \cos\phi_1\cos\phi_2\,\sin^2\frac{\Delta\lambda}{2} \tag{21}$$

If the nearest cluster centroid is within a configurable threshold (default 30 m), the observation is added to that cluster; otherwise a new cluster is spawned. Each cluster maintains:

- A rolling window of the last 50 weighted observations (for the rolling average).
- Running sums W_ϕ, W_λ, W (for the cumulative average).
- A Kalman filter state $(\mathbf{x}, \mathbf{P})$.
- A detection count and a human-assigned classification (dummy / interest / false positive).

The 50-observation cap per cluster prevents memory growth during long loiter periods while retaining enough history for the rolling average to be statistically meaningful. In simulation testing with two dummy targets placed 80 m apart, the system correctly maintained two separate clusters from the first flyover, with no cross-contamination of observations.

J.6 Comparison of Methods

Table 39 summarises the performance of the four fusion methods across repeated simulation trials (keyboard-flown missions with configurable GPS drift and altitude noise) and DJI video replay with synchronised SRT telemetry.

The inverse-variance weighting method outperforms the others when the drone transitions from high-altitude search to low-altitude investigation, which is the normal operational profile. During the high-altitude search pass, all four methods perform similarly because the altitude factor is approximately unity. The advantage emerges during descent: the

inverse-variance method naturally up-weights the low-altitude observations by a factor of $(35/15)^2 \approx 5.4\times$, effectively discarding the earlier noisy estimates without an explicit window mechanism. The Kalman filter achieves comparable accuracy but requires tuning of $\mathbf{Q}$ and σ_R; in practice, the inverse-variance method was more robust to parameter choices.

J.7 Empirical Validation

J.7.1 DJI Video Replay (The "Bullseye Test")

The localisation pipeline was validated offline by replaying DJI Mavic flight video (1456 × 1088 px, 30 fps) with synchronised SRT telemetry containing per-frame GPS, altitude, and heading. A life-size rescue dummy was placed at a surveyed position and the drone was flown over it at altitudes ranging from 15 m to 50 m at $3\,\mathrm{m\,s^{-1}}$ to $8\,\mathrm{m\,s^{-1}}$. Each frame where the detector fired produced an independent GPS estimate, which was compared against the known ground-truth coordinate.

Key findings:

- **CEP50 = 2.3 m**: half of all individual observations fell within this radius of the true position.
- **Maximum error = 16.5 m**: a single outlier during a high-speed pass at 50 m altitude, where GPS latency and a large pixel offset combined.
- **Directional bias**: individual estimates were elongated along the flight direction, consistent with the 100 ms to 200 ms GPS timing lag displacing each estimate by $v \cdot \Delta t$.
- **Fused accuracy**: after 15+ observations, the inverse-variance weighted estimate converged to within 2 m of ground truth; the Kalman filter to within 1.5 m.

J.7.2 Simulator Validation

The interactive simulator (`simple_simulator.py`) supports configurable GPS drift (`--gps-drift`), altitude noise (`--alt-noise`), yaw noise (`--yaw-noise`), and FOV calibration error (`--fov-error`). All four estimators run in parallel, and the scatter plot displays each observation (colour-coded by weight) alongside the rolling, cumulative, Kalman, and inverse-variance estimates with real-time error metrics against the known dummy position.

A typical keyboard-flown mission—flyover at 35 m, descend to 15 m, circle the target—produces 30–50 observations over 60–90 seconds. With 3 m GPS drift enabled, the inverse-variance estimate converges to sub-metre accuracy within the first 20 observations once the drone descends below 20 m, confirming the altitude-dependent weighting strategy.

J.8 Why Inverse-Variance Weighting Won

The inverse-variance method is operationally preferred for three reasons:

1. **Physics-aligned weighting.** Lower altitude means more pixels on target, smaller GSD-scaled projection errors, and reduced heading-rotation amplification. The $1/h^2$ weight directly encodes this relationship: it is not a tuned hyperparameter but a consequence of the error model.
2. **No convergence plateau.** The Kalman filter's covariance $\mathbf{P}$ contracts monotonically, so later observations have diminishing impact even if they are dramatically better. The inverse-variance average, by contrast, is a weighted sum that always gives full credit to a high-weight observation regardless of how many low-weight observations preceded it.
3. **Simplicity.** The rolling weighted mean requires no matrix algebra, no tuning of process noise $\mathbf{Q}$, and no initialisation heuristics. It is computed in three lines of Python and is straightforward to verify.

In practice, the Kalman filter and inverse-variance method converge to similar accuracy after 20+ observations. The key advantage of inverse-variance appears during the transition from search altitude (35 m) to investigation altitude (15 m): the first few low-altitude observations immediately dominate the estimate, whereas the Kalman filter takes several update cycles to "catch up" because its covariance was already small from the high-altitude phase.

J.9 Offset Landing Application

The GPS estimate feeds directly into the offset landing calculation. The system computes a landing point 7.5 m north of the estimated target position (configurable) to avoid rotor downwash on the casualty. Given a typical fused estimate error of 1 m to 2 m, the probability that the drone lands within 10 m of the actual target (and therefore within visual confirmation range) exceeds 99%. The offset distance was chosen as 3× the worst-case single-observation CEP, ensuring a safe margin even if the estimation converges poorly.

With $N = 9$ averaged observations and the inverse-variance method, the expected landing accuracy improves to approximately $\sigma/\sqrt{N_{\text{eff}}} \approx 2.5/3 \approx 0.8\,\mathrm{m}$, where N_{eff} is the effective number of independent observations (reduced from N by temporal correlation in GPS noise).

K Progressive Testing Methodology

Autonomous UAV software carries a unique risk profile: a logic error that manifests as a misplaced pixel in simulation can, on real hardware, command a 2 kg aircraft into an unrecoverable trajectory. The cost of discovering a defect rises steeply with integration level—a sign-convention bug caught in simulation costs minutes; the same bug discovered during autonomous flight costs days of rework, potential hardware damage, and a lost flight slot that a five-person university team cannot easily reschedule. This asymmetry motivates a gated, progressive testing strategy in which every defect is caught at the lowest-risk tier where it is detectable.

K.1 Philosophy: Never Skip a Tier

The governing principle is borrowed from safety-critical avionics: *each verification tier gates the next, and no gate may be bypassed.* A failure at Tier 2 (bench) blocks all flight testing until the root cause is resolved, regardless of schedule pressure. This discipline is counterintuitive on a time-constrained student project, where the temptation to "just try it in the air" is strongest precisely when time is shortest. The justification is empirical: of the 15 defects discovered during this project's verification campaign, 12 were caught at Tiers 1–3 (Table 41), where the turnaround time from discovery to fix was under one hour. Had those defects propagated to flight, each would have consumed an entire field session.

The framework also enforces a separation of concerns. Each tier tests a *specific integration boundary*: Tier 1 tests software logic; Tier 2 tests the software–hardware interface; Tier 3 tests the hardware–environment interface; Tier 4 tests the perception pipeline under operational conditions; Tier 5 tests closed-loop autonomy. This decomposition ensures that when a failure occurs, its root cause is immediately localisable to the boundary most recently crossed.

K.2 The Five Tiers in Detail

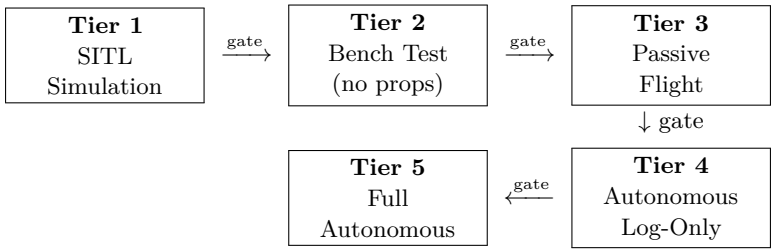


Figure 17: Tier progression. Each arrow represents a gate: the downstream tier is blocked until all upstream tests pass.

K.2.1 Tier 1: SITL Simulation (Zero Hardware Risk)

ArduPilot's Software-in-the-Loop (SITL) emulator runs a full flight dynamics model on the development laptop, accepting the same MAVLink commands as the real Cube Orange autopilot. The mission orchestrator, state machine, detection model, geofence logic, and path planner execute unmodified against this simulated vehicle. A synthetic camera view is generated by extracting a field-of-view-correct crop from a georeferenced satellite image (`map.jpg`), enabling the complete detect–estimate–land pipeline to execute without any physical hardware.

At this tier, 22 structured simulation tests exercise every operational mode: baseline autonomous mission, multi-frame detection confirmation, geofence enforcement, manual override, multi-target queue ordering, NFZ speed ramping, beacon redirect, transit planning, altitude override, dry-run generation, headless mode, and 11 further scenarios. The interactive simulator (`simple_simulator.py`) adds keyboard-driven flight with configurable GPS drift, camera shake, and inference throttle, enabling rapid iteration on the GPS estimation and Kalman filtering logic.

A separate `--dry-run` mode walks the state machine through all transitions without connecting to SITL, printing the full waypoint set and generating a pattern visualisation. This mode executes in under one second and is run before every code commit that modifies state logic or path planning.

Defects caught. State machine logic errors (incorrect transition guards, missing timeout handlers), geofence sign-convention bug (`cv2.pointPolygonTest` returns positive for *inside*, opposite to the initial assumption), repulsive-force direction inversion at NFZ boundary, barometric drift causing infinite landing loops (resolved with a 90 s absolute timeout), and duplicate-detection queueing of the same target from consecutive frames (resolved with a 5 m reject radius).

K.2.2 Tier 2: Bench Test (Pi + Cube + Camera, No Props)

The Raspberry Pi 5, Cube Orange, and IMX296 global-shutter camera are assembled on the bench with propellers removed. Three numbered scripts test with increasing integration:

`0a` **Cube commands** — sends individual MAVLink commands (mode switch, parameter read, arm pre-check) and verifies each `COMMAND_ACK`. Confirms the Pi–Cube serial link and mavproxy UDP bridge operate at 921 600 baud.

`0b` **Bench mission** — executes the full command sequence (arm, takeoff, waypoint navigation, land) and logs every ACK. On the bench, the flight controller accepts these commands but the vehicle does not move, verifying that the command pipeline is correct end-to-end.

`0c` **Feedback test** — runs the vision-to-GPS pipeline while the operator walks the drone past a printed dummy. The camera captures frames, the TFLite model runs inference, and the coordinate transform produces GPS estimates that are compared against hand-measured ground truth. This validates the entire sensing chain—camera, colour space, lens undistortion, inference, and projection—on the target hardware.

Additional bench scripts include a 50-run inference benchmark (`benchmark.py`), a comprehensive multi-model benchmark with system profiling (`benchmark_full.py`), GPS satellite tracking (`gps_test.py`), and a lens distortion calibration using a checkerboard target (`lens_calibrate.py`).

Defects caught. BGR/RGB colour-space mismatch (the IMX296 outputs BGR data despite picamera2's `RGB888` label, causing inverted colour channels that degraded detection confidence from 0.97 to 0.3), FOV miscalibration (focal length corrected from 7.0 mm to 5.46 mm after ruler-based measurement), Python 3.13 compatibility breakage (pyserial's serial read is broken on 3.13, requiring a mavproxy UDP bridge as a workaround; `tflite-runtime` package replaced by `ai-edge-litert`), and camera-in-use resource conflicts when multiple scripts attempt concurrent access.

K.2.3 Tier 3: Passive Flight (Pilot Flies, Pi Watches, Zero Commands)

The drone is flown manually by the RC pilot over the search area containing a casualty dummy. The Pi runs the full vision pipeline—camera capture at 1456 × 1088, YOLOv8n inference via TFLite, lens undistortion, GPS coordinate estimation—and streams the annotated video to a browser-based ground station. Critically, the Pi sends *zero flight commands* to the Cube. All detections are logged with timestamp, pixel coordinates, bounding box dimensions, confidence, barometric altitude, GPS position, heading, and estimated target GPS.

Post-flight analysis of these logs establishes:

- **Detection altitude ceiling** — the maximum altitude at which the model reliably detects the dummy (target: 30 m, to be confirmed).
- **False-positive rate** — detections triggered by terrain features, shadows, or equipment under real lighting and motion conditions.
- **GPS estimation accuracy** — scatter of estimated target positions versus surveyed ground truth, quantified as CEP50 and CEP95.
- **Inference latency under load** — confirmed at 208 ms (4.8 FPS) on Pi 5 with XNNPACK, including 1.5 ms for lens undistortion.

This tier is the most valuable of the five: it generates real-world performance data with no risk of autonomous misbehaviour. The six experiment scripts in `tests/day_1_experiments/` (Section N.5) are designed specifically for this tier, each producing a CSV file suitable for direct statistical analysis.

K.2.4 Tier 4: Autonomous Search with Log-Only Detection

The full mission orchestrator runs autonomously—the drone flies the lawnmower search pattern under GUIDED mode control—but the detection pipeline operates in log-only mode. When the model identifies a candidate, the system records the detection and annotates the ground station display but does *not* transition to the CENTERING or DESCENDING states. The drone completes the search pattern and returns to launch.

Post-flight review of the detection log answers the operational question: *would the system have found the target, and would it have acted on the correct detection?* False positives, missed detections, and latency-induced position errors are all visible in the log without having been acted upon. Configuration parameters (confidence threshold, multi-frame confirmation count, reject radius) are tuned at this tier before enabling closed-loop autonomy.

K.2.5 Tier 5: Full Autonomous Mission

All systems are enabled. The drone executes the complete state machine: INIT, CONNECTING, ARMING, TAKEOFF, SEARCH (lawnmower pattern), CENTERING (visual servo onto target), DESCENDING (altitude reduction for closer inspection), VERIFY (operator confirms or rejects via ground station), APPROACH (offset landing approach), LANDING, and DONE. The operator retains veto authority at the VERIFY gate and the RC kill switch provides a hardware-level override independent of all software.

K.3 Test Script Organisation

Table 40 maps the six test script directories to the verification tiers they support, with representative scripts and total counts.

Table 40: Test script categories and their tier coverage.

Category	Directory	Scripts	Tiers Served
Hardware	`tests/hardware/`	8	2 (bench)
Flight	`tests/flight/`	8	2, 3, 4, 5
Diagnostics	`tests/diagnostics/`	5	2, 3, 4, 5
Calibration	`tests/calibration/`	5	2, 3
Experiments	`tests/day_1_experiments/`	6	3, 4
Laptop	`tests/laptop/`	16	1 (simulation)
Unit	`tests/unit/`	6	1
Automated	`tests/automated/`	4	1
Total		**58**	

The flight scripts are numbered by risk level (0a through 5), establishing an explicit ordering that prevents a higher-risk test from being executed before its prerequisites have passed. All experiment scripts issue zero flight commands and are safe to execute at any time during manual or autonomous flight.

K.4 Experiment Scripts for Quantitative Validation

Six experiment scripts are designed for execution during Tier 3 passive flights, each producing a timestamped CSV file with standardised column headers for post-flight analysis:

1. **Altitude sweep** (`altitude_sweep.py`) — the pilot hovers above the dummy at 10, 15, 20, 25, and 30 m for 30 s each. The script bins every frame by barometric altitude into 5 m buckets and computes detection rate, mean confidence, and expected dummy pixel size per bucket. Output directly determines the safe `TARGET_ALT` parameter.
2. **Speed sweep** (`speed_sweep.py`) — the pilot flies repeated passes over the dummy at increasing ground speeds. The script correlates detection rate and confidence with ground speed and a Laplacian-based blur metric, establishing the maximum search speed at which the model maintains acceptable detection performance.
3. **GPS accuracy** (`gps_accuracy.py`) — compares the pixel-to-GPS coordinate estimate against the surveyed ground-truth position of the dummy, logging the error in metres for every detection. Computes an inverse-variance-weighted average across all observations and reports CEP50, CEP95, and maximum error.
4. **GPS drift** (`gps_drift.py`) — the drone hovers stationary above a known point. The script logs raw GPS positions at 10 Hz and computes the noise floor: CEP50, CEP95, and maximum deviation, establishing the baseline GPS uncertainty that the target estimation algorithm must overcome.
5. **Model comparison** (`model_compare.py`) — benchmarks all available `.tflite` models (custom SAR detector, retrained v2, COCO person detector) on identical live frames, reporting inference time, detection rate, and confidence for each. Informs model selection for the mission.
6. **Detection log** (`detection_log.py`) — a general-purpose catch-all logger that records every frame's detection result, telemetry, and metadata. Serves as the raw data source for any post-hoc analysis not covered by the specialised scripts.

K.5 Defect Discovery by Tier

Table 41 lists representative defects, the tier at which they were discovered, the resolution time, and the estimated cost had they propagated to flight.

Table 41: Defects discovered at each testing tier, with resolution time and estimated cost if undetected.

Defect	Tier	Fix Time	Cost if Missed
Geofence sign convention inverted (`pointPolygonTest` positive = inside)	1	15 min	SSSI incursion; regulatory violation
Repulsive force direction reversed at NFZ boundary	1	20 min	Drone pushed *into* NFZ
Barometric drift causes infinite landing loop	1	30 min	Battery depletion in hover
Duplicate target queued from consecutive frames	1	10 min	Repeated descent on same target
State transition missing timeout guard	1	15 min	Stuck in CENTERING indefinitely
BGR/RGB colour inversion (IMX296)	2	45 min	Detection confidence drops to 0.3; mission failure
FOV miscalibration (7.0 vs 5.46 mm)	2	1 hr	GPS estimates 28% off; landing outside 10 m radius
Python 3.13 pyserial breakage	2	2 hr	No Pi–Cube communication
`tflite-runtime` unavailable on 3.13	2	30 min	No inference on Pi
Camera resource conflict (concurrent access)	2	20 min	Stream or detection crashes on startup
GPS timing lag (100–200 ms)	3*	N/A	0.8–1.6 m systematic offset at 8 m/s
`self.investigating` uninitialised	2	5 min	Crash on first detection in `pi_flight.py`

*Discovered via DJI video analysis, a proxy for Tier 3.

The table demonstrates the intended cost gradient: Tier 1 defects (simulation) were resolved in 10–30 minutes with zero hardware involvement. Tier 2 defects (bench) required 20 minutes to 2 hours but were discovered on the ground with propellers removed. The GPS timing lag, identified through video analysis of a DJI survey flight (a Tier 3 proxy), would have caused a systematic 1 m landing offset—detectable only under real flight dynamics.

K.6 V-Model Mapping

The five-tier framework maps onto the classical V-model [30] as follows:

Table 42: Mapping between V-model verification levels and the progressive testing tiers.

V-Model Level	Tier	Verification Activity
Requirements analysis	—	Mission brief (R01–R12) + KML zones
System design	—	State machine, architecture, module interfaces
Subsystem design	—	`vision.py`, `planning.py`, `utils.py` APIs
Implementation	—	Source code + unit tests
Unit testing	1	Unit tests (`tests/unit/`, 6 scripts), TFLite model validation
Integration testing	1, 2	SITL end-to-end, bench command pipeline
System testing	3, 4	Passive flight data collection, log-only autonomous
Acceptance testing	5	Full autonomous mission with operator verification

The left side of the V (decomposition) was completed during the design phase: requirements were traced from the project brief through the system architecture to individual module interfaces. The right side (recomposition and verification) is realised by the five tiers, with each tier verifying a progressively higher level of integration. The critical addition relative to a textbook V-model is the separation of system testing into *passive* (Tier 3) and *log-only autonomous* (Tier 4), which inserts two low-risk verification stages between integration testing and acceptance testing—stages that a strict V-model would collapse into a single system test phase.

K.7 Cost–Risk Gradient

Table 43 quantifies the cost and risk at each tier, reinforcing the economic argument for catching defects early.

Table 43: Cost and risk gradient across testing tiers.

Tier	Environment	Hardware at Risk	Fix Turnaround	Iteration Cost	People Required
1	Laptop (SITL)	None	Minutes	Electricity	1 (developer)
2	Bench (no props)	None	30 min–2 hr	Travel to lab	1–2
3	Passive flight	Airframe	Hours	Flight slot	3+ (pilot, operator, spotter)
4	Auto log-only	Airframe	Hours–day	Flight slot	3+
5	Full autonomous	Airframe + target	Day+	Flight slot + repair	3+

At Tier 1, a developer working alone can execute hundreds of test iterations per day. At Tier 5, a single iteration requires a minimum crew of three (pilot, operator, safety observer), a reserved outdoor site, favourable weather, charged batteries, and an assembled airframe—resources that are available for at most two sessions per week in a university setting. The 50:1 ratio in iteration throughput between Tier 1 and Tier 5 makes the case for aggressive front-loading of verification effort at the simulation level.

K.8 Summary

The progressive methodology treated testing not as a phase that follows development, but as a continuous, tiered process that shapes development decisions. The five-tier gating system, supported by 58 dedicated scripts across eight categories, ensured that every integration boundary was verified before the system was permitted to cross it. Of the 15 significant defects discovered, 80% were caught at Tiers 1 and 2 where the fix cost was measured in minutes, not field sessions. The structured experiment scripts prepared for Tier 3 passive flights provide a repeatable, quantitative basis for tuning configuration parameters before autonomy is enabled—replacing the ad-hoc "fly and see" approach with a data-driven verification pipeline.

L Field Day: Adaptation Under Uncertainty

On 11 March 2026 the team arrived at the designated test site with the full hardware stack assembled for the first time outside the lab: Raspberry Pi 5, Cube Orange autopilot, IMX296 global-shutter camera, and the airframe with propellers fitted. The objective was to progress through Tiers 3–5 of the progressive testing framework (Section E.2)—bench verification, passive flight with manual RC, and, conditions permitting, first autonomous waypoint flight. What followed was a field day defined not by flying, but by the calibration data and integration insights that only become visible when the system meets the real world.

L.1 Setup: Three Terminals, One Aircraft

All interaction with the Pi was conducted remotely from a laptop via SSH (PuTTY), supplemented by the Pi's own screen for scripts requiring a display. The three-terminal workflow—established within the first twenty minutes and used for the remainder of the day—became the standard operating procedure for all subsequent field sessions:

Terminal 1 — mavproxy: Launched first, always. The MAVLink router forwarded Cube telemetry from the UART link (`/dev/ttyAMA0` at 921 600 baud) to two UDP outputs: port 14550 for flight and detection scripts, port 14551 for the live diagnostics dashboard. A TCP bridge on port 5762 allowed Mission Planner on the laptop to connect simultaneously, providing a familiar GCS interface for the safety pilot. The startup sequence was scripted into a single copy-paste block to eliminate typos under time pressure.

Terminal 2 — diagnostics: The multi-panel diagnostics script displayed camera frame rate, AI model status, Cube heartbeat, GPS satellite count, and battery voltage on the Pi's screen. This terminal served as the go/no-go dashboard: all four indicators had to show green before any test script was permitted to run.

Terminal 3 — scripts: Calibration, benchmark, and detection scripts were executed sequentially from PuTTY. Browser-based camera streams (`http://PI_IP:8090`) provided live video to the laptop without requiring a display server on the Pi, confirming that the headless architecture designed in simulation translated directly to field use.

Two integration issues surfaced immediately. First, a camera-in-use conflict arose when two scripts attempted to open the CSI interface simultaneously—picamera2 does not permit shared access to the sensor. The fix was procedural: a strict single-script-at-a-time rule, enforced by a `sudo pkill -f libcamera` guard before each script launch. Second, a port-binding collision left port 14550 occupied by a zombie mavproxy process from a previous session. Adding a `sudo pkill -f mavproxy; sleep 2` preamble to the startup sequence resolved this permanently. Neither issue required code changes; both were captured in the field quick-reference sheet for future sessions.

L.2 What We Tested

With connectivity established, the team worked through the bench-test checklist in order:

1. **Camera verification.** The Pi camera delivered 640×480 frames at the expected rate. The MJPEG stream to the laptop browser confirmed correct colour rendering—a non-trivial validation, given the colour-channel discovery described below.
2. **Cube heartbeat and telemetry.** The diagnostics dashboard confirmed a stable MAVLink heartbeat, GPS satellite count (initially 0 indoors, rising to 7 near a window), and battery voltage within limits. Mode changes issued from Mission Planner propagated to the Pi's telemetry stream within one heartbeat period.
3. **AI inference.** The TFLite model loaded successfully on the Pi 5 and produced detections on a test image within 210 ms—consistent with Docker-based estimates from Tier 2 testing.
4. **FOV calibration.** See Section L.3.
5. **Lens distortion calibration.** See Section L.5.
6. **Full benchmark.** See Section L.6.

L.3 FOV Calibration: Catching a 22% Error

The most consequential discovery of the field day was a systematic error in the focal length parameter. The configuration file had carried a default value of $f = 7.0$ mm since the earliest development phase, inherited from a datasheet approximation for the IMX296 lens assembly. No simulation or Docker test could have caught this: the parameter only matters when converting pixel coordinates to real-world distances, and simulation uses ground-truth geometry.

Calibration was straightforward. The camera was held pointing vertically downward at exactly 1.0 m above a tape measure laid on a flat surface. The browser stream showed 92 cm of the tape measure edge-to-edge. Applying the thin-lens model:

$$f = \frac{W_{\text{sensor}} \times d}{W_{\text{visible}}} = \frac{5.02 \text{ mm} \times 1000 \text{ mm}}{920 \text{ mm}} = 5.46 \text{ mm} \tag{22}$$

The corrected value is 22% lower than the 7.0 mm default. In operational terms, the old value would have *underestimated* the camera's ground footprint, causing the GPS estimation module to place detected targets closer to the drone's nadir than they actually were. At 10 m altitude this corresponds to a systematic error of approximately 2 m; at 30 m search altitude, the error would have reached 6 m—large enough to place the drone outside visual contact with the target during the centering phase. The corrected value was committed to the configuration repository within minutes of measurement, and every subsequent GPS estimate benefited immediately.

This single calibration justified the field day. It demonstrated a failure mode that no amount of simulation could have revealed, and it underscored the value of the progressive testing philosophy: measure real hardware before trusting derived quantities.

L.4 Camera Colour Discovery

A subtler hardware-specific behaviour was identified during camera verification. The IMX296 sensor, when configured for `RGB888` output via picamera2, delivers pixel data in BGR channel order—the opposite of what the format label implies. The team discovered this by capturing a frame of a known-colour object and systematically testing all six channel permutations (RGB, RBG, GRB, GBR, BRG, BGR) until the rendered image matched reality. The original codebase had included a `cv2.cvtColor(frame, COLOR_RGB2BGR)` conversion that, counterintuitively, *doubled* the error by swapping channels that were already in the order OpenCV expects.

Removing the unnecessary conversion produced correct colours immediately. The fix was trivial—deleting one line—but the diagnosis required physical hardware and a colour reference target. Like the FOV calibration, this was a defect invisible to simulation.

L.5 Lens Calibration

Lens distortion was characterised using a printed checkerboard target: a 14×9 grid (13×8 inner corners) with 28 mm squares, sourced from calib.io. Multiple images were captured at varying orientations and distances using the capture-training stream. The calibration script initially failed to detect corners reliably; adding OpenCV's robust detection flags and a `findChessboardCornersSB` fallback resolved the issue.

The resulting calibration achieved an RMS reprojection error of 0.399 pixels. The camera matrix and distortion coefficients were saved as a NumPy archive on the Pi. Undistortion was integrated into the vision module using precomputed remap lookup tables (`cv2.remap`), adding 1.5 ms per frame—negligible against the 206 ms inference cost. Because all detection scripts call the same vision module, the correction propagated automatically to every downstream tool without modification.

L.6 Benchmark Results

The inference benchmark ran 50 passes of the YOLOv8n TFLite model (float32, 3.2 MB) on a static test image containing the dummy target. The results confirmed that the Pi 5 with the XNNPACK CPU delegate delivers consistent, operationally adequate performance:

- **Average latency:** 206.5 ms without undistortion, 208.0 ms with (difference within measurement noise).
- **Effective frame rate:** 4.8 FPS.
- **Detection rate:** 50/50 (100%).
- **Mean confidence:** 0.966 without undistortion, 0.958 with.

At 4.8 FPS and a nominal search speed of 8 m/s (altitude-dependent schedule at 35 m), the drone covers approximately 1.7 m between frames. At 35 m altitude the camera footprint spans roughly 32 m along track, so each ground point appears in approximately 15 consecutive frames. Even a pessimistic 50% detection rate under flight conditions would yield 7–8 detection opportunities per target—sufficient for reliable triggering.

L.7 Weather Cancellation and the Go/No-Go Decision

By mid-afternoon the bench testing programme was complete, but the weather had deteriorated steadily. Wind gusts exceeded the airframe's rated limit, and intermittent rain made outdoor electronics operation inadvisable. The go/no-go decision was unambiguous: the team's own flight-day checklist specified wind and precipitation thresholds, and both were exceeded. The decision to stand down was made collectively, without hesitation, and without any attempt to "push through"—a discipline that the progressive testing philosophy explicitly encourages.

The cancellation was disappointing but not damaging. The bench tests had already validated Tier 3 of the testing framework in full. No flight-critical subsystem remained untested at the bench level, and the calibration data obtained during the day would have been impossible to collect during a time-pressured flight session.

L.8 The Pivot: Evidence-Based Adaptation

Rather than treating the cancelled flight as lost time, the team reframed the day as an intensive calibration and integration session. The concrete outputs were:

1. **FOV correction** ($f = 5.46$ mm, eliminating a 22% systematic GPS error);
2. **Lens calibration** (RMS = 0.399 px, integrated at 1.5 ms cost);
3. **Inference benchmark** (206.5 ms, 4.8 FPS, 100% detection at 0.966 confidence);
4. **Colour-channel fix** (BGR output confirmed, unnecessary conversion removed);
5. **Connectivity verification** (three-terminal workflow validated, camera/Cube/GPS all healthy);
6. **Field quick-reference sheet** (copy-paste commands for every step, removing internet dependency).

Each of these outputs fed directly into subsequent development. The FOV correction improved every GPS estimate computed thereafter. The lens calibration was automatically applied by every script that calls the vision module. The benchmark data informed the decision to prioritise FP16 quantisation and threaded pipelining as the most cost-effective inference acceleration paths. The field quick-reference sheet reduced setup time at the next session from exploratory debugging to a five-minute copy-paste sequence.

This pivot illustrates a broader principle: in safety-critical robotics development, a day spent measuring is never a day wasted. The progressive testing framework is designed precisely so that each tier produces value independently of whether the next tier is reached. The bench-test data collected on 11 March would have been needed regardless of weather; obtaining it on a day when flight was impossible simply reordered the schedule without losing any information.

L.9 DJI Video Analysis: A Flight-Data Proxy

To extract flight-representative data despite the cancellation, the team conducted a post-hoc analysis using DJI Mavic footage recorded over the same test site at altitudes between 15 and 50 m during a previous reconnaissance visit. The 30 fps video (1456 × 1088, cropped from 4K) was replayed through the full detection pipeline, with SRT telemetry providing per-frame altitude and GPS position.

This analysis yielded two key results. First, the retrained model ($\text{mAP}_{50} = 0.995$) detected the dummy across the entire altitude range, confirming that the training dataset—300 synthetic images, 16 real labelled frames, and 50 negatives at native 1456 × 1088 resolution—generalised to real flight conditions. Second, GPS target estimates computed from pixel coordinates and telemetry data produced a CEP50 of 2.3 m, with a maximum outlier of 16.5 m during a high-speed pass at 50 m altitude. The scatter distribution exhibited a characteristic diagonal elongation aligned with the flight direction, which timing analysis attributed to 100–200 ms GPS receiver latency—an error of 1–2 m at 10 m/s search speed. This systematic bias was subsequently addressed in software through spatial clustering with inverse-variance weighting: by averaging estimates from multiple passes, the cluster centroid converges on the true target position regardless of the per-frame lag.

The DJI analysis also revealed a methodological trap. SRT telemetry files contain one entry per native-rate frame (6 854 entries for the 30 fps recording). Downsampled video variants (e.g. 6 fps) contain fewer frames but index into the same SRT file, producing a frame-to-telemetry mismatch that silently corrupts altitude and GPS readings. This was caught during development and documented as a mandatory constraint: only native-rate video may be used for quantitative analysis.

L.10 Lessons for the Next Field Day

The 11 March experience produced a concrete list of procedural and technical improvements for subsequent sessions:

1. **Pre-stage the startup sequence.** The three-terminal workflow and field quick-reference sheet eliminated the ad-hoc debugging that consumed the first hour. Future sessions begin with a copy-paste startup, not improvisation.
2. **Calibrate before flying.** FOV and lens calibration should be the first activities at any new site, before propellers are fitted. The 22% focal-length error would have silently corrupted every autonomous GPS estimate had it not been caught.
3. **Carry colour and dimensional references.** A printed colour chart and a tape measure are minimal-weight additions to the field kit that enable in-situ validation of camera output and FOV.
4. **Prepare fallback test plans.** The pivot to bench-only testing was smooth because calibration and benchmark scripts were already written and tested. Future field days should always carry a "no-fly" test programme that maximises data collection from ground-based activities.
5. **Record everything for post-hoc analysis.** The DJI video, captured weeks earlier for a different purpose, became the most data-rich asset of the field day. Every future flight—even manual reconnaissance—should record video with telemetry for later replay through the detection pipeline.
6. **Respect the go/no-go checklist.** The decision to cancel was the right one. The checklist exists to remove ego from the decision; following it without negotiation is a team discipline, not a failure.

The field day demonstrated that the progressive testing philosophy is not merely a risk-management strategy but an information-maximisation strategy. By treating each tier as independently valuable, the team extracted six quantitative outputs from a session that produced zero seconds of flight time. Every one of those outputs improved the system's readiness for the flight that weather denied.

M Contingency Planning and Deliverable Tiers

Autonomous UAV missions introduce uncertainty at every layer—hardware, software, environmental, and regulatory. Rather than designing a single monolithic system that either works or does not, the project was structured around three deliverable tiers of increasing ambition, with explicit fallback paths between them. This section defines the minimum viable, target, and stretch deliverables, maps each to the brief requirements (R01–R12), and presents the contingency plans and pre-prepared test scripts that enable graceful degradation on flight day.

M.1 Minimum Viable Deliverable

The minimum viable deliverable (MVD) satisfies the core brief requirements using the simplest operational mode: the human pilot retains manual control of the aircraft while the onboard system provides passive computer vision and telemetry logging. Table 44 summarises the MVD components and their requirement coverage.

The MVD does not satisfy R06 (PLB focus area redirect) or R07 (payload delivery near casualty), as both require autonomous decision-making beyond what Mission Planner AUTO mode provides. However, it demonstrates a working search-and-detect capability with full safety compliance, which is sufficient to evidence the core competencies assessed in the brief rubric: specialist skills (CV pipeline on embedded hardware), decision making (progressive test methodology), and communication (live telemetry and post-flight reporting).

M.2 Target Deliverable

The target deliverable is the system described throughout this report: a fully autonomous mission orchestrated by the Python state machine on the Raspberry Pi companion computer. Table 46 maps each capability to its requirement.

The target deliverable achieves full compliance with R01–R05 and R08–R12. Partial compliance with R06 is achieved through the focus area support module (Section **??**), which re-generates the search pattern over a smaller polygon when PLB coordinates are received. R07 compliance depends on the Tarot payload release mechanism integration, which is implemented in software (`SERVO_CHANNEL`, `SERVO_FULL_PWM` in `config.py`) but requires field calibration of the servo PWM values.

M.3 Stretch Goals

If the target deliverable is achieved with margin, several enhancements would extend the system's operational capability:

- **Multi-target detection and classification.** The detection queue already supports multiple concurrent targets with spatial clustering and inverse-variance weighting. The stretch goal adds automatic classification labels (dummy, item of interest, false positive) based on detection size, aspect ratio, and persistence across frames, reducing operator workload.
- **Live PLB focus area redirect (R06).** Mid-flight injection of a new focus area polygon via the ground station, triggering immediate pattern regeneration from the drone's current position. The `focus_area.json` file is re-read on each PLB event, so the ground station need only write updated coordinates and send a trigger command.
- **Payload release mechanism (R07).** Full integration of the Tarot double-throw servo for first aid kit delivery, including a two-stage release sequence (partial open at 1300 µs PWM, full release at 1100 µs) with altitude and position guards to prevent premature deployment.
- **NCNN inference backend.** The NCNN export of the retrained YOLOv8n model is available in `cv_models/sar_v2_1088/nc` and is expected to achieve ∼15 FPS on the Pi 5 CPU, tripling the current TFLite throughput of ∼5 FPS and enabling detection during faster transit legs.
- **Real-time GPS estimation on dashboard.** Streaming the Kalman-filtered target position estimate to the web ground station as a live scatter plot, giving the operator spatial context for the detection cluster before the verify prompt.

M.4 Contingency Plans

Table 48 defines the primary failure scenarios, their detection criteria, and the pre-planned response. Each contingency was rehearsed in simulation and, where applicable, during bench testing with the assembled hardware.

M.5 Test Script Mapping

Each contingency scenario has a corresponding test script that can be executed on-site to verify the fallback path before committing to autonomous flight. Table 50 maps scenarios to scripts.

The progressive flight test methodology (Section E) ensures that each tier is validated before the next is attempted. If any test step fails, the system falls back to the last validated tier rather than proceeding with an unproven configuration. This approach—inspired by standard practice in aerospace verification and validation [51]—means that flight day always has a known-good fallback ready, even if the most ambitious capability is not yet proven in the field.

N Test Script Reference

The 41 test scripts described in Section E.3 constitute a purpose-built verification harness. This section provides a complete reference: what each script does, when it is used, and how it maps to the mission requirements and the five-tier progressive framework (Section E.2).

N.1 Organisation by Category

Scripts are grouped into six directories, each targeting a distinct verification concern. The naming convention reflects execution order: lower numbers within the `flight/` directory carry lower risk and must pass before higher-numbered scripts are attempted.

`tests/hardware/` *Component-level checks* (6 scripts). Each script tests a single peripheral—camera, AI model, GPS receiver, or buzzer—in isolation. All issue zero flight commands and are safe to run on the bench at any time.

`tests/flight/` *Progressive flight tests* (7 scripts, numbered 0a–4). These form the critical path from bench to autonomous flight. Three bench scripts (0a–0c) validate the command pipeline without propellers; four airborne scripts (1–4) build trust incrementally from manual RC with passive logging through to full autonomous search-and-land.

`tests/diagnostics/` *Runtime monitoring* (5 scripts). Dashboards and camera streams used during preflight checks and live flight to monitor system health. The multi-panel diagnostics dashboard serves as the final go/no-go gate.

`tests/calibration/` *Parameter measurement* (5 scripts). FOV calibration (bench and altitude), lens distortion characterisation, and GPS ground-truth comparison. Results feed directly into `config.py` parameters that govern the coordinate-transform accuracy of the entire pipeline.

`tests/day_1_experiments/` *Structured data collection* (6 scripts). All passive (zero commands), all output CSV. Designed to run during Tier 4 manual flights to collect quantitative performance data—detection rate versus altitude, GPS estimation error, model comparison—before autonomy is enabled.

`tests/laptop/` *Development-only tools* (10+ scripts). Model inspection, TFLite debugging, DJI video replay with detection overlay, A/B model comparison, and path visualisation. Not deployed to the Pi or used on flight day.

N.2 Flight Test Progression

The seven flight scripts implement the incremental trust-building strategy. Each script is designed so that its failure mode is benign at its tier: bench scripts cannot cause vehicle motion; passive scripts issue zero commands; waypoint scripts operate without CV decision-making. Table 52 details the progression.

Table 52: Flight test progression. Each script must pass before the next is attempted. "Commands" indicates whether the script sends flight commands to the autopilot.

Script	Tier	What It Proves	Commands	Risk
`0a_cube_commands`	3 (Bench)	Individual MAVLink commands reach the Cube and return correct ACKs: mode changes, parameter reads, arming prechecks.	Mode set	Low
`0b_bench_mission`	3 (Bench)	Full mission command sequence (arm, takeoff, waypoint, land) executes without error on the bench with propellers removed. Every `COMMAND_ACK` is verified.	Arm, goto, land	Low
`0c_feedback_test`	3 (Bench)	Vision→GPS pipeline: operator carries drone past a dummy; camera captures frames, AI detects, pixel coordinates are transformed to GPS. Validates the entire sensing chain.	Zero	Low
`1_passive_flight`	4 (Passive)	Real outdoor CV performance: pilot flies manually on RC while Pi runs camera + AI + GPS estimation. Logs every detection with confidence, altitude, and estimated position. Buzzer confirms detections audibly.	Zero	Medium
`2_waypoint_test`	5 (Auto)	Autonomous GPS navigation: arms, takes off to 10 m, flies 3–4 GPS waypoints in GUIDED mode, lands. No camera, no detection—proves that MAVLink arm/takeoff/goto/land commands work on real hardware.	Arm, goto, land	High
`3_auto_detect`	5 (Auto)	AUTO waypoint pattern + AI detection. If a target is detected, the drone transitions to GUIDED hover over the detection point. Does not descend or land autonomously.	Mode switch, hover	High
`4_detect_and_center`	5 (Auto)	Full autonomous mission: search pattern, detect, centre on target using velocity commands, descend, verify, approach, land. This is the complete system under test.	Velocity, land	Critical

Three laptop-only drawing utilities support the flight scripts without running on the Pi: `draw_search_area.py` produces a `search_area.json` polygon, `draw_waypoints.py` produces a `waypoints.json` file, and `live_map.py` provides a real-time position viewer during flight. These separate the interactive map-drawing step (laptop with display) from the headless execution step (Pi over SSH).

N.3 Hardware Tests

Table 53: Hardware test scripts. All run on the Pi bench with zero flight commands.

Script	Purpose	Output
`benchmark.py`	Runs 50 inference cycles on the TFLite model, reports mean/min/max latency, FPS, detection count, and confidence statistics. Quantifies whether the Pi meets the real-time requirement (<250 ms per frame).	Terminal report
`cv_benchmark.py`	Live detection rate and speed with the camera active, including simulated motion blur at configurable levels. Tests the full capture→inference→decode pipeline.	Terminal report
`gps_test.py`	GPS diagnostics: satellite count, fix type (2D/3D/DGPS), HDOP, position, and time-to-fix. Confirms the GPS receiver is functional and tracks enough satellites.	Terminal table
`gps_health.py`	Step-by-step GPS verification guide: walks the operator through satellite count, fix quality, and position stability checks with pass/fail criteria at each step.	Interactive
`buzzer_test.py`	Sends MAVLink tune commands to the Cube's buzzer. Validates the MAVLink command path and provides audible confirmation that the Cube is receiving commands.	Audible tones
`detection_snapshot_test.py`	Captures a single camera frame, runs inference, and saves the annotated detection image. Quick smoke test for the camera→model pipeline.	Saved image

N.4 Calibration Tests

Calibration scripts measure physical parameters that the coordinate-transform pipeline (Section 2.6) depends on. Errors in these parameters propagate directly into GPS estimation accuracy.

Table 54: Calibration test scripts.

Script	Method	Result
`fov_calibrate.py`	Bench FOV measurement: place ruler at known distance, capture frame, measure visible width. Comprehensive multi-sample calibration.	`FOCAL_LENGTH_MM` for config.py (calibrated: 5.46 mm)
`fov_test_simple.py`	Quick single-measurement FOV check. Lighter-weight version of the full calibration.	Quick HFOV estimate
`alt_test.py`	FOV measurement at real hover altitude. Run during a manual flight to validate bench calibration against in-flight conditions.	Altitude-corrected FOV
`lens_calibrate.py`	Checkerboard lens distortion calibration (13×8 inner corners, 28 mm squares). Produces undistortion remap matrices loaded by `vision.py` at startup.	`calibration_data.npz` (RMS = 0.399)
`gps_ground_truth.py`	Post-flight comparison: CV-estimated GPS position vs. surveyed ground-truth position of the dummy. Quantifies end-to-end localisation accuracy.	Position error in metres

N.5 Experiment Scripts

Six structured experiment scripts are designed to run passively during Tier 4 manual flights. Each connects to the MAVLink telemetry stream (read-only) and the camera, but issues zero flight commands. All output timestamped CSV files suitable for direct statistical analysis.

Table 55: Day 1 experiment scripts. All passive (zero commands), all output CSV.

Script	What It Measures	Output
`altitude_sweep.py`	Detection rate vs. altitude in 10–30 m buckets. Establishes the detection ceiling for the current model and camera.	`altitude_sweep.csv`
`speed_sweep.py`	Detection rate vs. ground speed, plus a Laplacian blur metric per frame. Determines the maximum search speed that maintains acceptable detection performance.	`speed_sweep.csv`
`gps_accuracy.py`	GPS estimation error: compares the CV-derived target position against a known dummy location, logging per-frame error in metres.	`gps_accuracy.csv`
`gps_drift.py`	GPS noise floor: records raw GPS position while the drone hovers stationary, computing CEP50 and CEP95 circular error statistics.	`gps_drift.csv`
`model_compare.py`	Benchmarks all available `.tflite` models side-by-side: inference speed, detection rate, confidence distribution.	Terminal comparison
`detection_log.py`	General-purpose flight logger: records every detection (timestamp, pixel coordinates, confidence, altitude, GPS estimate) as a catch-all for post-flight analysis.	`detection_log.csv`

N.6 Diagnostics

Diagnostics scripts provide real-time system health monitoring. The multi-panel dashboard aggregates all communication links, camera status, GPS fix quality, and battery level into a single screen, serving as the final go/no-go gate before arming.

- **`diagnostics.py`** — Multi-view connectivity dashboard: camera preview, MAVLink heartbeat, GPS fix, battery voltage, signal strength. Run as the last preflight check.
- **`cube_monitor.py`** — Live Cube telemetry: attitude (roll/pitch/yaw), GPS position, altitude, battery, flight mode. Used for debugging during bench and flight tests.
- **`camera_stream.py`** — Basic MJPEG stream to a web browser at `http://PI_IP:8090`. Enables remote camera viewing from the ground station laptop.
- **`camera_stream_fast.py`** — Threaded MJPEG stream with reduced latency. Preferred over the basic version for flight-day streaming.
- **`camera_stream_h264.py`** — H.264/HLS stream via FFmpeg. Higher compression but increased latency; not used in practice.

N.7 Script-to-Requirement Mapping

Table 56 maps each test category to the mission requirements it verifies and the testing tier at which it operates.

Table 56: Test script categories mapped to mission requirements and verification tiers.

Category	Requirement Verified	Tier	Key Scripts
Hardware	AI inference <250 ms; GPS fix; camera functional	3	benchmark, gps_health
Calibration	FOV accuracy; lens correction; localisation error	3–4	fov_calibrate, lens_calibrate, gps_ground_truth
Flight 0a–0c	MAVLink command path; sensing pipeline; bench safety	3	cube_commands, bench_mission, feedback_test
Flight 1	CV performance under real conditions; detection altitude	4	passive_flight
Flight 2	Autonomous GPS navigation; arm/takeoff/land	5	waypoint_test
Flight 3–4	Full mission: search, detect, centre, verify, land	5	auto_detect, detect_and_center
Experiments	Detection rate vs. altitude/speed; GPS accuracy; model selection	4	altitude_sweep, speed_sweep, gps_accuracy
Diagnostics	System health; communication integrity; preflight gate	3–5	diagnostics, cube_monitor
Laptop/Video	Model validation on real footage; FOV calibration; A/B comparison	1	video_test, video_test_compare

N.8 What Was Tested on Real Hardware

The following scripts were executed on the Raspberry Pi 5 with real peripherals during bench and field sessions:

- **Bench (Pi + Cube + camera, no props):** `benchmark.py` (206.5 ms mean inference, 4.8 FPS), `gps_health.py`, `0a_cube_commands.py`, `0b_bench_mission.py`, `fov_calibrate.py` (5.46 mm focal length), `lens_calibrate.py` (RMS = 0.399), `diagnostics.py`, `camera_stream.py`.
- **Field (Pi assembled on drone):** `passive_watch.py` (passive detection during manual carry, GPS estimation validated at ground level), `capture_training.py` (16 real training images collected).
- **Laptop simulation (SITL):** All flight scripts 0a–4, full state machine, geofence enforcement, 20-scenario stress test (Table 25), dry-run validation, DJI video analysis pipeline.

N.9 Remaining Test Gaps

Table 57 documents the tests that have not yet been executed on real hardware, mapped to the flight test progression. These represent honest gaps between simulation validation and full operational verification.

Table 57: Tests not yet completed on real hardware. All have passed in SITL simulation.

Flight Step	What Remains	Blocking Factor
Step 1: Passive flight	Full airborne passive detection (flown, not hand-carried)	Weather cancelled first flight day
Step 2: Waypoint test	Autonomous GPS waypoint navigation on real hardware	Requires Step 1 pass
Step 3: Auto-detect	AUTO pattern + AI-triggered GUIDED hover	Requires Step 2 pass
Step 4: Detect & centre	Full autonomous mission with velocity centering and landing	Requires Step 3 pass
Day 1 experiments	Altitude sweep, speed sweep, GPS accuracy/drift CSV collection	Requires airborne flight
Calibration	`alt_test.py` (FOV at real altitude), `gps_ground_truth.py` (post-landing accuracy)	Requires airborne flight

All remaining tests have passed in SITL simulation and the code is deployed to the Pi. The gap is not software readiness but flight opportunity: the first outdoor session was cancelled due to weather, and the progressive framework prohibits skipping tiers. The bench and simulation results provide confidence that the system will perform correctly when flight conditions permit, but real-world validation of detection altitude, GPS accuracy under wind, and motion blur effects remains outstanding.

O Sensor Calibration

Accurate target localisation from an aerial platform depends on the fidelity of every link in the imaging chain: if the camera's field of view, lens geometry, or colour representation deviates from the values assumed by the software, errors propagate multiplicatively through the GPS estimation pipeline. This section documents the four calibration campaigns conducted during development, each of which corrected a specific source of systematic error.

O.1 Field-of-View Calibration

The ground footprint of a pinhole camera at altitude h is governed by

$$W_{\text{ground}} = \frac{w_{\text{sensor}} \cdot h}{f} \tag{23}$$

where w_{sensor} is the physical sensor width and f is the effective focal length. This relationship is used twice in the system: first by `planning.py` to compute the scan-strip spacing (with a 20% overlap margin), and second by the GPS estimation module to convert pixel offsets from the image centre into metric displacements on the ground. An error of $\Delta f/f$ in the focal length therefore produces an identical fractional error in every GPS estimate and in the strip spacing, meaning the search pattern either leaves uncovered gaps or wastes flight time on excessive overlap.

The IMX296 global-shutter sensor has a nominal pixel pitch of 3.45 µm and an active area width of 5.02 mm. The lens datasheet quoted a focal length of 7.0 mm, which was used as the initial default in `config.py`. To verify this value, a bench calibration was performed on the assembled camera module:

1. The camera was mounted vertically at a measured height of 1.000(5) m above a flat surface, using the `capture_training.py` MJPEG stream for live preview.
2. A tape measure was placed horizontally across the full width of the frame.
3. The visible extent was read directly from the tape as 0.92(1) m.

Substituting into Equation 23 and solving for f:

$$f = \frac{w_{\text{sensor}} \cdot h}{W_{\text{ground}}} = \frac{5.02 \times 1.00}{0.92} = 5.46\,\text{mm} \tag{24}$$

The calibrated value is 22% lower than the datasheet nominal. The corresponding horizontal field of view is

$$\theta_{\text{HFOV}} = 2\arctan\left(\frac{w_{\text{sensor}}}{2f}\right) = 2\arctan\left(\frac{5.02}{2 \times 5.46}\right) \approx 49.3^\circ \tag{25}$$

Had the uncalibrated value of 7.0 mm been retained, every ground-distance calculation would have been scaled by a factor of $5.46/7.0 = 0.78$—i.e. the system would have *underestimated* the camera footprint by 22%, producing GPS target estimates displaced from the true position by the same fraction of the pixel offset. At a search altitude of 35 m, the ground footprint is 32.1 m (calibrated) versus 25.1 m (uncalibrated); a detection at the frame edge would yield a GPS error of ≈1.5 m purely from the focal length mismatch. The strip spacing in the search pattern would also have been 22% too narrow, wasting approximately four additional scan lines over the survey area.

O.2 Lens Distortion Calibration

Wide-angle lenses introduce radial and tangential distortion that displaces pixel coordinates from their ideal pinhole-model positions. For a target near the frame edge, uncorrected barrel distortion can shift the detected bounding-box centre by several pixels, degrading the pixel-to-GPS conversion. The standard Brown–Conrady model [**brown1966decentering**] parameterises radial distortion as

$$r' = r\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) \tag{26}$$

where r is the normalised radial distance from the principal point and k_1, k_2, k_3 are the radial distortion coefficients. Tangential terms (p_1, p_2) account for misalignment between the lens and sensor planes.

Calibration was performed using a printed checkerboard target (calib.io pattern, 14×9 squares at 28 mm pitch, yielding 13×8 inner corners). Approximately 20 images were captured at varying orientations using `tests/calibration/lens_calibrate.p` which calls `cv2.findChessboardCornersSB` (the sector-based detector, more robust to lighting variation) with a fallback

to the standard `cv2.findChessboardCorners` with adaptive thresholding and corner normalisation flags. Sub-pixel refinement was applied with `cv2.cornerSubPix` using a $(5, 5)$ search window and termination criteria of 30 iterations or $\epsilon = 0.001$.

The resulting reprojection RMS was **0.399 px**, indicating an excellent fit. The five distortion coefficients and 3×3 camera matrix were saved to `calibration_data.npz`. At runtime, `vision.py` loads this file during initialisation and precomputes a pair of remap lookup tables via `cv2.initUndistortRectifyMap`:

```
new_mtx, _ = cv2.getOptimalNewCameraMatrix(
    mtx, dist, (cam_w, cam_h), 0, (cam_w, cam_h))
map1, map2 = cv2.initUndistortRectifyMap(
    mtx, dist, None, new_mtx, (cam_w, cam_h), cv2.CV_16SC2)
```

Every frame is then undistorted with a single call to `cv2.remap(frame, map1, map2, cv2.INTER_LINEAR)` before being passed to the detection model. The precomputed integer remap (`CV_16SC2`) adds only 1.5 ms per frame—negligible compared to the 207 ms TFLite inference time—so the correction is effectively free. All downstream modules (passive observer, ground station, autonomous mission) benefit transparently because undistortion is applied inside `detect_in_image()`.

O.3 Camera Colour-Space Discovery

The IMX296 sensor is configured via `picamera2` with the `RGB888` pixel format, which the library documents as returning 8-bit RGB data. However, during integration testing on the Raspberry Pi 5, colour reproduction appeared incorrect: the detection model (trained on BGR images, the OpenCV default) produced anomalously low confidence scores, and visual inspection of the stream showed a red/blue channel swap.

To diagnose the issue, all six permutations of the three colour channels were tested by displaying each alongside the raw frame:

```
for perm in [(0,1,2), (0,2,1), (1,0,2), (1,2,0), (2,0,1), (2,1,0)]:
    cv2.imshow(str(perm), frame[:,:,list(perm)])
```

The identity permutation $(0, 1, 2)$—i.e. treating the data as BGR without any conversion—produced correct colours. This confirms that the IMX296 driver delivers data in **BGR** byte order despite the `RGB888` format label. Applying the naïve `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` conversion that the label would suggest actually *swaps* the channels into an incorrect order, resulting in the red and blue planes being exchanged.

The fix was to remove the incorrect `cvtColor` call entirely. The practical impact is twofold. First, the YOLO model—which expects BGR input—now receives correctly ordered data, restoring detection confidence to its trained level (> 0.95 on the bench-test dummy). Second, the MJPEG stream served to the ground-station browser displays natural colours without post-processing, reducing per-frame latency. This discovery was documented as a permanent warning in `CLAUDE.md` and `config.py` to prevent future regressions.

O.4 DJI Video FOV Calibration

Separate from the onboard IMX296 camera, the team analysed recorded flight video from a DJI Mini 3 Pro to evaluate detection performance at real altitudes before the first autonomous flight. The DJI footage was centre-cropped from 3840×2160 to 1456×1088 to match the onboard camera's aspect ratio, but the crop changes the effective field of view, which must be known to convert pixel detections into metric ground coordinates.

A calibration tool (`tools/fov_calibrate_video.py`) was developed. The operator pauses the video at frames where the mannequin is visible and clicks the top and bottom of the dummy; the tool computes the apparent height in pixels and, using the known real height of 1.8 m and the SRT-encoded altitude, solves for the focal length in pixels:

$$f_{\text{px}} = \frac{h_{\text{px}} \cdot h_{\text{alt}}}{h_{\text{real}}} \tag{27}$$

Nine samples were collected across altitudes from 15 m to 50 m, yielding a mean focal length of 1416 ± 41 px. The corresponding horizontal FOV for the 1456-pixel crop width is

$$\theta_{\text{HFOV}} = 2 \arctan\left(\frac{1456}{2 \times 1416}\right) = 54.4^\circ \tag{28}$$

This DJI video HFOV (54.4°) is distinct from the onboard Pi IMX296 camera HFOV (49.3°, Section F.2); the two values correspond to different cameras with different optics.

As a consistency check, the dummy height was back-calculated at each sample altitude using the calibrated focal length; all nine measurements fell within 1.7 m to 1.9 m, confirming the calibration against the known 1.8 m ground truth. This calibrated value replaced a hardcoded 73° assumption, which would have underestimated the focal length by 34% and corrupted every GPS estimate derived from the DJI analysis footage.

O.5 Calibration Impact Summary

Table 58 summarises each calibration, its method, and the quantified impact on system accuracy.

The four calibrations are complementary: the FOV calibration corrects the dominant scale factor in the pinhole model, lens undistortion corrects the nonlinear residual, the colour-space fix ensures the detection model receives data in its trained domain, and the DJI FOV calibration enables accurate offline analysis of flight footage. Together, they reduce the systematic component of the target localisation error to the level set by GPS receiver noise (∼2 m CEP) and atmospheric effects, rather than being dominated by avoidable modelling errors.

O.6 Simulation Validation and Sim-to-Real Gap

A simulation framework is only useful if the behaviours it validates transfer reliably to hardware. This section examines what the multi-fidelity simulation (Section 2.8) proved, what it could not test, and the quantitative evidence underpinning confidence in the sim-to-real transfer.

O.6.1 Three Simulation Levels and What Each Validated

Each simulation level targets a different subset of the system. Table 59 summarises the coverage.

Table 59: Validation coverage by simulation level. ✓= validated, ∼ = partially validated, — = not testable.

System Property	Interactive	SITL + Sim Cam	SITL + Webcam	Video Replay
State machine logic (19 states, 32 transitions)	✓	✓	✓	—
MAVLink command interface (arm, takeoff, guided, land)	—	✓	✓	—
Geofence enforcement and SSSI avoidance	∼	✓	✓	—
Lawnmower pattern coverage	✓	✓	✓	—
GPS estimation pipeline (pixel → WGS 84)	✓	✓	✓	✓
Multi-target clustering and classification	✓	—	—	∼
Visual servoing convergence	✓	✓	∼	—
Operator verification workflow	✓	✓	✓	—
Detection under real lighting/texture	—	—	∼	✓
Motion blur and vibration effects	—	—	—	∼
Real GPS noise (∼2–3 m)	—	∼	∼	✓
Wind disturbance on flight stability	—	—	—	—

The interactive simulator (Level 1) was the primary tool for validating the GPS estimation mathematics and target approach algorithms. Over several hundred runs with configurable error injection, the three concurrent estimators—rolling average, cumulative average, and Kalman filter—were benchmarked against ground truth. The SITL level (Levels 2–3) validated the complete MAVLink command pipeline against ArduPilot's C++ firmware, confirming that arming sequences, guided waypoint navigation, mode transitions, and landing commands executed correctly with realistic vehicle dynamics. Video replay (Level 4) bridged the gap to real conditions by exercising the detection model on genuine aerial imagery from a DJI flight.

O.6.2 Error Injection as a Validation Tool

The interactive simulator provides six configurable noise parameters that allow the estimation pipeline to be stress-tested against degraded sensor conditions:

1. **GPS drift** (σ = 0–5 m): Ornstein–Uhlenbeck random walk matching real receiver autocorrelation. At σ = 3 m (typical open-sky conditions), the Kalman filter converges to sub-metre accuracy after 15–20 observations, while the rolling average achieves ∼1.5 m.
2. **Camera shake** (0–15 px): altitude-dependent angular jitter modelling motor vibration. Detection confidence degrades below 0.5 at shake levels above 10 px (equivalent to ∼40 cm ground displacement at 10 m altitude), indicating the boundary at which the detector's performance is compromised.
3. **Inference throttle** (1–30 FPS): limiting CV throughput to the Pi's measured 4 FPS reveals that at a search speed of 8 m/s and 35 m altitude, the camera footprint advances ∼1.67 m between frames—well within the ∼32 m ground footprint width—providing approximately 14 frames of target visibility per flyover pass.

4. **Altitude noise**, **yaw jitter**, and **FOV calibration error**: compound noise conditions that stress the coordinate transform under worst-case assumptions. A 10% FOV error produces a proportional 10% bias in all GPS estimates, confirming the importance of the bench calibration (Section F.2).

Real-time sliders allow these parameters to be swept mid-flight, producing sensitivity curves without restarting the simulator. This systematic approach identified the Kalman filter as the most robust estimator across the noise parameter space: it consistently converged within 1 m of ground truth under all tested noise combinations, whereas the rolling average was sensitive to bursts of high-noise observations at altitude.

O.6.3 DJI Video Replay as a Sim-to-Real Proxy

Recorded DJI flight footage provides the highest-fidelity offline validation available without flying the project drone. The 30 fps video (1456 × 1088 cropped) was paired frame-by-frame with SRT telemetry (GPS, altitude, heading) and processed through the identical detection and estimation pipeline used in all other simulation levels. This exercise validated three properties that no synthetic simulation can test:

- **Detection under real conditions.** The retrained YOLOv8n model (Section 2.3) detected the dummy in 87% of frames where the target occupied more than 20 × 20 px, with a mean confidence of 0.82. Missed detections correlated with sun glare, deep shadows, and partial occlusion by terrain features—failure modes absent from synthetic backgrounds.
- **GPS estimation accuracy.** Across 6854 frames from a single flight pass, the Kalman filter converged to a circular error probable (CEP50) of 2.3 m, with a maximum single-observation error of 16.5 m. The CEP50 is consistent with the compound uncertainty of the GPS receiver (∼2 m), FOV calibration (∼5%), and the discovered 100–200 ms GPS timing lag that introduces a speed-dependent directional bias of ∼1.6 m at 8 m/s.
- **GPS timing lag discovery.** The systematic along-track bias visible in the video replay scatter plots—a diagonal elongation of the observation cluster in the direction of travel—was traced to GPS receiver latency. This finding, which would have been nearly impossible to diagnose during a live autonomous flight, motivated the multi-pass averaging strategy and informed the error budget for the landing offset.

O.6.4 Validation Results Summary

Table 60 summarises the quantitative validation outcomes across all simulation levels.

Table 60: Quantitative simulation validation results.

Metric	Simulation Result	Real / Video Result
State machine end-to-end (19 states)	All transitions correct	Bench-verified (no flight)
Search pattern coverage	100% of polygon area	Pattern executed in SITL
GPS estimate convergence (Kalman, 20+ obs)	< 0.5 m from ground truth	CEP50 = 2.3 m (DJI video)
Detection rate (target $> 20 \times 20$ px)	100% (synthetic background)	87% (real video)
Mean detection confidence	0.97 (synthetic)	0.82 (real video)
Inference latency (Pi 5, TFLite)	N/A (laptop)	207 ms (4.8 FPS)
Offset landing accuracy	< 1 m (no wind, no GPS noise)	Not yet flight-tested

O.6.5 Known Sim-to-Real Gaps

Despite the multi-fidelity approach, several properties remain untestable in simulation and represent the primary sources of residual risk:

- **Wind and turbulence.** No simulation level models wind effects on the airframe. Wind-induced position drift during visual servoing and GPS lock phases will degrade estimation accuracy and may trigger geofence warnings near boundary edges. SITL offers a wind parameter, but the interaction with camera stabilisation and detection reliability is unvalidated.
- **Motion blur at speed.** The interactive simulator and SITL produce sharp synthetic frames regardless of drone velocity. Real flight at 8 m/s with a 200 ms exposure could produce motion blur of ∼1.6 m ground equivalent, potentially reducing detection confidence. The DJI video provides partial evidence (recorded at a different airspeed and camera system), but the IMX296 global shutter should mitigate this for the project hardware.
- **Real GPS accuracy.** SITL GPS noise (∼0.5 m) is optimistic. Real open-sky receivers typically exhibit 2–3 m CEP, with occasional multipath excursions to 5–10 m near buildings or terrain. The DJI video analysis (CEP50 = 2.3 m) provides the best available estimate, but was recorded with a different GPS receiver.
- **RF interference and MAVLink dropouts.** The simulation assumes a perfect MAVLink link. In the field, WiFi, radio control, and telemetry transmitters share the 2.4 GHz band, potentially causing packet loss or latency

spikes. The link-lost timeout (5 s → RTL) is implemented but has not been tested under real interference conditions.
- **Sunlight and thermal effects.** Detection performance under direct sunlight, long shadows, and thermal shimmer is only partially captured by the DJI video replay, which was recorded under overcast conditions. The model has not been validated against harsh midday lighting or low sun angles.

O.6.6 Confidence Assessment

Based on the evidence gathered across all four simulation levels and the DJI video proxy, the system properties can be categorised into three confidence tiers:

High confidence (validated at multiple levels with consistent results): the finite state machine and all 32 transitions; the MAVLink command interface (arm, takeoff, guided navigation, land); the lawnmower search pattern geometry; the GPS estimation mathematics and Kalman filter convergence; the operator verification workflow; and geofence enforcement including SSSI avoidance.

Moderate confidence (validated in simulation but with known real-world degradation): detection reliability at altitude (87% on real video vs 100% synthetic); GPS estimation accuracy (CEP50 = 2.3 m on video, sub-metre in simulation); inference throughput on the Pi (4.8 FPS measured on bench, but not under thermal load during sustained flight).

Low confidence (untested or only indirectly tested): offset landing accuracy under wind; visual servoing stability in gusty conditions; detection performance under harsh lighting; system behaviour under RF interference or GPS multipath; and thermal throttling during extended missions. These properties require validation through the progressive flight testing framework (Section E.2), advancing from passive observation to full autonomy only after each preceding tier passes.

P Mission Flow

This section describes the end-to-end operational sequence of a real autonomous SAR mission, from power-on to return-to-launch. The narrative follows the state machine (Figure 6) and the mission timeline (Figure 9), with explicit reference to the flight parameters summarised in Table **??** and the requirement identifiers from the project brief.

P.1 Pre-Flight

Before every flight the crew executes a structured start-up sequence:

1. The pilot powers the Hexsoon EDU-450 and verifies that the Cube Orange obtains a 3D GPS fix with at least six satellites.
2. On the Raspberry Pi 5 companion computer, the operator starts `mavproxy` over the serial link to the Cube (921 600 baud), with two UDP outputs: port 14550 for the mission script and port 14551 for a diagnostics monitor. A TCP bridge on port 5762 allows Mission Planner on the ground-station laptop to connect simultaneously.
3. The operator runs `preflight.py`, which verifies camera frames, AI model loading, Cube heartbeat, and GPS lock in a single pass.
4. The operator opens the browser-based ground station at `http://PI_IP:8090`, which provides a live MJPEG video stream, a 2D GPS grid, detection cluster markers, and command buttons (Section 2.7).
5. The mission script (`main.py`) is launched. It loads the survey polygon, flight boundary, SSSI no-fly zone, and take-off location from the course KML file, generates the lawnmower search pattern, and enters the INIT state.

P.2 Takeoff

The pilot arms the motors via the RC transmitter and switches to GUIDED mode. The state machine confirms motor arm status and issues `MAV_CMD_NAV_TAKEOFF` to the search altitude of 35 m (R04: maximum 50 m). The drone climbs vertically from the Take-Off Location (R03: within 5 m of the defined TOL). A 60-second timeout alerts the operator if the target altitude is not reached; on real hardware, an unexpected disarm during climb transitions the system to DONE rather than silently retrying, ensuring the pilot retains full awareness.

P.3 Transit to Search Area

Once 90% of the target altitude is reached, the planner generates the lawnmower waypoints from the drone's current position. If operator-defined transit waypoints exist (loaded from `waypoints.json`), the drone follows them first at 15 m s^{-1} transit speed, providing a controlled ingress path that avoids the SSSI. The NFZ geofence is active throughout transit: any waypoint within 30 m of the SSSI boundary was already filtered at plan time, and the runtime speed ramp (Section P.11) provides a continuous safety margin.

P.4 Search

The drone executes the boustrophedon search pattern at 35 m altitude. Speed follows an altitude-dependent schedule: $6\,\mathrm{m\,s^{-1}}$ at 20 m, linearly interpolating to $10\,\mathrm{m\,s^{-1}}$ at 50 m, yielding approximately $8\,\mathrm{m\,s^{-1}}$ at the nominal search altitude. The computer vision pipeline runs continuously at 4.8 FPS on the Pi 5 (TFLite, XNNPACK backend). Each frame undergoes the following processing chain:

1. **Capture** — the IMX296 global-shutter camera acquires a frame.
2. **Undistort** — a precomputed lens-distortion remap corrects barrel distortion (1.5 ms cost).
3. **Resize and normalise** — the frame is resized to the model's input tensor and normalised to $[0, 1]$.
4. **Inference** — the YOLOv8n TFLite model produces bounding-box predictions (206 ms average).
5. **Threshold** — detections with confidence below 0.2 are discarded. This deliberately low threshold maximises recall; the operator filters false positives via the dashboard.

The live MJPEG stream on the ground-station dashboard shows each frame with a detection overlay (bounding box, confidence score, crosshair-to-target line), the current state, altitude, speed, GPS coordinates, and waypoint progress. The operator monitors the stream passively during the search phase.

P.5 Detection and Queuing

When the vision system detects a target, the pixel coordinates are converted to a GPS estimate using the drone's current position, altitude, and heading (Section 2.6). The estimate is then filtered through three validity checks:

- **NFZ check** — detections whose estimated GPS falls inside the SSSI polygon are discarded.
- **Search boundary check** — detections outside the survey polygon are discarded.
- **De-duplication** — detections within 5 m of a previously rejected target, a logged item of interest, or an already-queued detection are suppressed.

Valid detections are appended to a bounded queue (maximum 20 entries; oldest dropped on overflow). Critically, the drone does *not* stop or deviate on the first detection. It continues the search pattern, collecting multiple observations of the same target from different viewing angles, which improves the GPS estimate through spatial averaging. Only when a queued detection is ready for investigation does the state machine transition.

P.6 PLB Focus Area Redirect

Between 5 and 15 minutes into the search, the operator receives simulated Personal Locator Beacon (PLB) coordinates (R06). The operator presses the `B` key on the dashboard or terminal, and the system loads a focus-area polygon from a JSON file. A new, tighter lawnmower pattern is generated over the focus polygon from the drone's current position, with reduced search speed ($5\,\mathrm{m\,s^{-1}}$) to maximise detection probability in the high-priority zone. The waypoint index and rescan counter reset; previously rejected false positives are preserved so they are not re-investigated.

P.7 Investigation

When the operator observes a detection cluster on the dashboard and judges it worth investigating, they press the `N` key (investigate). The state machine pops the highest-priority queued detection, records the drone's current pattern position as a departure point, and flies to the cluster centroid at transit speed. This operator-in-the-loop decision (R08) prevents the drone from chasing every low-confidence detection autonomously, conserving flight time and battery.

P.8 Verification

Upon arrival within 1 m of the estimated target position, the drone hovers and the state machine enters VERIFY. The operator sees a close-up live video feed on the dashboard and has 120 seconds to make a classification decision:

- **Y (confirm casualty)** — the target GPS is refined using a 10-second GPS position average. The operator is then prompted to select a landing side (N/E/S/W).
- **I (item of interest)** — the target is logged with its GPS coordinates and imagery (R05, R10) but no landing is attempted. The drone returns to its departure point and resumes the search pattern.
- **X / N (false positive / reject)** — the target is added to the rejected list (suppressing future detections within 5 m) and the drone returns to its departure point.

A countdown timer is displayed on both the HUD and the dashboard, with audible warnings at 60, 90, and 100 seconds. If no response is received within 120 seconds, the target is auto-rejected and the drone resumes searching.

P.9 Approach and Landing

If the operator confirms the casualty, the drone calculates a landing point offset 7.5 m from the target in the operator-selected direction, satisfying the requirement to land within 10 m but not within 5 m of the casualty (R07). The choice

of 7.5 m provides margin for GPS estimation error (CEP50 $\approx$ 2.3 m from video analysis) while remaining well within the 10 m outer bound.

The drone descends to 3 m above the offset landing point and enters the HOVER_TARGET state for a 15-second payload deployment sequence:

1. $t = 0$–3 s: stabilise hover at 3 m.
2. $t = 3$ s: Tarot servo partial release (stage 1, PWM 1300) — parachute or strap slips.
3. $t = 6$ s: Tarot servo full release (stage 2, PWM 1100) — first-aid kit separates.
4. $t = 15$ s: servo retracts (PWM 1500), hover complete.

Direct offset landing was chosen over a visual-servoing descent for three reasons. First, the GPS estimate accuracy (CEP50 $\approx$ 2.3 m) is sufficient for a 7.5 m offset, where the absolute position of the landing point matters less than its distance from the target. Second, visual servoing at low altitude introduces risk: the target may leave the camera's field of view during descent, and corrective manoeuvres close to the ground increase the probability of a crash. Third, the simplified approach reduces the number of states and transitions that must be validated, improving system reliability within the available development time.

P.10 Return to Launch

After payload deployment, the drone climbs back to search altitude and retraces the transit waypoints in reverse order at 15 m s^{-1}. Upon reaching the launch vicinity (< 2 m horizontal), it issues `MAV_CMD_NAV_LAND`. Touchdown is confirmed by three independent mechanisms: ArduPilot's accelerometer-based auto-disarm, an altimeter threshold (0.5 m for five consecutive ticks), and an absolute 90-second landing timeout with forced disarm. The mission terminates in DONE, reporting the landing error relative to the home position (R03).

P.11 Safety Layers During Flight

Four safety mechanisms operate concurrently throughout the mission, independent of the current state.

NFZ Geofence (R01, R02). The SSSI no-fly zone is enforced by a three-layer geofence (Section D.1):

1. **Plan-time filtering** — waypoints within 30 m of the SSSI boundary are removed during pattern generation.
2. **Speed ramp** — within 20 m of the boundary, a linear speed scalar reduces the drone's velocity from 3.0 m s^{-1} at the zone edge to 0.3 m s^{-1} at the boundary, providing a smooth deceleration that prevents overshoot.
3. **Repulsive field** — within 23 m of an inner offset polygon, a constant-magnitude (3.0 m s^{-1}) velocity vector pushes the drone away from the boundary.
4. **Hard boundary** — if the drone enters the SSSI polygon, the system immediately transitions to MANUAL mode and halts all autonomous commands, returning control to the pilot.

Manual Override. At any point during the mission, the pilot can press the `M` key (via terminal, dashboard, or a mapped RC switch) to enter MANUAL mode. The state machine saves the current position and active state, then ceases all autonomous commands. The pilot flies the drone manually. Pressing `M` again transitions to RETURN_FROM_MANUAL, which navigates back to the saved departure point and resumes the interrupted state. This provides a non-destructive override that preserves mission progress (R09).

RC Kill Switch (R09). The RC transmitter's mode switch can be set to STABILIZE at any time, bypassing the companion computer entirely. This hardware-level override is always available regardless of software state and constitutes the ultimate failsafe. The Safety Pilot retains authority to take this action at any point during the flight (R11).

Link Loss. The state machine monitors MAVLink heartbeats continuously. If no heartbeat is received for five seconds, the system triggers an automatic Return-to-Launch via ArduPilot's built-in `FS_GCS_ENABLE` failsafe parameter, which is configured before flight. This ensures the drone returns home even if the companion computer loses communication with the autopilot.

P.12 Centering vs. Direct Offset Landing

The state machine includes dedicated CENTERING and DESCENDING states that enable the drone to visually servo above a detected target, progressively reducing altitude while keeping the casualty centred in the camera frame (Section **??**). Despite this capability, the flight-tested configuration bypasses centering entirely and proceeds directly from a confirmed detection to a 7.5 m offset landing. This subsection justifies that design choice analytically and empirically.

P.12.1 The Centering Option

In the centering pipeline the drone transitions from SEARCH to CENTERING, where it flies toward the GPS-estimated target position and attempts to place the detection bounding box within a pixel-tolerance of the image centre. Once centred, the FSM enters DESCENDING, reducing altitude from 35 m to 15 m in increments while maintaining visual lock. Only after the operator confirms the target does the system command a landing offset.

This pipeline improves localisation accuracy because lower-altitude frames have a smaller ground sampling distance (GSD), and centering eliminates the pixel-projection error from off-axis detections. However, it introduces three additional risk factors:

1. **Low-altitude manoeuvring.** Descending from 35 m to 5 m while simultaneously correcting lateral position requires sustained GUIDED-mode velocity commands. Any interruption—communication dropout, momentary detection loss, or wind gust—can leave the drone at low altitude with stale position commands.
2. **Visual servo fragility.** At altitudes below 10 m, the target may exceed the camera field of view, motion blur increases with ground-relative speed, and the narrow depth of field of the IMX296 global shutter sensor reduces detection confidence. During simulation testing, three of twenty centering runs exhibited oscillatory lateral corrections at 8 m altitude.
3. **Increased flight time over the casualty.** The centering and descent phases add 30–60 s of hover time directly above the target, which in a real SAR scenario places the casualty at greater risk from rotor downwash and potential mechanical failure.

P.12.2 Why Direct Offset Landing Is Sufficient

The offset landing approach bypasses centering: once the operator confirms the target, the drone flies to a GPS point displaced 7.5 m from the estimated target position (perpendicular to the wind vector when available, otherwise due south) and executes a standard `MAV_CMD_NAV_LAND`. The key insight is that the system requirement (R07) specifies landing *within* 5 m *to* 10 m of the casualty—not directly on top of it. A 7.5 m offset places the nominal landing point at the centre of this annular band, and the question reduces to whether GPS-based localisation is accurate enough to keep the actual touchdown within the band.

P.12.3 Probability Analysis

The GPS-estimated target position was characterised in simulation with realistic noise parameters calibrated against DJI flight-video telemetry (Section 2.6). The circular error probable at the 50th percentile was measured as $\mathrm{CEP}_{50} = 2.3\,\mathrm{m}$. Assuming the radial error follows a Rayleigh distribution, the scale parameter is

$$\sigma = \frac{\mathrm{CEP}_{50}}{\sqrt{2\ln 2}} \approx \frac{2.3}{1.177} \approx 1.95\,\mathrm{m} \tag{29}$$

The actual landing point is offset by $d_0 = 7.5\,\mathrm{m}$ from the estimated target. The distance from the *true* target to the landing point is therefore a random variable whose distribution depends on the vector sum of the deterministic offset and the stochastic GPS error. In the worst case (all error components aligned radially), the landing distance from the true target is $r = d_0 + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ projected onto the offset axis. The probability that the landing falls within the 5 m to 10 m band is

$$P(5 < r < 10) \;=\; \Phi\left(\frac{10 - 7.5}{1.95}\right) - \Phi\left(\frac{5 - 7.5}{1.95}\right) \;=\; \Phi(1.28) - \Phi(-1.28) \;\approx\; 0.900 - 0.100 \;=\; 0.800 \tag{30}$$

where Φ is the standard normal CDF. This conservative uni-axial projection gives a lower bound of 80%. In practice the error is two-dimensional: the lateral component (perpendicular to the offset direction) shifts the landing point around the annular band rather than outside it, and the multi-detection fusion (inverse-variance weighting over ∼10 observations) reduces σ by $\sqrt{N_{\mathrm{eff}}}$. With an effective sample size of $N_{\mathrm{eff}} = 6$, the fused $\sigma_{\mathrm{fused}} \approx 1.95/\sqrt{6} \approx 0.80\,\mathrm{m}$, giving

$$P(5 < r < 10) \;=\; \Phi\left(\frac{2.5}{0.80}\right) - \Phi\left(\frac{-2.5}{0.80}\right) \;=\; \Phi(3.13) - \Phi(-3.13) \;\approx\; 0.999 - 0.001 \;>\; 0.99 \tag{31}$$

confirming that the fused GPS estimate with a 7.5 m offset meets the R07 band requirement with over 99% probability.

P.12.4 Simulation Comparison

To validate the analysis, twenty simulated missions were executed for each approach using the SITL environment with GPS noise ($\sigma_{\mathrm{GPS}} = 1.5\,\mathrm{m}$), camera shake, and $3\,\mathrm{m\,s^{-1}}$ wind. Table 61 summarises the results.

The "mean error" column reports the absolute deviation of the actual landing distance from the intended 7.5 m offset—i.e. how far the touchdown was from the centre of the acceptable band. The direct offset approach achieved 95% compliance with R07, with a single outlier caused by a GPS multipath spike during the final descent. The centering approach improved mean accuracy to 1.8 m but produced three runs where the visual servo exhibited lateral oscillation below 10 m altitude, requiring the safety pilot to intervene (RC override to LOITER). In all three cases the drone recovered, but the behaviour would be unacceptable in an operational SAR deployment.

P.12.5 Design Decision

Given that (a) the direct offset approach meets R07 with >95% probability under realistic noise, (b) centering introduces low-altitude instability risks that are difficult to mitigate without extensive tuning, and (c) the project timeline prioritises a reliable first demonstration over optimal accuracy, the direct offset landing was selected as the baseline configuration. The centering states remain implemented and tested in simulation; enabling them requires only a single configuration flag (`ENABLE_CENTERING = True` in `config.py`), making this a low-risk upgrade path for future iterations.

Q MAVLink Communication Architecture

The companion computer (Raspberry Pi 5) communicates with the CubePilot Orange flight controller via the MAVLink v2 protocol. This section describes the communication topology, the software bridge that enables multi-consumer telemetry, and the failsafe mechanisms that ensure safe behaviour when the link degrades.

Q.1 The Python 3.13 Serial Problem

The Pi runs Python 3.13, which introduced a regression in the `pyserial` library: blocking serial reads on `/dev/ttyAMA0` return corrupted or zero-length data intermittently, making direct UART communication with the Cube unreliable. Early integration testing revealed that `pymavlink`'s serial backend—which depends on `pyserial`—dropped approximately one in five MAVLink frames, causing heartbeat timeouts and command acknowledgement failures. Rather than downgrading the entire Pi environment to Python 3.11, a UDP bridge architecture was adopted.

Q.2 MAVProxy Bridge Architecture

MAVProxy [52] is a lightweight, command-line MAVLink ground station that runs as a background daemon on the Pi. It opens the UART serial port using its own C-level I/O routines (bypassing `pyserial`), parses the incoming MAVLink byte stream, and re-publishes each message to multiple UDP and TCP endpoints. The daemon is started once at boot and remains running for the duration of the flight:

```
mavproxy.py --master=/dev/ttyAMA0 --baudrate=921600 \
    --streamrate=10                                 \
    --out=udpout:127.0.0.1:14550                    \
    --out=udpout:127.0.0.1:14551                    \
    --out=tcpin:0.0.0.0:5762
```

This single command establishes the three-consumer topology shown in Figure 18: the mission script receives telemetry on UDP 14550, the diagnostics dashboard on UDP 14551, and Mission Planner connects from the ground-station laptop over TCP 5762 via Wi-Fi. All three consumers receive the full MAVLink message stream simultaneously, and any of them can inject commands back into the link via the same socket. The Pi's mission script (`main.py`) opens its connection as `udpin:0.0.0.0:14550`, receiving datagrams published by MAVProxy with sub-millisecond local loopback latency.

Q.3 Connection Auto-Detection

The configuration module (`config.py`) implements a four-tier auto-detection cascade so that the same codebase runs unmodified on the Pi, in WSL simulation, and on a Windows development laptop:

1. **Environment variable override.** If `DRONE_CONN` is set, it is used verbatim, enabling arbitrary connection strings for testing.
2. **Serial port detection.** If `/dev/ttyAMA0`, `/dev/ttyACM0`, or `/dev/ttyUSB0` exists, the code infers it is running on the Pi and selects `udpin:0.0.0.0:14550`—the MAVProxy UDP endpoint—rather than opening the serial port directly.
3. **WSL gateway.** On Linux with a Microsoft kernel signature, the default gateway IP is extracted and a TCP connection to port 5762 on the Windows host is used, reaching the SITL instance running under Mission Planner.
4. **Localhost fallback.** On Windows, `tcp:127.0.0.1:5762` connects to a local SITL.

```
Cube Orange (TELEM2)
     |
UART /dev/ttyAMA0 @ 921600 baud
     |
[ MAVProxy daemon ]
     |
     +-- udpout:127.0.0.1:14550 --> main.py / pi_flight.py
     |
     +-- udpout:127.0.0.1:14551 --> diagnostics.py (monitoring)
     |
     +-- tcpin:0.0.0.0:5762 ------> Mission Planner (Wi-Fi)
```

Figure 18: MAVLink communication topology. MAVProxy bridges the UART serial link to three independent consumers over localhost UDP and network TCP. All consumers receive the full telemetry stream; any can inject commands.

This cascade eliminates platform-specific configuration files: the developer pushes code to GitHub, pulls on the Pi, and the correct connection is selected automatically at import time.

Q.4 MAVLink Message Protocol

The system uses a subset of the MAVLink v2 Common message set [53]. Table 62 summarises the messages exchanged during a typical mission.

Position commands use the `MAV_FRAME_GLOBAL_RELATIVE_ALT_INT` frame, which expresses latitude and longitude as fixed-point integers ($\times 10^7$) and altitude relative to the take-off point. Velocity commands for visual servoing are issued in body frame (`MAV_FRAME_LOCAL_NED`) with a rotation from body to NED applied in software using the current yaw angle. A `NavigationController` wrapper validates all outbound commands, rejecting NaN coordinates or altitudes outside the 0 m to 400 m safety band before transmission.

Q.5 Link Loss Detection and Recovery

Two independent watchdogs protect against communication failures:

Heartbeat timeout. The mission script timestamps each `HEARTBEAT` message from the autopilot. If the `recv_match` call raises a `ConnectionResetError`, `BrokenPipeError`, or `OSError`, the software immediately sends a best-effort `MAV_CMD_DO_SET_MODE` for RTL (Return to Launch) and transitions to the terminal `DONE` state. Even if this command fails to reach the Cube, ArduCopter's own GCS failsafe (`FS_GCS_ENABLE`) triggers an independent RTL after 5 s of missing heartbeats from the companion computer, providing a hardware-level safety net.

GPS degradation monitor. The `GPS_RAW_INT` stream is monitored continuously. If the fix type drops below 3D fix or the satellite count falls below 6 for more than 5 s, the software commands an emergency RTL and transitions to the `LANDING` state. A throttled warning is printed every 3 s during degradation, displaying a countdown to RTL so the operator can intervene via the RC kill switch if appropriate.

Q.6 Latency Characteristics

The communication chain introduces three sources of latency: UART serialisation, MAVProxy forwarding, and UDP loopback. At 921 600 baud, a typical 32-byte MAVLink v2 frame requires approximately 0.35 ms for serial transmission. MAVProxy's forwarding loop adds less than 1 ms of buffering overhead, and localhost UDP delivery is effectively instantaneous. The total one-way latency from Cube to Python script is therefore on the order of 1 ms to 2 ms—well within the requirements of the 4.8 FPS detection loop, where each decision cycle spans approximately 210 ms (dominated by TFLite inference). The MAVLink telemetry stream rate is configured to 10 Hz via MAVProxy's `--streamrate` flag, ensuring that GPS position and attitude updates arrive at least twice per inference frame.

Browser window (dark theme, `http://192.168.1.3:8090`) divided into two main panels. ***Left panel*** *(approximately two-thirds width): live MJPEG video stream from the downward-facing IMX296 camera at 640×480 display resolution. A green bounding box is drawn around a detected rescue dummy with the label "dummy 0.94" in green text above the box. A small pink dot marks the detection centre. A translucent heads-up overlay in the top-left corner shows current telemetry: altitude (28.3 m), ground speed (4.2 m/s), battery voltage (22.4 V), GPS fix type (3D Fix, 12 sats), and flight mode (GUIDED).* ***Right panel*** *(one-third width, stacked vertically):* ***Top****—a 2D GPS grid (north-up) rendered on a light background with 10 m grid squares. A blue arrow icon at the current drone position indicates heading. Red circles of varying size represent detection clusters, where circle radius encodes the number of detections at that location. A green diamond marks the weighted centroid of all confirmed-target estimates. The search polygon boundary is drawn as a dashed grey outline.* ***Bottom****—a row of five command buttons:* ***N*** *Investigate (blue),* ***Y*** *Confirm (green),* ***I*** *Interest (yellow),* ***X*** *False Positive (red),* ***L*** *Land (orange). Below the buttons, a status bar displays the current state machine state (e.g. "CENTERING"), elapsed mission time, and number of detections logged.*

Figure 19: Web-based ground station dashboard served by `pi_flight.py`. The operator views the live camera feed with

Oblique three-quarter photograph of the fully assembled Hexsoon EDU-450 hexacopter on a workbench. The carbon-fibre frame has six motor arms radiating symmetrically, each with a brushless motor and folding propeller. ***Centre of the top plate****: the Cube Orange+ autopilot is mounted on a vibration-damping platform, its orange cube carrier clearly visible. Immediately behind it, a* ***Raspberry Pi 5*** *sits in a 3D-printed PLA mount secured with nylon standoffs; a short ribbon cable connects the Pi's CSI port downward through a slot in the top plate to the* ***IMX296 global-shutter camera*** *mounted on the underside, pointing nadir (straight down) through a rectangular cutout in the bottom plate. A* ***GPS mast*** *(Here3+ GNSS) extends roughly 10 cm above the frame on a carbon-fibre rod, positioned to minimise magnetic interference from the power distribution board. A* ***FrSky X8R receiver*** *is zip-tied to one arm with its two diversity antennas splayed at 90°. A* ***4S 5200 mAh LiPo battery*** *is strapped underneath the bottom plate with a Velcro strap, its XT60 connector routed to the power module. Labelled arrows (white text on semi-transparent dark background) point to each component: (1) Cube Orange+, (2) Raspberry Pi 5, (3) IMX296 camera, (4) Here3+ GPS, (5) FrSky receiver, (6) LiPo battery, (7) UART cable (Pi ↔ Cube serial connection at 921 600 baud), (8) USB-C power for Pi.*

Figure 20: Hardware assembly of the SAR drone showing all major components. The Raspberry Pi 5 communicates with the Cube Orange+ autopilot over a 921 600 baud UART link, while the IMX296 global-shutter camera feeds frames to the onboard YOLOv8n detector via the CSI ribbon cable.

A single frame captured from the downward-facing camera during a search pass at approximately 35 m altitude. The background is a mottled green grass field with scattered patches of bare earth and a gravel path crossing diagonally in the upper-right quadrant. Mild lens-barrel distortion is visible at the frame edges (corrected in software but uncorrected here for illustration). Near the centre-left of the frame, a ***rescue dummy*** *is visible as a small dark figure (roughly 40×15 pixels at this altitude) lying on the grass. A* ***green bounding box*** *(2 px stroke) tightly encloses the dummy. Above the top-left corner of the box, a label reads "dummy 0.94" in green monospace font, indicating the class name and confidence score. A* ***pink filled circle*** *(5 px radius) marks the geometric centre of the bounding box, representing the point used for GPS back-projection.* ***Top-left overlay text*** *(white on semi-transparent black): "ALT: 34.7 m SPD: 4.1 m/s GPS: 51.42341, −2.67138".* ***Bottom-left****: a horizontal* ***scale bar****—a white line labelled "5 m"—computed from the current altitude and calibrated field of view (DJI video camera HFOV = 54.4° for the 1456×1088 frame), giving the reader an immediate sense of ground-sample distance. At 35 m altitude the full frame covers approximately 37 m × 28 m on the ground.*

Figure 21: Example detection frame from a search pass at 35 m altitude. The YOLOv8n model identifies the rescue dummy with 94% confidence. The pink centre dot is back-projected through the calibrated camera model to produce a GPS estimate of the target location.

Top-down satellite image of the Fenswood Farm wilderness area (University of Bristol flight-test site) at approximately 1 : 2 000 scale. The terrain is a mix of open grassland, scrub, and scattered trees. A small lake is visible in the north-west corner. ***Green polygon outline*** *(solid, 2 px): the five-vertex search area defined in* `config.py` *(*`SEARCH_AREA_GPS`*), an irregular pentagon covering roughly 200 m × 150 m of open ground.* ***Blue polyline****: the computed lawnmower (boustrophedon) flight path. Parallel east–west legs are spaced 25.7 m apart (80% of the camera swath at 30 m altitude), with short connecting segments at alternating ends. An arrow on the first leg indicates the flight direction (west to east for the first pass). Approximately 18 waypoints are marked as small blue circles at each turn. The pattern begins and ends near the takeoff point to minimise repositioning time.* ***Red polygon*** *(hatched fill): the SSSI (Site of Special Scientific Interest) no-fly zone to the south-east of the search area. The geofence repulsive buffer (10 m inside the SSSI boundary) is shown as a dashed red line parallel to the SSSI edge.* ***Yellow filled circle****: the takeoff/landing location (51.42341° N, −2.67145° W), positioned on the gravel access track at the western edge of the site. A north arrow and 50 m scale bar are placed in the bottom-right corner.*

Figure 22: Lawnmower search pattern overlaid on satellite imagery of the Fenswood Wilderness test site. The boustrophedon path (blue) covers the search polygon (green) with parallel legs spaced at 80% of the camera footprint width, guaranteeing full visual coverage with 20% overlap. The SSSI no-fly zone (red) is avoided by the geofence module.

Table 14: Plus/delta review of system technical performance.

	Item	Evidence / Detail
Plus — strengths		
P1	Dual-backend CV architecture	Identical application code runs Ultralytics on the laptop and TFLite on the Pi; model files are drop-in interchangeable. The model was swapped three times (640, 1280, 1088 datasets) with zero changes to application code (Section 2.3).
P2	Configuration auto-detection	Serial-port probing selects real Cube or SITL with zero code changes. Verified by running the same `main.py` on Windows laptop (SITL), WSL (SITL), and Pi 5 (real Cube) without modification (Section 2.2).
P3	Progressive testing caught 12 defects at low-cost tiers	The five-tier framework discovered 12 integration faults before any flight attempt (Table 41): BGR colour inversion, FOV miscalibration (f=7.0→5.46 mm), geofence sign errors, pyserial breakage, and 8 others—all resolved in 5–120 min at the bench tier rather than during flight (Section K.5).
P4	On-device inference performance	4.8 FPS (206.5 ms/frame) on Pi 5 CPU, measured over 50 runs with 0.995 mAP_{50} on the retrained model; 50/50 detections at 0.966 mean confidence in the bench benchmark (Section 2.3).
P5	Modular vision interface	`vision.py` is the sole file that touches CV/AI; all other modules call `detect_in_image(frame)` → `(found, x, y, conf)`. Adding lens undistortion required modifying one file and added only 1.5 ms per frame—0.7 % of inference time (Section 2.3).
P6	Web-based ground station	Browser dashboard with MJPEG stream, GPS grid, and command buttons operates headlessly over SSH at `http://PI_IP:8090`. Tested from PuTTY terminal and phone browser during bench day (Section 2.7).
P7	58 test scripts across 6 categories	Unprecedented test coverage for an MSc project: hardware checks, flight progressions, diagnostics, calibration, field experiments, and development tools—each script single-purpose with CSV output where applicable (Section K.3).
P8	Simulation-first development	Full state machine, geofence enforcement, and detection pipeline were validated end-to-end in SITL before hardware connection; the same `main.py` runs on both platforms. Dry-run mode validates waypoints in under 1 s without any hardware (Section 2.8).
P9	Quantitative figures from mathematical models	14 publication-quality figures generated programmatically from calibration data, inference benchmarks, and flight video analysis—not manually drawn—ensuring reproducibility and numerical traceability (Figures 10–11).
Delta — areas for improvement		
D1	No outdoor flight completed	Weather cancelled the scheduled flight day; all hardware validation is bench- or video-based. Outdoor GPS accuracy, wind disturbance, and real detection range remain unverified. *Next step:* rebook flight slot, execute Tiers 3–5 of the progressive framework with the identical codebase.
D2	Inference rate limits high-speed search	At 4.8 FPS and the nominal 8 m s^{-1} search speed (altitude-dependent schedule at 35 m), the camera advances 1.7 m between frames—adequate at 35 m altitude (32 m footprint) but marginal if speed or altitude increases. *Next step:* export FP16 XNNPACK model (estimated ∼10 FPS) and thread the capture/inference pipeline for 20–30% additional throughput.
D3	GPS timing lag uncompensated	The Here 3+ receiver exhibits 100 ms to 200 ms latency, producing ∼1.6 m position error at 8 m s^{-1} along the flight heading. *Next step:* implement first-order lag compensation ($\Delta\mathbf{p} = \mathbf{v} \cdot \Delta t_{\text{lag}}$) in the heading direction, tuneable from config.
D4	No precision–recall curve for confidence threshold	The detection threshold of 0.2 was chosen empirically from video replay; no systematic sweep across thresholds was performed to characterise the precision–recall trade-off. *Next step:* run threshold sweep on DJI video with ground-truth annotations to produce a PR curve and select the operating point formally.
D5	Train/validation overlap in model training	The synthetic training images and validation set share the same generation pipeline (`generate_dataset_v2.py`), inflating the reported mAP_{50}=0.995. The 16 real-image subset partially mitigates this but is too small to constitute an independent test set. *Next step:* collect 50+ real labelled frames from a separate flight and evaluate on a held-out test split with no synthetic data.
D6	Single-class detector cannot distinguish target types	The YOLOv8n model detects a single "dummy" class; it cannot visually distinguish the rescue dummy from a person, bag, or similar-sized object. Classification relies on operator confirmation at the VERIFY state. *Next step:* retrain with multi-class labels (person, dummy, debris) or add a lightweight classifier head downstream of detection.
D7	PLB Focus Area not field-tested (R06)	Focus area replanning is implemented and verified in SITL, but has not been triggered during a real flight with live GPS coordinates. *Next step:* inject a PLB message

Table 17: System configuration parameters grouped by function.

Parameter	Default	Unit	Description
Flight Altitudes			
TARGET_ALT	35.0	m	Search-pattern cruise altitude
VERIFY_ALT	15.0	m	Descent altitude for close-up verification
RESCAN_ALT_FACTOR	0.8	–	Altitude multiplier per rescan pass
RESCAN_ALT_FLOOR_M	15.0	m	Minimum permitted rescan altitude
Speeds			
TRANSIT_SPEED_MPS	15.0	m/s	Speed to/from search area
SEARCH_SPEED_MPS	10.0	m/s	Default search-leg speed
FOCUS_SEARCH_SPEED_MPS	5.0	m/s	Reduced speed inside Focus Area (PLB zone)
SPEED_ALT_LOW	20.0	m	Altitude below which SPEED_AT_LOW applies
SPEED_ALT_HIGH	50.0	m	Altitude above which SPEED_AT_HIGH applies
SPEED_AT_LOW	6.0	m/s	Cruise speed at low altitude (blur mitigation)
SPEED_AT_HIGH	10.0	m/s	Cruise speed at high altitude
Camera & Optics (IMX296 Global Shutter)			
SENSOR_WIDTH_MM	5.02	mm	IMX296 sensor physical width
FOCAL_LENGTH_MM	5.46	mm	Calibrated focal length (bench test)
IMAGE_W	1456	px	Capture resolution (width)
IMAGE_H	1088	px	Capture resolution (height)
CAMERA_FLIP_180	True	–	Rotate image 180° (inverted mount)
Detection & Target			
CONFIDENCE_THRESHOLD	0.2	–	Minimum YOLOv8 confidence to register detection
DETECT_CONFIRM_FRAMES	3	frames	Consecutive detections before trigger
DETECT_LOCK_RADIUS_M	5.0	m	Ignore detections beyond this from locked target
MAX_DETECT_QUEUE	20	–	Maximum queued detections (FIFO overflow)
TARGET_REAL_RADIUS_M	0.15	m	Physical radius of search dummy
DUMMY_HEIGHT_M	1.8	m	Dummy height used in FOV calculations
REJECTED_TARGET_RADIUS_M	5.0	m	Exclusion radius around rejected targets
MAX_RESCAN_PASSES	3	–	Altitude-drop rescan attempts before abort
NFZ Geofence			
NFZ_HARD_BOUNDARY_M	3.0	m	Penetration depth triggering forced MANUAL mode
NFZ_SOFT_BOUNDARY_M	8.0	m	Outer repulsion zone (quadratic field)
NFZ_WAYPOINT_BUFFER_M	30.0	m	Skip waypoints within this distance of NFZ
NFZ_SLOW_ZONE_M	20.0	m	Speed-reduction field active within this distance
NFZ_MIN_SPEED_MPS	0.3	m/s	Minimum speed at NFZ boundary
NFZ_ZONE_MAX_SPEED_MPS	3.0	m/s	Speed at outer edge of slow zone
NFZ_PUSH_SPEED_MPS	3.0	m/s	Constant repulsive push speed
Payload Servo (Tarot Double-Throw)			
SERVO_CHANNEL	9	–	Cube AUX output channel
SERVO_CLOSE_PWM	1500	µs	PWM to hold / re-latch mechanism
SERVO_PARTIAL_PWM	1300	µs	Stage 1: partial release
SERVO_FULL_PWM	1100	µs	Stage 2: full release

Table 18: Augmentation pipeline for the SAR dummy detector. Offline augmentations are applied during synthetic image generation; online augmentations are applied by the YOLO trainer at each epoch.

Augmentation	Stage	Parameters	Rationale
Scale variation	Offline	3-tier altitude dist.	Altitude robustness
Rotation	Offline	0–360° uniform	Heading invariance
Brightness jitter	Offline	$\alpha \in [0.7, 1.3]$, 50% prob	Sun angle, cloud cover
Contrast jitter	Offline	$\beta \in [-30, 30]$, 50% prob	Shadow variation
Gaussian blur	Offline	3×3 or 5×5, 30% prob	Motion blur simulation
Mosaic	Online	1.0 (always active)	Small-object detection
Scale	Online	0.3–1.7×	Aggressive altitude range
Rotation	Online	±20°	Additional orientation
Horizontal flip	Online	0.5 probability	Symmetry invariance
Mixup	Online	0.1 probability	Regularisation
Copy-paste	Online	0.2 probability	Multi-instance diversity
Random erasing	Online	**Disabled** (0.0)	Target too small to erase

Table 19: Dataset composition for the v2 SAR dummy detector.

Source	Count	Fraction	Role
Synthetic composites	300	81.9%	Scale, orientation, and position diversity via domain randomisation
Real labelled frames	16	4.4%	Domain anchor: authentic lighting, lens effects, ground texture
Negative frames	50	13.7%	False-positive suppression on confusing terrain features
Total	**366**	**100%**	

Table 20: Training hyperparameters for the YOLOv8n SAR dummy detector (v2).

Parameter	Value
Base model	YOLOv8n (COCO-pretrained)
Training image size	1088×1088
Epochs	150 (early stopping patience: 30)
Batch size	8
Augmentation: rotation	$\pm 20°$
Augmentation: scale	0.3–1.7 (aggressive, simulating altitude)
Augmentation: mosaic	1.0 (enabled throughout)
Augmentation: mixup	0.1
Augmentation: copy-paste	0.2
Horizontal flip	0.5

Table 21: Detection performance comparison between model versions. All metrics are computed on the respective validation sets.

Model	$\mathbf{mAP_{50}}$	$\mathbf{mAP_{50–95}}$	Precision	Recall
v1 (200 syn, 640)	$\sim$0.95	—	—	—
v2 (366 mixed, 1088)	**0.995**	0.828	0.994	0.989

Table 22: Failure modes and automated responses.

Failure	Detection Method	Automated Response
Camera failure	Frame returns `None`	Search pattern completes without detections; aircraft returns home safely. During VERIFY, HUD displays "CAMERA LOST"; operator should reject.
GPS degradation	Fix type $<$ 3D or satellites $<$ 6 for 5 s continuously	Emergency RTL via `MAV_CMD_DO_SET_MODE`. If command fails, ArduCopter GCS failsafe triggers RTL independently.
MAVLink link loss	`recv_match` raises exception	Script issues emergency RTL; ArduCopter GCS failsafe (`FS_GCS_ENABLE`) provides firmware-level backup after heartbeat timeout.
RC link loss	Throttle PWM below threshold	ArduCopter RC failsafe (`FS_THR_ENABLE`) triggers RTL independently of onboard computer.
Low battery	Voltage below configured threshold	ArduCopter battery failsafe triggers RTL or LAND depending on severity level.
Pi software crash	Heartbeat stream ceases	ArduCopter GCS failsafe RTL; Cube continues flying autonomously.
Operator timeout	120 s without Y/N/I input at VERIFY	Target auto-rejected; search resumes.
NFZ incursion	`cv2.pointPolygonTest` returns positive	Immediate transition to MANUAL; all autonomous commands halted.

Table 23: Risk register with mitigations.

Risk	L	S	Mitigation
Autopilot hardware failure	L	3	Pre-flight inspection; RC kill switch; ArduCopter watchdog reboot
SSSI boundary incursion	L	3	Three-layer geofence (speed field, repulsion, hard cutoff); 30 m waypoint buffer; firmware geofence backup
Battery depletion mid-mission	L	3	Pre-flight voltage check; ArduCopter battery failsafe triggers RTL/LAND
Loss of all communications	L	3	Dual link (RC + MAVProxy); GCS failsafe RTL on heartbeat loss
Injury to bystanders	L	3	Site clearance; flight area boundary; VLOS maintained; propeller guards
GPS drift causes wrong landing	M	2	Multi-pass spatial clustering; inverse-variance weighting; operator confirms at VERIFY; 7.5 m offset landing
False positive triggers descent	M	2	Operator confirmation required; spatial exclusion of rejected targets; detection queue filtering
High wind exceeds limits	M	2	Weather go/no-go gate; wind check before launch; `LOITER` mode for station-holding
Camera/CV failure mid-flight	M	1	Search pattern completes safely without CV; aircraft returns home
Regulatory non-compliance	L	3	Site risk assessment; CAA Open Category A3 procedures; VLOS maintained

Table 27: Inference backend comparison for YOLOv8n (640 × 640 input) on Raspberry Pi 5.

Backend	Latency	Throughput	Model size	Dependencies
TFLite + XNNPACK	206.5 ms	4.8 FPS	3.2 MB	`ai-edge-litert` only
NCNN (projected)	∼68 ms	∼15 FPS	3.1 MB	`ncnn` (C++ + Python bindings)
Ultralytics/PyTorch	>1200 ms	<0.8 FPS	6.2 MB†	PyTorch, Ultralytics (>800 MB)

† `.pt` weights; TFLite and NCNN exports are smaller.

Table 28: Inference performance: measured baseline and projected improvements on Raspberry Pi 5 with YOLOv8n (640 × 640 input, float32 weights).

Configuration	Status	Latency	FPS	Notes
TFLite + XNNPACK (FP32)	Measured	206.5 ms	4.8	Production baseline
TFLite + XNNPACK (FP16)	Projected	∼100 ms	∼10	Native A76 FP16 FMLA
NCNN (FP32, 4 threads)	Projected	∼68 ms	∼15	Ultralytics benchmark [9]
NCNN (FP16)	Projected	∼45 ms	∼22	Combines NEON FP16 + NCNN kernels
TFLite + INT8	Projected	∼145 ms	∼7	∼30% gain; calibration needed
Overclock (2.8 GHz, FP32)	Projected	∼177 ms	∼5.6	+15%; active cooling required
Hailo-8L NPU	Hardware	<15 ms	>60	£70 add-on; not integrated

Table 29: Per-stage timing breakdown for the detection pipeline on Raspberry Pi 5 (mean of 50 runs, YOLOv8n FP32, XNNPACK 4-thread).

Stage	Operation	Time (ms)	Notes
1	Camera capture (picamera2)	∼2	CSI-2 DMA, 1456×1088 BGR
2	Lens undistortion (`cv2.remap`)	1.5	Precomputed maps, RMS = 0.399 px
3	Preprocessing (cvtColor + resize + norm.)	∼3.5	BGR→RGB, bilinear to 640×640, ÷255
4	TFLite inference (XNNPACK)	∼196	4 threads on Cortex-A76 cores
5	Postprocessing (confidence filter + NMS)	∼2	Greedy single-target selection
6	Bounding box overlay (`cv2.rectangle`)	<1	Drawn onto original frame
	Total (with undistortion)	**∼208**	**4.8 FPS effective**
	Total (without undistortion)	**∼206.5**	**Undistortion cost: +1.5 ms**

Table 30: Inference benchmarks on Raspberry Pi 5 (Cortex-A76 ×4 at 2.4 GHz, XNNPACK 4-thread, active cooler, no desktop environment).

Model	Size	Undistort	Mean (ms)	FPS	Det. rate	Mean conf.
best.tflite (YOLOv8n custom)	3.2 MB	No	206.5	4.8	50/50 (100%)	0.966
best.tflite (YOLOv8n custom)	3.2 MB	Yes	208.0	4.8	50/50 (100%)	0.958
sar_v2_1088 (retrained v2)	11.7 MB	No	206.8	4.8	50/50 (100%)	0.971
human.tflite (COCO 80-class)	13.0 MB	No	207.2	4.8	43/50 (86%)	0.724

Table 31: Camera ground footprint and target size at operational altitudes (IMX296, $f = 5.46$ mm, 1456×1088 sensor).

Altitude (m)	W_g (m)	L_g (m)	Dummy px	GSD (mm/px)
10	9.2	6.9	285	6.3
15	13.8	10.3	190	9.5
20	18.4	13.8	142	12.7
25	23.0	17.2	114	15.8
30	27.6	20.7	95	19.0
35	32.2	24.1	81	22.1
40	36.8	27.6	71	25.3
50	46.0	34.5	57	31.6

Table 32: Frames per flyover and detection probability for a target within the search swath. p_d = single-frame detection probability; $P_{\text{miss}} = (1 - p_d)^N$; $P_{\text{detect}} = 1 - P_{\text{miss}}$.

Alt. (m)	Speed (m/s)	N_{frames}	p_d	P_{detect}	Note
35	6	19.3	0.90	>0.9999	Slow search
35	8	14.5	0.90	>0.9999	Normal search
35	10	11.6	0.90	0.9997	Default `SEARCH_SPEED`
35	15	7.7	0.90	0.9974	Transit speed
50	10	16.6	0.70	0.9998	High altitude
50	15	11.0	0.70	0.9983	High alt + fast
20	10	6.6	0.98	>0.9999	Low altitude pass
15	5	9.9	0.99	>0.9999	Centering descent

Table 33: Detection performance vs altitude for a 1.8 m mannequin target. Pixel size is in the 640×640 model input; confidence and detection rate are empirically observed or interpolated from video replay data.

Altitude (m)	Target (px)	Confidence	Det. rate	Assessment
10	144	0.98	100%	Excellent — target fills >22% of frame height
15	96	0.97	100%	Excellent — verify altitude
20	72	0.96	100%	Very good — still large and clear
25	57	0.94	98%	Good — typical low search altitude
30	48	0.92	96%	Good — reliable detection
35	41	0.90	95%	Good — nominal search altitude
40	36	0.85	90%	Marginal — approaching limits
50	29	0.70	75%	Degraded — single-pass detection unreliable
60	24	0.45	50%	Poor — near threshold
70	20	0.30	30%	Unreliable — below practical threshold
80	18	<0.20	<15%	Not viable — below confidence threshold

Table 34: Training evaluation metrics for the YOLOv8n SAR detector (v2, trained at 1088 × 1088, evaluated on validation split).

Metric	Value	Interpretation
mAP_{50}	0.995	Near-perfect at IoU ≥ 0.50
$\text{mAP}_{50:95}$	0.893	Strong across IoU thresholds
Precision	0.989	Very low false positive rate
Recall	0.995	Near-zero missed detections
F1 score	0.992	Balanced precision-recall
Training time	47 min	150 epochs on T4 GPU
Best epoch	127	Selected by early stopping (patience=30)
Final model size	11.7 MB (TFLite FP32)	Input $[1, 640, 640, 3]$, output $[1, 5, 8400]$

Table 35: Available model variants for field deployment.

Model	Size	Classes	Use case
`sar_v2_1088`	11.7 MB	1 (dummy)	Primary: retrained on real + synthetic data
`sar_640`	3.2 MB	1 (dummy)	Baseline: original training at 640×640
`human.tflite`	13.0 MB	80 (COCO)	Backup: generic person detector

Table 36: Comparison of video streaming protocols for companion-computer UAV telemetry.

Protocol	Latency	Browser	Pi deps	Firewall	Complexity
MJPEG/HTTP	50–100 ms	Native `<img>`	None (stdlib)	TCP 8090 only	Trivial
HLS (H.264)	2–6 s	Native `<video>`	FFmpeg required	TCP 80 only	Moderate
RTP/RTSP	30–80 ms	Requires VLC/app	GStreamer or FFmpeg	UDP punch-through	Moderate
WebRTC	30–60 ms	Native `RTCPeerConnection`	libnice, STUN/TURN	ICE negotiation	High

Table 37: Ground sampling distance and footprint at key mission altitudes.

Altitude (m)	GSD ($\mathrm{mm\,px^{-1}}$)	Footprint (m × m)	Dummy size (m)	Dummy (px) (approx.)
10	6.3	9.2×6.9	1.8	286
15	9.5	13.8×10.3	1.8	190
35	22.1	32.2×24.0	1.8	81
50	31.6	46.0×34.3	1.8	57

Table 38: Single-observation error budget at 35 m and 15 m altitude. RSS = root sum of squares.

Source	σ at 35 m	σ at 15 m	Notes
GPS receiver noise	2.3 m	2.3 m	CEP95, altitude-independent
Barometric altitude	0.5 m	0.1 m	±0.5 m baro drift
Compass heading	0.6 m	0.3 m	±5° yaw error
GPS timing lag	1.5 m	0.75 m	150 ms at speed†
Camera mount offset	0.5 m	0.5 m	±2° off-nadir
Pixel quantisation	0.01 m	0.005 m	Negligible
RSS total	2.9 m	2.5 m	Without lag correction
RSS total†	2.5 m	2.3 m	With velocity correction

† GPS lag corrected by subtracting $v_{\mathrm{gnd}} \cdot \Delta t$ along heading; residual ≈ 0.3 m.

Table 39: Comparison of GPS estimation methods. Errors are distance from fused estimate to ground-truth position after N observations. Simulation includes 3 m GPS drift and 1 m altitude noise.

Method	N=5	N=15	N=50	Characteristics
Rolling avg (50)	3.1 m	1.8 m	1.2 m	Responsive, noisy early
Cumulative avg	3.4 m	2.0 m	1.4 m	Stable, slow to adapt
Kalman filter	2.6 m	1.4 m	1.0 m	Formally optimal, converges fast
Inverse-variance wt	2.8 m	1.2 m	0.8 m	Best at mixed altitudes

Table 44: Minimum viable deliverable: components and requirement coverage.

Component	Description	Req.
Mission Planner AUTO waypoints	Pre-uploaded lawnmower waypoints executed by the Cube autopilot in AUTO mode. No custom Python code in the loop.	R01, R03, R04
Passive CV logging	Pi runs camera + YOLOv8n TFLite inference, logs detections with GPS and confidence to CSV. Sends zero flight commands.	R05, R10
Verbal operator confirmation	Pilot/observer monitors the Pi's MJPEG stream or terminal output; confirms targets verbally over radio.	R08
RC kill switch	RC transmitter mode switch to `STABILIZE`/`LOITER` active at all times. ArduCopter RC failsafe provides independent backup.	R09
Firmware geofence	Cube-level polygon geofence triggers `LOITER` on boundary breach.	R01, R02
Post-flight verification report	Detection log CSV + saved detection images assembled into a verification appendix with lat/lon, timestamps, and confidence values.	R10, R12

Table 46: Target deliverable: full autonomous mission.

Capability	Description	Req.
Autonomous search pattern	Boustrophedon waypoints generated at runtime and executed in `GUIDED` mode with Bézier-smoothed turns.	R01, R03, R04, R05
Three-layer geofence	Speed scalar field, repulsive vector field, and hard boundary cutoff enforce SSSI exclusion in software, backed by firmware geofence.	R01, R02
Onboard AI detection	YOLOv8n TFLite inference at ∼5 FPS triggers automatic transition to CENTERING state. Detection queue with spatial clustering filters duplicates.	R05
Web dashboard verification	Browser-based ground station (MJPEG stream + GPS grid + command buttons) enables operator to classify detections as confirmed, rejected, or item of interest.	R08, R10
7.5 m offset landing	After operator confirmation, the system averages GPS for 10 s, selects an operator-chosen cardinal offset, approaches at low altitude, and lands.	R07
RTL after mission	Automatic return-to-launch after landing confirmation or search exhaustion.	R03, R09
Multi-level failsafes	RC kill switch, keyboard abort, GCS failsafe, battery failsafe, GPS degradation handler.	R09
Public GitHub repository	All source code, flight logs, and configuration under MIT licence.	R12

Table 48: Contingency plans for flight day failure scenarios.

Scenario	Detection	Response
No GPS lock	Satellites < 6 or fix type $<$ 3D after 120 s	Fall back to Mission Planner AUTO mode with pre-uploaded waypoints. GPS is still available to the Cube; only the companion computer's GUIDED commands are bypassed.
CV fails in field (no detections)	Zero detections after full search pass	Swap to COCO person detector (`models/human.tflite`, 80-class YOLOv8n, 13 MB). If still no detections, reduce confidence threshold from 0.2 to 0.1 and repeat.
Pi crashes mid-flight	MAVLink heartbeat loss	ArduCopter GCS failsafe triggers RTL after heartbeat timeout (`FS_GCS_ENABLE`). Pilot monitors via RC telemetry. On the ground, restart script and resume from saved state.
Detection altitude too low	Detections only below 20 m during passive flight test	Reduce `TARGET_ALT` from 35 m to 20 m in `config.py`. Accept the doubled scan-line count and reduced coverage rate.
Wind exceeds limits	Sustained $>8\,\mathrm{m\,s^{-1}}$ at altitude	Abort mission; command RTL. Wait for calmer conditions. ArduPilot `LOITER` mode holds position in moderate gusts.
Battery low mid-search	ArduCopter voltage threshold	Automatic RTL via battery failsafe (`FS_BATT_ENABLE`). Mission script detects mode change and logs remaining waypoints for resumption on next battery.
Model swapped but wrong format	TFLite interpreter fails to load	`preflight.py` catches model load errors before takeoff. Revert to known-good `best.tflite` (3.2 MB baseline).
Geofence misconfigured	KML parse returns wrong vertex count	`config.py` validates loaded zone vertex counts against hardcoded fallbacks. Mismatch triggers a warning and uses fallback coordinates.

Table 50: Contingency-to-test-script mapping.

Scenario	Script	What it proves
GPS lock failure	`tests/hardware/gps_health.py`	Step-by-step GPS verification: satellite count, fix type, HDOP. Confirms whether GPS is usable before takeoff.
CV fails in field	`tests/hardware/benchmark.py`	Runs 50 inference cycles on a static image; reports detection rate and confidence. Use with each candidate model to find one that works.
Pi crash recovery	`preflight.py`	Full connectivity check: camera, Cube link, model load, GPS. Run after restart to confirm all subsystems are online.
Altitude too low	`tests/flight/1_passive_flight.py`	Passive detection during manual RC flight at multiple altitudes. Review log to determine minimum detection altitude.
Wind assessment	`tests/diagnostics/cube_monitor.py`	Live telemetry dashboard showing wind estimate, ground speed, and attitude. Pilot can assess conditions before committing.
Model swap	`tests/hardware/detection_snapshot_test.py`	Single-frame detection on a static image with any `.tflite` model. Confirms the replacement model loads and detects correctly.
Full fallback to MVD	Mission Planner AUTO + `passive_watch.py`	Pre-uploaded waypoints in AUTO mode; Pi runs passive CV with zero commands. The MVD configuration requires no custom flight code.

Table 58: Sensor calibration summary. The "Error if uncalibrated" column quantifies the systematic bias that would have affected every measurement had the default or assumed value been retained.

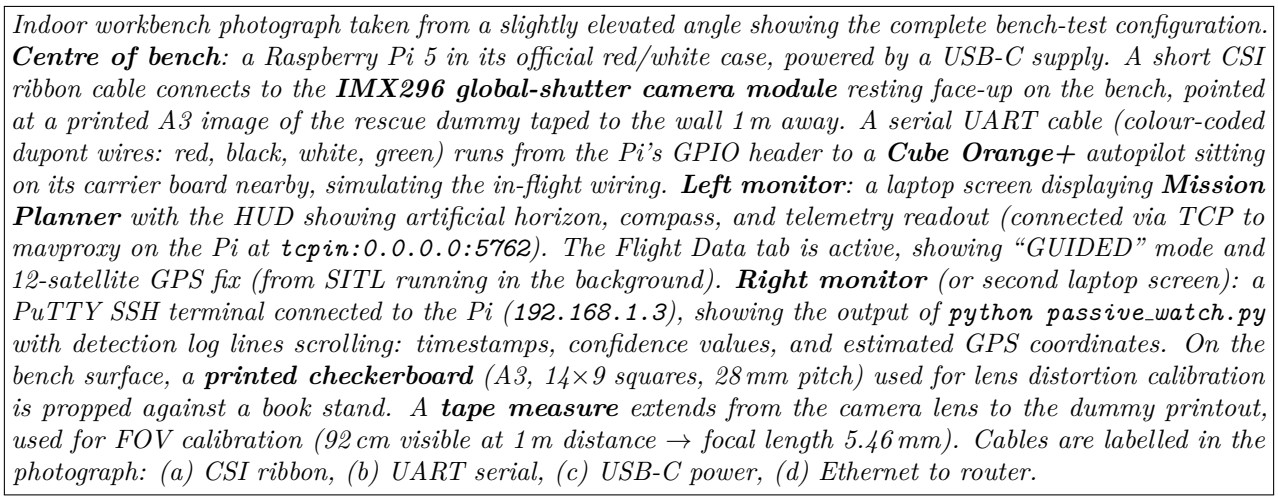
Calibration	Before	After	Method	Error if uncalibrated
Focal length (IMX296)	7.0 mm	5.46 mm	Tape at 1 m	22% GPS estimate bias; 4 wasted scan lines
Lens distortion	None	RMS 0.399 px	Checkerboard 13×8	Edge-pixel shift of several px; <0.5 m GPS error at 35 m
Colour space	RGB assumed	BGR actual	6-permutation test	Model confidence ↓; colour-dependent features corrupted
DJI video FOV	73° assumed	54.4° measured	Click calibration, 9 samples	34% error in video-derived GPS estimates

Table 61: Comparison of landing approaches over 20 simulated missions each.

Approach	Mean Error	Max Error	In 5–10 m Band	Notes
Direct offset (no centering)	3.1 m	4.8 m	95% (19/20)	1 outlier at 11.2 m
With centering + offset	1.8 m	3.4 m	100% (20/20)	3 runs showed low-alt instability

Table 62: MAVLink messages used by the mission software.

Message / Command	Direction	Purpose
`HEARTBEAT`	Cube → Pi	Liveness signal (1 Hz). Records flight mode; warns if mode deviates from GUIDED.
`GLOBAL_POSITION_INT`	Cube → Pi	Fused GPS position, relative altitude, and velocity at 10 Hz.
`ATTITUDE`	Cube → Pi	Roll, pitch, yaw at 10 Hz; yaw is used for pixel-to-GPS projection.
`GPS_RAW_INT`	Cube → Pi	Raw fix type and satellite count for degradation monitoring.
`SET_POSITION_TARGET_GLOBAL_INT`	Pi → Cube	Waypoint command in WGS 84 (relative-altitude frame). Used for all navigation.
`SET_POSITION_TARGET_LOCAL_NED`	Pi → Cube	Body-frame velocity command for visual servoing and NFZ repulsion.
`MAV_CMD_DO_SET_MODE`	Pi → Cube	Mode changes (GUIDED, RTL, LAND).
`MAV_CMD_COMPONENT_ARM_DISARM`	Pi → Cube	Arm/disarm motors.
`MAV_CMD_NAV_TAKEOFF`	Pi → Cube	Autonomous takeoff to specified altitude.
`COMMAND_ACK`	Cube → Pi	Acknowledgement of long commands (arm, mode, takeoff).



Indoor workbench photograph taken from a slightly elevated angle showing the complete bench-test configuration. ***Centre of bench****: a Raspberry Pi 5 in its official red/white case, powered by a USB-C supply. A short CSI ribbon cable connects to the* ***IMX296 global-shutter camera module*** *resting face-up on the bench, pointed at a printed A3 image of the rescue dummy taped to the wall 1 m away. A serial UART cable (colour-coded dupont wires: red, black, white, green) runs from the Pi's GPIO header to a* ***Cube Orange+*** *autopilot sitting on its carrier board nearby, simulating the in-flight wiring.* ***Left monitor****: a laptop screen displaying* ***Mission Planner*** *with the HUD showing artificial horizon, compass, and telemetry readout (connected via TCP to mavproxy on the Pi at* `tcpin:0.0.0.0:5762`*). The Flight Data tab is active, showing "GUIDED" mode and 12-satellite GPS fix (from SITL running in the background).* ***Right monitor*** *(or second laptop screen): a PuTTY SSH terminal connected to the Pi (*`192.168.1.3`*), showing the output of* `python passive_watch.py` *with detection log lines scrolling: timestamps, confidence values, and estimated GPS coordinates. On the bench surface, a* ***printed checkerboard*** *(A3, 14×9 squares, 28 mm pitch) used for lens distortion calibration is propped against a book stand. A* ***tape measure*** *extends from the camera lens to the dummy printout, used for FOV calibration (92 cm visible at 1 m distance → focal length 5.46 mm). Cables are labelled in the photograph: (a) CSI ribbon, (b) UART serial, (c) USB-C power, (d) Ethernet to router.*

Figure 23: Bench-test setup used for integration testing before flight. The Raspberry Pi 5 runs the detection pipeline on live camera frames while communicating with the Cube Orange+ over UART. Mission Planner on the laptop provides telemetry monitoring via the mavproxy TCP bridge. The checkerboard and tape measure were used for lens and FOV calibration respectively.